CSE 4345 Big Data Analytics

Final exam guide · Weeks 5–10 · 13 October 2026

WHERE QUESTIONS COME FROM
The teacher said the end-of-chapter pools. All 126 pool questions are here, answered at exam length.

HOW TO STUDY
1. Read a chapter's pool. 2. Learn its 6 topic cards. 3. Drill a card in every format. 4. Sit the mock. 5. Revise from the sheet.

COLOURS
yellow = the answer · red = traps · blue = maths

Chapters

WEEK 5
YARN, file formats and compression
19 pool questions · 6 topics

WEEK 6
Apache Spark: RDDs and in-memory computing
22 pool questions · 6 topics

WEEK 7
Hive, HiveQL and SQL-on-Hadoop
21 pool questions · 6 topics

WEEK 8
Data integration: Kafka, Lambda/Kappa, data lakes
23 pool questions · 6 topics

WEEK 9
Data visualization
19 pool questions · 6 topics

WEEK 10
Big data in real applications
22 pool questions · 6 topics

CHAPTER 1 · WEEK 5

YARN, file formats and compression

How a shared Hadoop cluster hands out CPU and RAM to many jobs (YARN), and how the way data is laid out and compressed on disk decides how much of it a query has to read. Part 2 tests the same ideas on LinkedIn's 5,000-node YARN cluster.

Exam question pool

19 questions from the slides · click a question for its answer

MCQ **Q1.** Which component in YARN is responsible for global resource allocation across the entire cluster?

(a) NodeManager
(b) ApplicationMaster
(c) ResourceManager
(d) NameNode

(c) ResourceManager
One RM per cluster allocates containers globally; the AM manages only its own job.

MCQ **Q2.** Which Hadoop file format stores data in columnar format and is best suited for analytical, read-heavy workloads?

(a) Avro
(b) Text/CSV
(c) Parquet
(d) JSON

(c) Parquet
Columnar, with min/max stats for predicate pushdown; Avro, CSV and JSON are row formats. ORC is also columnar: the instructor accepts it as partly correct with a justification (best for Hive), but Parquet is the broader standard.

MCQ **Q3.** What does “splittable” mean for a file format in Hadoop?

(a) The file can be deleted in parts by HDFS
(b) Multiple Mappers can process different parts of the file simultaneously
(c) The file format supports splitting schema across multiple NameNodes
(d) The file can be distributed across multiple data centres

(b) Multiple Mappers can process different parts of the file simultaneously
Reading can start anywhere in the file, which also allows data-local tasks.

MCQ **Q4.** YARN was introduced in which major Hadoop version, and what did it replace?

(a) Hadoop 1.0; replaced the NameNode
(b) Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster
(c) Hadoop 3.0; replaced DataNodes with Containers
(d) Hadoop 2.0; replaced the Secondary NameNode with a Standby NameNode

(b) Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster
(d) is the trap: the Standby NameNode is HDFS, not YARN.

TRUE/FALSE **1.** Parquet is a row-oriented file format optimised for write-heavy workloads.

False. Parquet is column-oriented and optimised for read-heavy analytics; the row-oriented, append-friendly format is Avro.

TRUE/FALSE **2.** YARN allows multiple computation frameworks such as Spark and Tez to share resources on the same Hadoop cluster.

True. Each framework brings its own ApplicationMaster, so YARN runs MapReduce, Spark and Tez on one cluster.

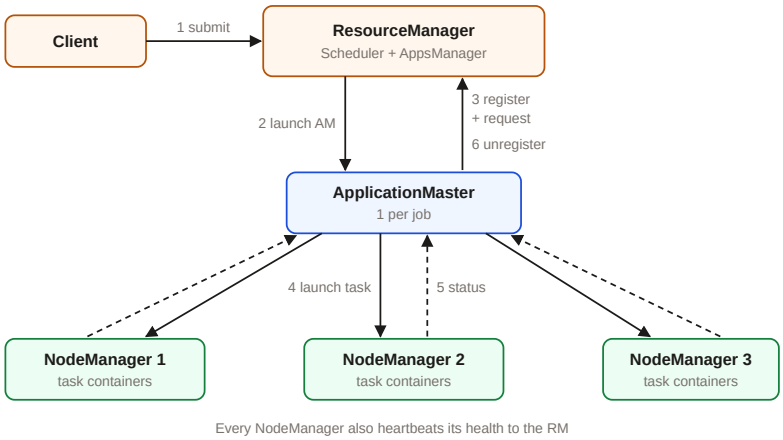
TRUE/FALSE **3.** Gzip-compressed files are natively splittable in Hadoop, allowing multiple Mappers to process them in parallel.

False. Gzip files are not splittable, so one Mapper reads the whole file; use bzip2 or Parquet + Snappy for parallel reads.

TRUE/FALSE **4.** ORC format is specifically optimised for Hive workloads and achieves among the highest compression ratios of any Hadoop format.

True. ORC is columnar, best for Hive, with the highest compression, Hive ACID and vectorized reads.

DESCRIPTIVE **D1.** Explain YARN's three-component architecture — ResourceManager, NodeManager, and ApplicationMaster. Draw a diagram showing how they interact when a Spark job is submitted to the cluster.



A job on YARN: the client submits to the ResourceManager, which starts the job's ApplicationMaster; the AM requests containers and has NodeManagers launch tasks, which report status back.

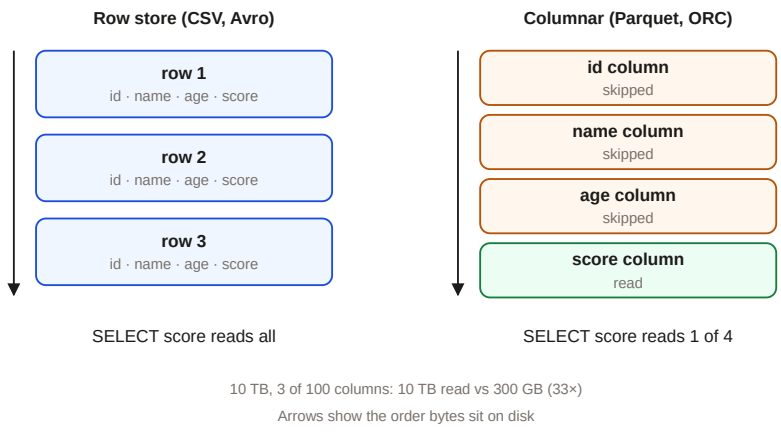
ResourceManager (1 per cluster): Scheduler grants containers; ApplicationsManager accepts jobs and starts each AM.
NodeManager (1 per node): launches and monitors containers, heartbeats to the RM, enforces CPU/RAM limits.
ApplicationMaster (1 per job): requests containers, has NMs launch tasks, retries failures, releases containers.

Diagram (see the diagram in the YARN and its three components card): Client, RM, Spark AM, NM1–NM3, with numbered arrows:

1. Client submits the job to the RM.
2. RM launches the Spark AM in a container.
3. AM registers with the RM and requests containers; the RM Scheduler grants them.
4. AM tells NM1–NM3 to launch executors.
5. Executors report status to the AM.
6. AM unregisters from the RM when done.

Example: the Spark AM asks for 100 containers {4 cores, 8 GB}; the busy RM grants 80 and the AM runs executors in them.

DESCRIPTIVE **D2.** What is columnar storage? Why does it improve performance for analytics queries that select only a subset of columns? Use a concrete example with numbers to illustrate the I/O reduction.



A row store keeps whole records together, so reading one column reads them all; a columnar store keeps each column together and skips the rest.

Columnar storage keeps all values of one column together on disk (Parquet, ORC); a row store (CSV, Avro) writes whole records one after another.

Why faster: the query reads only the columns it names; Parquet's min/max stats skip whole row groups (predicate pushdown); same-type values compress better.

Example: 100 columns, 10 TB, query on 3 columns.

Format	Data read
CSV	10 TB
Parquet	10 × 3/100 = 0.3 TB = 300 GB

I/O cut 97%, about **33×** faster.

DESCRIPTIVE **D3.** Compare Avro and Parquet. Explain what “schema evolution” means and why Avro handles it better. In which scenarios would you use each?

	Avro	Parquet
Layout	row	columnar
Strength	schema evolution, appends	analytic reads, pushdown
Codec	Snappy/Deflate	Snappy/Gzip

Schema evolution: the schema changes over time (e.g. a new field) while old files and readers keep working.

Why Avro handles it: the writer schema is stored with the data, the reader supplies its own schema, fields match by name; missing fields get defaults, unknown fields are ignored. Backward and forward compatible.

Use: Avro for Kafka → HDFS ingest and data exchange; Parquet for Spark, Presto and ML analytics.

ANALYTICAL **A1.** Data Lake Performance Migration: A Dhaka-based telecom company stores 10 TB of daily call-detail records (CDR) on HDFS as gzip-compressed CSV files. Their data engineers report that a Hive query selecting 4 of 80 columns takes 6 hours. – Identify two specific reasons why this configuration is slow – Propose a migration strategy: new format, compression codec, and partitioning scheme – Estimate the expected I/O reduction (show your calculation) – What YARN scheduler would you use and why, given that multiple teams share the cluster?

i) Why slow

1. Gzip is not splittable: one Mapper per file, the cluster idles.
2. CSV is a row store: reading 4 columns parses all 80; no stats to skip data.

ii) Migration: ORC (Hive) or Parquet (if Spark reads it too); Snappy for hot data (Zlib for ORC in Hive, Zstd for archive); partition by date and region, e.g. `dt=YYYY-MM-DD/region=` ; convert once, then nightly.

iii) I/O estimate

Fraction read	4/80 = 0.05
Data read	10 TB × 0.05 = 500 GB
Reduction	**95%** (20× less)
Time if I/O-bound	360 min × 0.05 ≈ **18 min**

iv) Scheduler: Capacity: guaranteed queues per team (ETL, analysts, data science), idle capacity borrowed, so no team starves another. FIFO lets one long job block all.

ANALYTICAL **A2.** YARN Failure Trace: A NodeManager (NM2) hosting 5 running containers for a MapReduce job fails mid-execution. Trace the complete failure recovery sequence: which components detect the failure, in what order, what state changes occur in the ResourceManager, and how are the 5 containers’ tasks restarted.

1. NM2 crashes; its 5 containers die; heartbeats stop.

2. **RM** detects the missed heartbeats after the timeout (>10 min) and declares NM2 lost.
3. **RM state:** NM2 removed from schedulable capacity; its 5 containers marked **failed**.
4. RM notifies the MapReduce **AM**.
5. AM requests 5 replacement containers.
6. RM grants them on healthy nodes (e.g. NM1, NM3), preferring nodes with HDFS replicas of the input.
7. AM has those NMs relaunch the 5 tasks from their input splits; finished maps whose output was on NM2's disk may rerun.

If the AM was on NM2, the RM restarts it (default 2 attempts); MapReduce recovers state from HDFS checkpoints.

DISCUSSION **L1.** Resource consolidation increases utilisation: merging 3 siloed clusters into one raised average utilisation from 45% to 78% — the same capacity served 73% more workload

Claim: one shared cluster uses hardware better than three silos.

For: LinkedIn's silos (MapReduce, Spark, Samza) were idle at different times; merged, utilisation rose 45% → 78%, and 78/45 = 1.73, so 73% more work on the same hardware. No cross-cluster copying, one config instead of three.

Against: workloads now compete, so queues, pre-emption and RM high availability become essential.

DISCUSSION **L2.** Capacity Scheduler enables SLAs: production queues guaranteed minimum resources; analyst adhoc queries cannot starve revenue-generating pipelines

Claim: guaranteed queues protect SLAs (promised finish times).

For: production holds 40% at LinkedIn, so the 10% adhoc queue cannot delay revenue pipelines, while idle capacity is still lent out.

Against: fixed percentages need tuning, and analysts may wait at busy times; Fair gives short jobs faster turnaround.

DISCUSSION **L3.** Pre-emption needs tuning: aggressive pre-emption killed long-running Spark jobs; LinkedIn set 30-second grace periods and added Spark checkpoint-on-preempt hooks

Claim: pre-emption is needed but must match the workload.

For: without reclaiming borrowed containers, queue guarantees are empty promises.

Against: killing a container wastes its work; long Spark jobs restarted repeatedly. LinkedIn's fix: a 30-second grace period and checkpoint-on-preempt hooks so jobs resume.

DISCUSSION **L4.** YARN is the OS; frameworks are processes: the paradigm shift — the cluster is the computer, YARN is its kernel

Claim: YARN is to a cluster what an OS kernel is to a computer.

For: the RM allocates CPU and RAM as containers, NMs isolate them with cgroups, and frameworks are just applications with their own AM; the same three-tier design lives on in Kubernetes.

Against: YARN sizes containers only by CPU and RAM, relies on HDFS for storage, and Kubernetes has replaced it for most new deployments.

DISCUSSION **L5.** Samza on YARN: stream processing is just another set of long-running containers; YARN's fault tolerance restarts failed Samza tasks

Claim: a stream job is just long-running containers.

For: Samza shares the cluster in its 15% queue, no separate stream cluster, and YARN restarts failed tasks on other nodes.

Against: the containers hold capacity permanently, and the >10 min NodeManager timeout is slow for real-time data, so Samza must resume from the right Kafka offset itself.

LIVE EXERCISE

LE. Task (20 minutes, pairs) Model LinkedIn's 5,000-node YARN cluster in Python. Given: • 5,000 nodes; each: 64 GB RAM, 16 vCPUs • Capacity Scheduler: production 40%, spark 25%, samza 15%, adhoc 10%, reserve 10% • Container sizes: Hive mapper: {4 GB, 2 vCPU}; Spark executor: {8 GB, 4 vCPU}; Samza task: {2 GB, 1 vCPU} Questions: 1 Max concurrent containers per queue? 2 Which resource is the bottleneck (CPU or RAM)? 3 If production needs 10,000 Hive mappers, how many nodes are required?

```
NODES    = 5_000
RAM_GB   = 64
VCPUS    = 16

queues = {
  "production": 0.40,
  "spark":      0.25,
  "samza":      0.15,
  "adhoc":      0.10,
  "reserve":    0.10,
}
containers = {
  "production": {"ram": 4, "cpu": 2},
  "spark":      {"ram": 8, "cpu": 4},
  "samza":      {"ram": 2, "cpu": 1},
  "adhoc":      {"ram": 4, "cpu": 2},
}

for q, frac in queues.items():
    nodes_q = NODES * frac
    ram_q    = nodes_q * RAM_GB
    cpu_q    = nodes_q * VCPUS
    if q in containers:
        c = containers[q]
        by_ram = ram_q // c["ram"]
        by_cpu = cpu_q // c["cpu"]
        max_c = int(min(by_ram, by_cpu))
        bottleneck = ("RAM" if by_ram < by_cpu
                            else "CPU")
        print(f"{q}: {max_c} containers "
              f"(bottleneck: {bottleneck})")
```

Task: run the skeleton; add Q3:

```
import math
c = containers["production"]
per_node = min(RAM_GB // c["ram"], VCPUS // c["cpu"])
print(per_node, math.ceil(10_000 / per_node))
```

Expected output:

```
production: 16000 containers (bottleneck: CPU)
spark: 5000 containers (bottleneck: CPU)
samza: 12000 containers (bottleneck: CPU)
adhoc: 4000 containers (bottleneck: CPU)
8 1250
```

1. production 16,000; spark 5,000; samza 12,000; adhoc 4,000; reserve has no container size, so none.

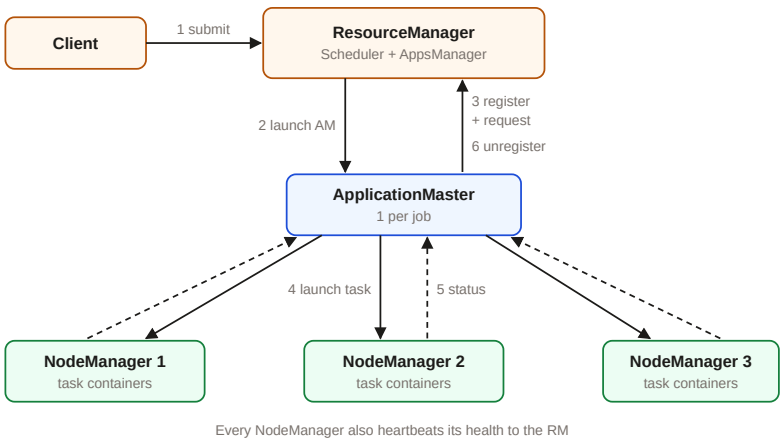
2. CPU. A node has 64/16 = 4 GB per vCPU; every container asks 2 GB per vCPU, so vCPUs run out with half the RAM idle.

3. Per node min(64/4, 16/2) = 8 mappers; 10,000 / 8 = **1,250 nodes**, which fits in production's 2,000.

Topic cards

6 topics · everything the pool tests

YARN and its three components



A job on YARN: the client submits to the ResourceManager, which starts the job's ApplicationMaster; the AM requests containers and has NodeManagers launch tasks, which report status back.

FACTS

- **YARN** (Yet Another Resource Negotiator) arrived in **Hadoop 2.0 (2013)** and replaced the Hadoop 1.x **JobTracker**.
- Hadoop 1.x JobTracker did resource management and job scheduling in one process: MapReduce only, ceiling about **4,000 nodes**.
- **ResourceManager (RM)**: one per cluster, on a master node. Two parts: **Scheduler** (gives out containers) and **ApplicationsManager** (accepts jobs, starts each AM).
- **NodeManager (NM)**: one per worker node. Launches and monitors containers, sends **heartbeats** (regular I-am-alive reports) to the RM.
- **ApplicationMaster (AM)**: one per running job. Asks the RM for containers and tells NMs to start tasks in them.
- **Container**: a slice of one node, {vcores: N, memory: M MB}. Replaced Hadoop 1's fixed **slots**. Ancestor of the Kubernetes Pod.

HOW IT WORKS

1. Client submits the job to the ResourceManager.
2. The RM's ApplicationsManager launches a container for the job's ApplicationMaster.
3. The AM registers with the RM and requests containers (e.g. 100 of 4 cores, 8 GB).
4. The RM's Scheduler grants what is free (e.g. 80) and picks the nodes.
5. The AM tells those NodeManagers to launch tasks in the containers.
6. Tasks report status to the AM; NMs heartbeat node health to the RM.
7. When all tasks finish, the AM releases containers and unregisters from the RM.

WHY · PROS · LIMITS

- Splitting resource management (RM) from per-job logic (AM) removed the single JobTracker bottleneck: YARN scales past 10,000 nodes.
- The AM holds the framework-specific logic, so YARN is **framework-agnostic**: MapReduce, Spark, Tez and Samza share one cluster.
- Containers are flexible in size, unlike fixed slots, so CPU-heavy and RAM-heavy tasks both fit.
- NMs isolate containers with Linux **cgroups** (kernel CPU limits) and memory limits; a container over its memory limit is killed.

COMPARE

	Hadoop 1.x	YARN (Hadoop 2.x)
Who manages resources	JobTracker (also schedules jobs)	ResourceManager only
Per-job manager	inside the JobTracker	one ApplicationMaster process per job
Frameworks	MapReduce only	any (Spark, Tez, Samza ...)
Resource unit	fixed slot	flexible container
Max nodes	about 4,000	over 10,000
RM high availability	no	yes (ZooKeeper)

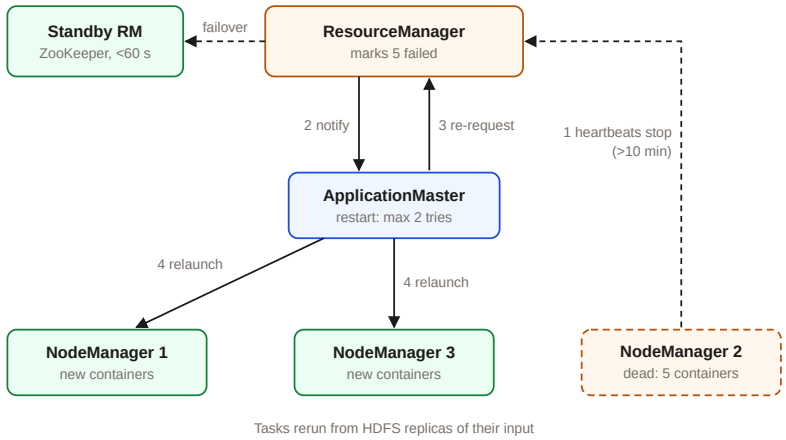
- Limit: containers are sized only by CPU and RAM. Kubernetes has largely replaced YARN for new deployments.

	How many	Job
ResourceManager	1 per cluster	global allocation of containers
NodeManager	1 per node	launch and watch containers, heartbeat
ApplicationMaster	1 per application	request containers, run and retry tasks

TRAPS

- NameNode and DataNode are HDFS (storage). They are not YARN parts and do not allocate CPU or RAM.
- The AM is per job, the RM is per cluster. Q1's wrong favourite is the AM.
- Hadoop 2.0 also added the Standby NameNode. That is HDFS high availability, not YARN (Q4 distractor).
- The AM requests containers; it does not grant them. Only the RM's Scheduler grants.

YARN fault tolerance



NodeManager 2 dies: the RM notices missed heartbeats, marks its containers failed and tells the AM, which re-requests and relaunches the tasks on healthy nodes; a Standby RM covers RM failure.

FACTS

- YARN handles failure in three layers: **NodeManager**, **ApplicationMaster**, **ResourceManager**.
- NM failure: RM sees missing heartbeats (timeout over **10 minutes**) and marks all containers on that node **failed**.
- AM failure: RM restarts the AM, up to `yarn.resourcemanager.am.max-attempts` (default **2**).
- A restarted AM can recover job state from HDFS checkpoints if the framework supports it: MapReduce yes, early Spark no.
- RM failure: **RM High Availability** (YARN 2.4+) uses ZooKeeper with an Active and a Standby RM; failover in under **60 seconds**.

WHY · PROS · LIMITS

HOW IT WORKS

1. A NodeManager crashes; its containers die and its heartbeats stop.
2. The RM waits out the heartbeat timeout (over 10 minutes) and declares the node lost.
3. The RM removes the node's CPU and RAM from the schedulable pool and marks its containers failed.
4. The RM notifies the job's ApplicationMaster.
5. The AM requests the same number of replacement containers.
6. The RM grants them on healthy nodes, ideally ones holding HDFS replicas of the input.
7. The AM has the new NMs relaunch the tasks; map tasks rerun from the start of their input split.

COMPARE

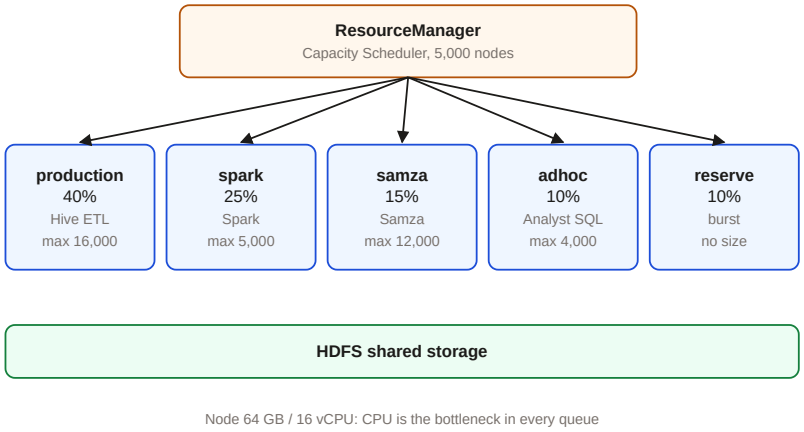
- Fault handling is layered: YARN detects node loss, but the AM decides how to retry, because only it knows the framework.
- Input survives in HDFS replicas, so a failed task can rerun on another node.
- Finished map output stored on the dead node's local disk is lost too, so those maps may rerun.
- Limit: a 10-minute heartbeat timeout is slow for streaming jobs; the framework must recover faster itself.
- Samza streams on YARN are long-running containers, so YARN's restarts give them fault tolerance without a separate stream cluster.

Failure	Detected by / how	Recovery
NodeManager	RM, missed heartbeats (>10 min)	containers marked failed; AM re-requests elsewhere
ApplicationMaster	RM	RM restarts AM (default max 2 attempts); state from HDFS checkpoint if supported
ResourceManager	ZooKeeper	Standby RM becomes Active in <60 s (YARN 2.4+)

TRAPS

- The AM, not the RM, re-requests the replacement containers. The RM only detects and marks.
- The NM timeout is over 10 minutes, not seconds. The 60 seconds figure is RM failover.
- Recovery from checkpoint depends on the framework: MapReduce can, early Spark could not.

Schedulers and capacity planning



The live exercise's Capacity Scheduler queues on LinkedIn's 5,000-node cluster, with each queue's maximum containers. (The case study's real split had tez 7% and reserve 3%; the exercise merges them into reserve 10%.)

FACTS

- The RM Scheduler is **pluggable** with three policies: **FIFO**, **Capacity**, **Fair**.
- **FIFO** (first in, first out): one queue in submission order; one long job blocks the rest.
- **Capacity**: queues with guaranteed shares (e.g. Engineering 60%, Finance 40%); idle capacity is borrowed. Default in HDP/CDH; used by Yahoo!, LinkedIn.
- **Fair**: running jobs get equal shares, so short jobs finish fast. Used by Facebook, Twitter.
- **DRF** (Dominant Resource Fairness): fairness across CPU and RAM together, for clusters mixing CPU-heavy and RAM-heavy jobs.

HOW IT WORKS

1. Give each queue its share of nodes: nodes × share.
2. Turn the queue's nodes into total RAM and total vCPUs.
3. Divide total RAM by the container's RAM: containers by RAM.
4. Divide total vCPUs by the container's vCPUs: containers by CPU.
5. The smaller count is the maximum, since a container needs both.
6. The resource giving the smaller count is the bottleneck.

- **Pre-emption**: taking borrowed containers back so a queue gets its guarantee. LinkedIn gave a **30-second** warning.

WHY · PROS · LIMITS

- Capacity guarantees protect SLAs (service level agreements, e.g. ETL done by 6 a.m.): analyst queries cannot starve production pipelines.
- Consolidation: LinkedIn merged 3 single-workload clusters into one 5,000-node YARN cluster; utilisation rose from 45% to 78%, 73% more work on the same hardware.
- Borrowing only works if capacity can be reclaimed, so Capacity needs pre-emption.
- Aggressive pre-emption killed long Spark jobs; the fix was a 30-second grace period and checkpoint-on-preempt hooks.
- Fixed queue percentages need regular tuning when workloads change.

COMPARE

	Policy	Good for / weakness
FIFO	one queue, submission order	simple; one long job starves the rest
Capacity	queues with guaranteed %, borrow idle capacity	multi-team SLAs; needs pre-emption and tuning
Fair	equal share to all running jobs	many short jobs; weaker guarantees for one team

CALCULATION

Live exercise (P2 slide 23): 5,000 nodes, each 64 GB and 16 vCPU. Queues production 40%, spark 25%, samza 15%, adhoc 10%, reserve 10%. Containers: Hive mapper {4 GB, 2 vCPU}, Spark executor {8 GB, 4 vCPU}, Samza task {2 GB, 1 vCPU}; adhoc uses the Hive mapper size.

Queue	Nodes	RAM GB	vCPU	By RAM	By CPU	Max
production	2,000	128,000	32,000	32,000	16,000	**16,000**
spark	1,250	80,000	20,000	10,000	5,000	**5,000**
samza	750	48,000	12,000	24,000	12,000	**12,000**
adhoc	500	32,000	8,000	8,000	4,000	**4,000**

Bottleneck: **CPU** everywhere. Nodes for 10,000 Hive mappers: **1,250**.

Step by step

1. Given 5,000 nodes and spark share 25% -> spark nodes = 5,000 × 0.25 = 1,250.
2. Given 1,250 nodes of 64 GB -> RAM = 80,000 GB; of 16 vCPU -> vCPU = 20,000.
3. Given 80,000 GB and 8 GB executors -> by RAM = 10,000.
4. Given 20,000 vCPU and 4-vCPU executors -> by CPU = 5,000.
5. Given by RAM 10,000 and by CPU 5,000 -> max = 5,000, bottleneck CPU.
6. Given a node of 64 GB / 16 vCPU and a mapper of 4 GB / 2 vCPU -> per node = min(16, 8) = 8.
7. Given 8 mappers per node and 10,000 mappers -> nodes = 10,000 / 8 = 1,250 (production has 2,000, so it fits).

Variants

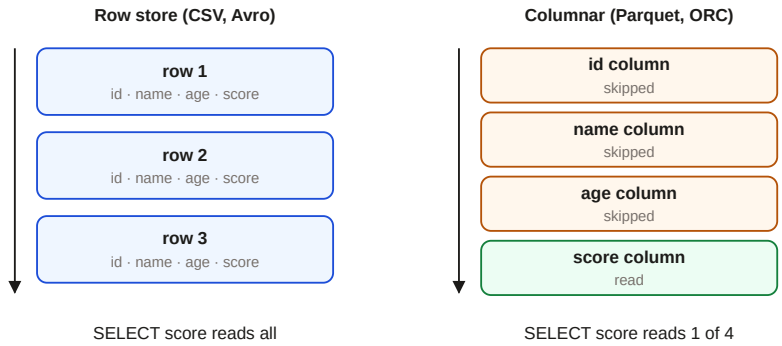
- Quick bottleneck test: node RAM per vCPU = 64/16 = 4 GB; every container asks 2 GB per vCPU, so CPU runs out first with half the RAM idle.
- 2,000 nodes of 128 GB / 32 vCPU, spark 25%: 500 nodes, 64,000 GB, 16,000 vCPU; by RAM 8,000, by CPU 4,000; max 4,000 (CPU).
- 3,000 Spark executors {8 GB, 4 vCPU} on 64 GB / 16 vCPU nodes: per node min(8, 4) = 4; nodes = 750.

- Utilisation 45% to 78%: 78 / 45 = 1.73, so 73% more work on the same hardware.

TRAPS

- Take the **minimum** of the by-RAM and by-CPU counts, never the RAM count alone.
- The reserve queue has no container size, so it gets no count.
- Capacity is not a hard cap: idle capacity is lent out and reclaimed by pre-emption.
- 73% is 78/45 − 1, not 78 − 45 = 33 points.

Row vs columnar storage and Parquet



A row store keeps whole records together, so reading one column reads them all; a columnar store keeps each column together and skips the rest.

FACTS

- **Row store** (CSV, JSON, Avro): one whole record after another on disk.
- **Columnar store** (Parquet, ORC): all values of one column stored together.
- Slide example: 100-column, 10 TB table, 3-column query: CSV reads 10 TB, Parquet about 300 GB, **33×** faster, **97%** less I/O.
- Parquet layout: file header, **row groups** (about 128 MB), each split into **column chunks**, each into **pages**; footer holds metadata and stats.
- The footer stores **min/max per column chunk**, which enables **predicate pushdown**: skipping row groups a filter cannot match.
- Spark pushes down by default: `spark.sql.parquet.filterPushdown=true`. A 10 TB Parquet query may read only 50 GB.

WHY · PROS · LIMITS

- Analytics queries touch few columns of many rows; a column store reads only those columns.
- Same-type values sit together, so they compress better.
- Min/max stats let whole row groups be skipped before any data is read.
- Row groups are independent, so each can go to its own Mapper (splittable).

HOW IT WORKS

1. The query, e.g. `WHERE score > 90`, first reads only the file footer.
2. It gets min/max of the score column for every row group.
3. Row group 1 has score in [60, 85]: max 85 is not above 90, so skip it with no I/O.
4. Row group 2 has score in [85, 99]: it may match, so read it.
5. Inside row group 2, read only the score column chunk, not the other columns.

COMPARE

	Row store (CSV, Avro)	Columnar (Parquet, ORC)
Disk layout	record by record	column by column
SELECT score FROM users	reads id, name, age, score	reads score only
Filter skipping	none in CSV	row groups skipped by min/max

- Limit: writing a single record is awkward, so row formats (Avro) are better for streaming appends.

Compression	lower	higher (similar values together)
Best for	writes, appends, exchange	read-heavy analytics

CALCULATION

Slide (P1 slides 6, 16): D = 10 TB, N = 100 columns, k = 3 read.

- Read = D × k / N = 10 TB × 3/100 = 0.3 TB = **300 GB**
- I/O reduction = 1 − 3/100 = **97%**
- Speedup ≈ N / k = 100/3 ≈ **33×**

Pushdown (slide 23): 10 TB on disk, 50 GB read: 50/10,000 = 0.005, so 99.5% less, 200× faster.

Step by step

1. Given k = 4 of N = 80 columns -> fraction read = 4/80 = 0.05.
2. Given fraction 0.05 and D = 10 TB -> read = 0.5 TB = 500 GB.
3. Given fraction 0.05 -> reduction = 1 − 0.05 = 95%; speedup = 80/4 = 20×.
4. Given 6 h I/O-bound and fraction 0.05 -> new time ≈ 360 × 0.05 = 18 min.
5. Given row group min/max [60, 85] and filter score > 90 -> max 85 < 90, skip.

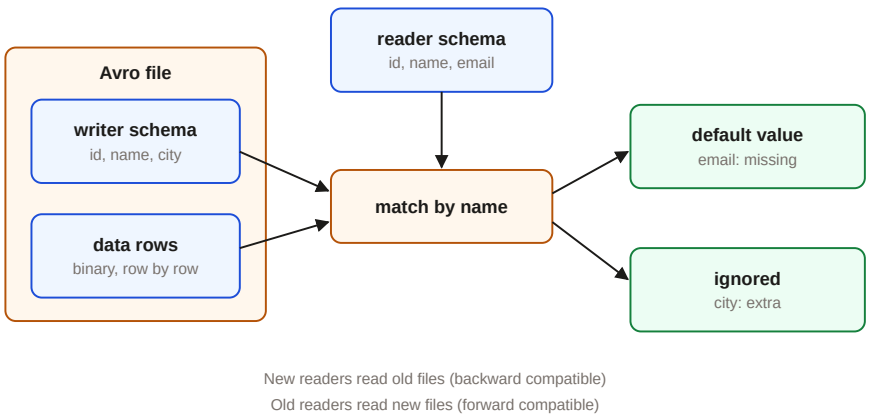
Variants

- D = 20 TB, N = 200, k = 5: read 0.5 TB, reduction 97.5%, speedup 40×.
- 2 TB on disk, 40 GB read after pushdown: 40/2,000 = 0.02, reduction 98%, speedup 50×.
- Filter score < 70 with RG1 [60, 85], RG2 [85, 99]: read RG1 (min 60 < 70), skip RG2 (min 85).

TRAPS

- Avro is a row format even though it is binary and self-describing.
- The 33× assumes columns of equal size and an I/O-bound query; it is an estimate.
- Skip a row group only when its range cannot match: for `> 90` look at the max, for `< 70` look at the min.
- Pushdown is per row group; the whole file is not skipped unless every row group fails.

File formats: Avro, Parquet, ORC



Avro schema evolution: the writer schema travels in the file, the reader brings its own, fields are matched by name, missing ones get defaults and extra ones are ignored.

FACTS

- Five formats: **Text/CSV** (row, external schema), **JSON** (row, not splittable raw), **Avro** (row), **Parquet** (columnar), **ORC** (columnar).
- Avro, Parquet and ORC are **self-describing**: the schema (field names and types) travels inside the file. Avro's schema is JSON.
- **Avro**: Snappy/Deflate; best for **schema evolution** and Kafka → HDFS ingest.
- **Parquet**: Snappy/Gzip; best for Spark, Presto, ML pipelines.
- **ORC**: Zlib/Snappy; best for Hive, **highest compression**, Hive ACID and vectorized reads.
- **Schema evolution**: the schema changes (e.g. a new field) and old files and old readers keep working.

WHY · PROS · LIMITS

- Avro is row-oriented, so appending one event at a time is cheap: good for streams.
- Reader/writer schema matching means producers can add fields without breaking consumers.
- Parquet's columns and stats make read-heavy analytics fast, but per-record appends slow.
- ORC compresses best and is built for Hive, so it fits warehouses and cold archives.
- CSV has no embedded schema and no stats, so every query reads and parses every column.

HOW IT WORKS

1. Avro writes the **writer schema** into the file with the data.
2. A reader brings its own **reader schema**.
3. Avro matches fields by name between the two schemas.
4. A field the reader expects but the file lacks gets its **default value**.
5. A field in the file that the reader does not know is **ignored**.
6. So new readers read old files (backward) and old readers read new files (forward).

COMPARE

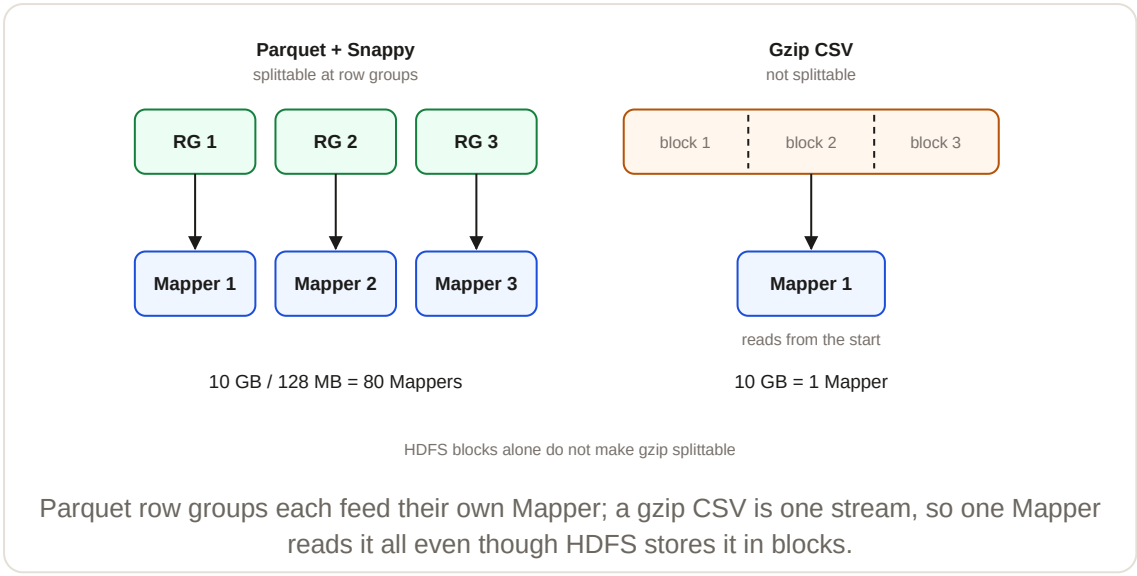
	Avro	Parquet	ORC
Orientation	row	columnar	columnar
Codec	Snappy/Deflate	Snappy/Gzip	Zlib/Snappy
Strength	schema evolution, appends	analytic reads, pushdown	highest compression, Hive ACID
Best for	Kafka → HDFS ingest	Spark, Presto, ML	Hive warehouse

Scenario	Format	Codec
Streaming ingest (Kafka → HDFS)	Avro	Snappy
Spark analytics pipeline	Parquet	Snappy
Hive SQL warehouse	ORC	Zlib
Data exchange / APIs	Avro or JSON	none
ML features	Parquet	Gzip
Archive / cold storage	ORC or Parquet	Zstd

TRAPS

- Avro is self-describing and binary, but it is row-oriented, not columnar.
- ORC is also correct for analytics on Hive; Parquet is the broader standard (Q2 note).
- Missing field = default value; extra field = ignored. Do not swap them.
- JSON lines is not splittable in raw form.

Compression codecs and splittability



Parquet row groups each feed their own Mapper; a gzip CSV is one stream, so one Mapper reads it all even though HDFS stores it in blocks.

FACTS

- **Splittable**: reading can start from any point in the file without decompressing from the beginning, so several Mappers read it in parallel.
- Snappy: fast, medium ratio, not splittable alone (Parquet row groups are).
- Gzip: slow, high ratio, **not splittable**. LZO: fast, medium, splittable with an index.
- Bzip2: slowest, highest ratio, splittable. Zstd: fast, high ratio, not splittable (block-level).
- **Gzip trap**: a 10 GB gzip CSV is read by exactly **1 Mapper**, though HDFS stores it in many blocks.
- Best practice: Parquet + Snappy for analytics, Avro + Snappy for streaming ingest, no raw gzip on big CSV.

HOW IT WORKS

1. HDFS stores the file in blocks across nodes.
2. MapReduce asks whether the format is splittable.
3. If splittable, each split (block or Parquet row group) becomes its own Mapper.
4. Each Mapper runs on a node holding its split (data-local scheduling).
5. If not splittable (gzip), one Mapper reads the whole file from the start.

WHY · PROS · LIMITS

- Splittability decides parallelism: no splits means one Mapper and an idle cluster.
- Splittable input lets the scheduler run tasks where the data is, avoiding network copies.
- Codecs trade speed for size: Snappy for hot data read often, Zstd or Bzip2 for cold data.
- A container format (Parquet, Avro) compresses inside its own blocks, so the file stays splittable even with Snappy.

COMPARE

Codec	Speed / ratio	Splittable?
Snappy	fast / medium	no (Parquet row groups are)
Gzip	slow / high	no
LZO	fast / medium	yes, with index
Bzip2	slowest / highest	yes
Zstd	fast / high	no (block-level)

CALCULATION

Slide (P1 slides 20-21): a 10 GB gzip CSV gets **1 Mapper**. The same 10 GB as Parquet with 128 MB row groups gets 10 × 1,024 / 128 = **80** splits, so up to 80 Mappers in parallel.

Rule: splits = ⌈F / B⌉ if splittable, 1 if not.

Step by step

1. Given a gzip file of any size -> splits = 1.

2. Given F = 10 GB -> F = 10 × 1,024 = 10,240 MB.
3. Given 10,240 MB and B = 128 MB -> splits = 10,240 / 128 = 80.
4. Given 80 splits -> up to 80 Mappers in parallel.

Variants

- 10 GB with 256 MB row groups: 10,240 / 256 = 40 Mappers.
- 1 GB with 128 MB row groups: 1,024 / 128 = 8 Mappers.
- 300 MB with 128 MB blocks: ⌈300 / 128⌉ = ⌈2.34⌉ = 3 Mappers.

TRAPS

- HDFS splitting a gzip file into blocks does not make it splittable for MapReduce.
- Snappy alone is not splittable; Parquet + Snappy is, at row-group boundaries.
- Bzip2 is splittable, gzip is not; T/F 3 hinges on this.
- Round splits up: a partial last block still needs a Mapper.

CHAPTER 2 · WEEK 6

Apache Spark: RDDs and in-memory computing

Spark keeps working data in RAM so iterative jobs stop paying MapReduce's disk cost on every pass. The chapter covers how Spark is built, how RDDs recover from failures through lineage, lazy evaluation, the narrow/wide split that decides shuffles and stages, and DataFrames.

Exam question pool

22 questions from the slides · click a question for its answer

MCQ **Q1.** What makes Spark significantly faster than MapReduce for iterative algorithms?

(a) Spark uses more CPU cores per node
(b) Spark writes intermediate results to faster SSDs
(c) Spark caches intermediate data as RDDs in memory
(d) Spark skips the shuffle phase entirely

(c) Spark caches intermediate data as RDDs in memory

Later iterations read from RAM instead of re-reading and re-writing HDFS; the hardware is the same.

MCQ **Q2.** How does Spark achieve fault tolerance in RDDs without full data replication?

(a) Checkpointing to HDFS every 60 seconds
(b) Tracking lineage and recomputing lost partitions from parent RDDs
(c) Copying each RDD partition twice across two Executors
(d) Using a write-ahead log on the Driver node

(b) Tracking lineage and recomputing lost partitions from parent RDDs

The recipe is kept, not copies; (c) is replication.

MCQ **Q3.** Which of the following is **NOT** a component of the Spark ecosystem?

(a) MLlib
(b) GraphX
(c) Structured Streaming
(d) HBase

(d) HBase

HBase is a separate NoSQL database on HDFS; Spark can only read it as a data source.

MCQ **Q4.** In Spark, a “transformation” such as map() or filter() is:

(a) Immediately executed when called and materialised to memory
(b) Lazily evaluated — only executed when a subsequent action is called
(c) Automatically persisted to HDFS after execution
(d) Only available in the Scala API, not in PySpark

(b) Lazily evaluated — only executed when a subsequent action is called

`rdd.filter(f)` alone does nothing; `rdd.filter(f).count()` runs it.

TRUE/FALSE **1.** Spark can only run on top of Hadoop YARN and has no other deployment option.

False. The Cluster Manager can be YARN, Kubernetes, Mesos or Spark Standalone.

TRUE/FALSE **2.** RDDs are mutable — individual elements can be modified in place after creation.

False. RDDs are immutable; every transformation creates a new RDD.

TRUE/FALSE **3.** Spark's DataFrame API provides schema-aware, named-column operations similar to SQL tables.

True. A DataFrame has named columns and a schema, like a relational table, and accepts SQL.

TRUE/FALSE **4.** MapReduce is generally better than Spark for machine learning workloads that require many iterations over the same dataset.

False. Spark is better: it caches the data in RAM and is 10–100× faster on iterative work, while MapReduce hits disk every iteration.

DESCRIPTIVE **D1.** Explain the concept of a **Resilient Distributed Dataset (RDD)**. What does *"resilient"* mean in this context? How does Spark recover a lost RDD partition without replication?

RDD: a read-only, partitioned collection of records, created from stable storage (HDFS, S3) or by transforming another RDD. Properties: immutable, partitioned, lazy, lineage, resilient.

Resilient: it survives a node failure by recomputing lost partitions from its lineage (the steps that built it) instead of keeping 3 copies. The computation is the backup.

Recovery:

1. Spark detects Part 2 of mappedRDD is lost.
2. Traces lineage: mappedRDD ← filteredRDD ← inputRDD.
3. Re-reads input Part 2 from HDFS.
4. Re-applies filter, then map.
5. Part 2 restored; the job is not restarted.

Diagram: draw the lineage chain shown in the diagram in the RDD card: inputRDD → filteredRDD → mappedRDD, with Part 2 of mappedRDD crossed out and the recovery path going back to inputRDD Part 2 on HDFS.

Cheap for narrow dependencies; checkpoint long chains.

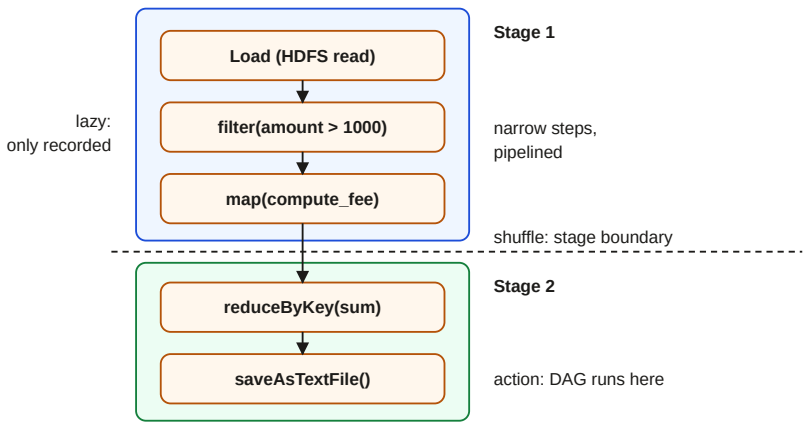
DESCRIPTIVE **D2.** Differentiate between **Spark transformations** and **actions**. Give two concrete examples of each. Why does Spark not execute transformations immediately?

	Transformation	Action
Runs	Lazily, only recorded in the DAG	Immediately, triggers the DAG
Returns	A new RDD	A value to the Driver, or output to storage
Examples	`map(f)`, `filter(f)`	`count()`, `collect()`

Why wait: with the whole DAG in view, Spark can

- optimise the plan (DAG scheduler, or Catalyst for DataFrames);
- pipeline filter and map in one pass, with no intermediate disk writes;
- run only the partitions needed, e.g. `first()` stops at the first match.

DESCRIPTIVE **D3.** Describe Spark's **lazy evaluation model**. What is the advantage of deferring computation until an action is called? Use the example `rdd.filter(x > 1000).first()` to illustrate.



Transformations are only recorded; the action saveAsTextFile runs the DAG, with narrow steps pipelined in Stage 1 and the reduceByKey shuffle starting Stage 2.

Model: transformations are only recorded in the DAG. When an action is called, Spark analyses the DAG, optimises it, pipelines compatible steps in one pass per partition, and runs only the partitions it needs.

Advantages: less work (early stop, skipped partitions), no intermediate disk writes, whole-plan optimisation.

Example:

1. `filter(x > 1000)` : recorded, no data read.
2. `first()` : action; Spark starts reading the first partition.
3. It returns the first value above 1000 and stops.
4. Eager execution would filter the entire dataset, then keep one value.

ANALYTICAL

A1. Iterative I/O Cost Analysis: You need to train a K-Means clustering model over **50 GB** of bKash transaction data with **100 iterations**. Assume HDFS read speed = 500 MB/s and RAM cache read speed = 5,000 MB/s.

- Calculate the total HDFS I/O for MapReduce (all 100 iterations)
- Calculate the total I/O for Spark (50 GB fits in cluster RAM)
- Estimate the wall-clock time for each approach
- Under what conditions would Spark NOT outperform MapReduce?

Given: D = 50 GB = 50,000 MB, k = 100, HDFS 500 MB/s (write assumed = read), RAM 5,000 MB/s.

	MapReduce	Spark
HDFS I/O	100 × (50 + 50) = **10 TB**	**50 GB** (read once); RAM reads 99 × 50 = 4,950 GB
Per pass	50,000/500 = 100 s read + 100 s write = 200 s	100 s first, then 50,000/5,000 = 10 s
Total time	100 × 200 = **20,000 s ≈ 5.56 h**	100 + 99 × 10 = **1,090 s ≈ 18.2 min**

Disk I/O 200× less; time speedup 20,000 / 1,090 ≈ **18.3×**.

Spark does not win when: the job is single-pass (nothing to reuse); the data does not fit in RAM (spill or evict and recompute); Executor memory (~8 GB minimum) is too costly; stable MapReduce code is already fast enough.

ANALYTICAL

A2. RDD Failure Recovery Trace: A Spark job computes `input` → `filteredRDD` → `mappedRDD` → `reducedRDD` over 10 partitions. Mid-execution, the Executor holding partitions 3 and 7 of `reducedRDD` crashes. Trace the complete recovery sequence: what does the Driver detect, which partitions of which RDDs need to be recomputed, and what is the worst-case recovery cost if `reducedRDD` involves a **wide** dependency (`groupByKey`)?

Stages: filter, map narrow → Stage 1 (input → filteredRDD → mappedRDD). groupByKey wide → Stage 2 (reducedRDD).

Driver detects: the Executor is lost; tasks for reducedRDD partitions 3 and 7 failed. The other 8 partitions are fine.

Lineage: reducedRDD ← groupByKey ← mappedRDD ← map ← filteredRDD ← filter ← input.

Case	Recomputed
If all narrow	Partitions 3 and 7 at each level: 2 × 4 = 8 computations (20%)
Wide, shuffle files survive	Only reducedRDD 3 and 7, re-fetching from all 10 mappedRDD outputs
Wide, worst case	Shuffle output lost too: all 10 input, filteredRDD and mappedRDD partitions (30) + reshuffle + reducedRDD 3, 7 (2) ≈ the whole job

Limit the cost: reduceByKey instead of groupByKey, cache mappedRDD, checkpoint long chains.

DISCUSSION

1. In-memory is a multiplier, not a panacea: Spark is 10–100× faster on iterative workloads; for single-pass ETL over data larger than RAM, gains are smaller (2–5×)

Claim: memory speeds up reuse; it does not fix every workload.

For: iterative jobs reuse the working set. RDD paper: logistic regression 127 s vs 0.96 s per iteration (132×).

Against: single-pass ETL reads each byte once, and data larger than RAM spills to disk, so the gain falls to 2–5×.

Choose by workload: more than 2 iterations favours Spark.

DISCUSSION **2. The lineage model scales elegantly**: re-computing a lost partition from lineage is cheaper than maintaining 3× replicated in-memory state

Claim: lineage beats replication for in-memory data.

For: 3× replication triples RAM and network writes on every step; lineage costs nothing until a failure and then rebuilds only lost partitions.

Against: with wide dependencies or long chains, one lost partition can need many ancestors recomputed, so checkpoints are still needed.

DISCUSSION **3. Shuffle is the bottleneck**: both at GraySort and in production; minimising wide dependencies (shuffle) is the primary Spark tuning skill

Claim: shuffles dominate Spark's cost.

For: a shuffle moves data over the network, writes to disk and forces a stage barrier. Spark's GraySort win came from a cheaper shuffle: sort-based shuffle (about 120 TB less I/O), all 8 SSDs as shuffle directories, one sortByKey. Tuning: reduceByKey over groupByKey, co-partition before joins.

Against: sorts and un-partitioned joins need a shuffle; you minimise it, you cannot remove it.

DISCUSSION **4. Benchmark marketing is real**: GraySort made Spark famous; it also hid cases where Spark's JVM memory overhead hurt performance (many small files, high-cardinality `groupBy`)

Claim: a headline benchmark sells a system but shows one workload.

For: 100 TB in 23 min on 206 nodes vs Hadoop's 72 min on 2,100 drove Spark's adoption.

Against: it ran on tuned large nodes (244 GB RAM, 8 SSDs). Many small files (per-task overhead) and high-cardinality groupBy (too many keys for JVM memory, GC pauses) were not tested.

Test on your own workload.

DISCUSSION **5. Open-source velocity**: the Apache Spark community grew to 1,000+ contributors in 5 years; Databricks' business model (cloud + support) aligned commercial incentives with open-source development

Claim: a free engine plus a paid service sped up Spark.

For: Databricks earns from cloud and support, not licences, so improving open Spark helps its business. 1,000+ contributors in 5 years produced Spark SQL, MLlib, Structured Streaming, GraphX.

Against: one company steering the project can shape the roadmap and push features to its paid product first (lock-in risk).

LIVE EXERCISE

Task. Task (25 minutes, pairs)
Run the following PySpark script locally (Spark local mode).
Goal: build and inspect an RDD lineage; observe lazy evaluation; compare cached vs. uncached timing.

```
from pyspark import SparkContext
import time
sc = SparkContext("local[4]", "RDDDemo")
sc.setLogLevel("WARN")
# Build lineage (lazy - no compute yet)
lines = sc.textFile("README.md")
words = lines.flatMap(lambda l: l.split())
pairs = words.map(lambda w: (w.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a+b)
# Inspect lineage BEFORE action
print(counts.toDebugString().decode())
# First action - triggers full DAG
t0 = time.time()
n = counts.count()
print(f"count={n}  time={time.time()-t0:.2f}s")
# Cache and re-run
counts.cache()
counts.count()          # warm up cache
t1 = time.time()
n = counts.count()      # served from cache
print(f"cached count={n} "
      f"time={time.time()-t1:.4f}s")
sc.stop()
```

`local[4]` = local mode, 4 threads. textFile, flatMap, map, reduceByKey only build lineage; the first `count()` runs the DAG. Key additions for the questions (run with `python rdd_demo.py`):

```
print(counts.toDebugString().decode())   # Q1 before cache
for i in (1, 2):                          # Q2 two uncached runs
    t0 = time.time(); counts.count()
    print(i, time.time() - t0)
counts.cache()
print(counts.toDebugString().decode())   # Q1 after cache
counts.count()                            # fills the cache
t1 = time.time(); counts.count()
print("cached", time.time() - t1)
# Q3: counts = pairs.groupByKey().mapValues(sum)
input("open localhost:4040, then Enter")
```

Expected: the lineage shows HadoopRDD → Python RDDs, with a `+-` indent at the reduceByKey shuffle; count = number of distinct lower-cased words.

LIVE EXERCISE

1. Call `rdd.toDebugString()` before and after `cache()` — what changes?

The parent chain does not change, because cache() is lazy and lineage is kept for recovery. After cache() the top line shows a storage level (e.g. `[Memory Serialized 1x Replicated]`). After an action fills the cache it also shows `CachedPartitions` and memory size (expected).

LIVE EXERCISE

2. Time `rdd.count()` twice without `cache()`. Then add `.cache()` and time again. Report the speedup.

Speedup = uncached time / cached time. Expected: run 1 slowest (start-up, read, shuffle); run 2 a bit faster (shuffle files reused, Stage 1 skipped); cached run fastest. On a small README start-up overhead dominates, so the ratio is modest; the RDD paper shows 127 s → 0.96 s (132×) at scale.

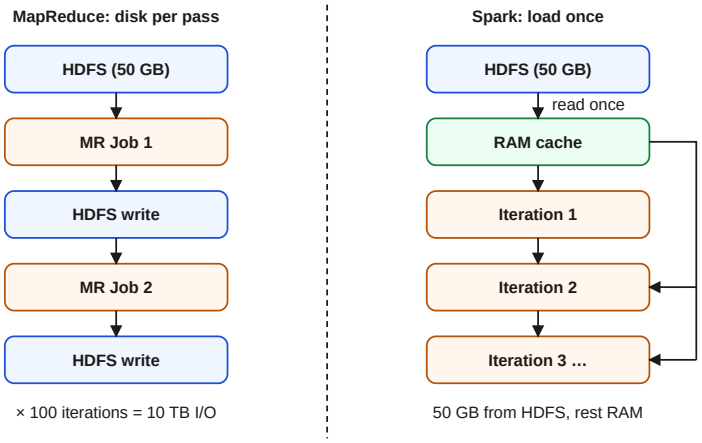
LIVE EXERCISE

3. Change `reduceByKey` to `groupByKey`. Use the Spark UI (localhost:4040) to compare shuffle read/write bytes between the two.

Use `pairs.groupByKey().mapValues(sum)`, then read Shuffle Write/Read on the UI's Stages tab. Expected: same counts, larger shuffle bytes for groupByKey, because reduceByKey combines each partition's pairs before the shuffle and groupByKey sends every `(word, 1)` pair.

Topic cards 6 topics · everything the pool tests

Why Spark: the MapReduce iteration bottleneck



MapReduce writes to HDFS between every job, while Spark reads the 50 GB once into a RAM cache that feeds every later iteration.

FACTS

- MapReduce writes every intermediate result to HDFS (the cluster's shared disk) before the next step can start.
- Iterative algorithms (K-Means, logistic regression, PageRank) pass over the same data many times, so MapReduce pays disk I/O every iteration.
- Spark reads the data from HDFS once, caches it in RAM, and runs every later iteration from memory.
- Slide example: K-Means, 100 iterations over 50 GB. MapReduce does 10 TB of disk I/O, Spark 50 GB: about 200× less.
- Spark is 10–100× faster on iterative work. For single-pass ETL over data larger than RAM the gain is only 2–5×.
- Use Spark for ML, graphs, interactive queries, streaming, or more than 2 iterations over the same data.

HOW IT WORKS

1. Iteration 1 reads the dataset from HDFS
2. Spark keeps the partitions in Executor RAM as a cached RDD
3. Each later iteration reads the cached partitions from memory, not from disk
4. No intermediate result goes back to HDFS between iterations
5. Only the final result is written out to storage

WHY · PROS · LIMITS

- RAM is much faster to read than disk, and Spark also skips the HDFS write after every step.
- Memory multiplies the benefit of reuse. A job that reads each byte once has nothing to cache.
- Spark runs on the same hardware as MapReduce. The speed comes from memory, not from more CPU cores.
- Limitation: if the data does not fit in cluster RAM, partitions spill to disk or are evicted and recomputed.
- Limitation: Spark needs about 8 GB per Executor. MapReduce stays reasonable for single-pass batch jobs on very large clusters where that

COMPARE

	MapReduce	Spark
Intermediate storage	Disk (HDFS) after every step	Memory (spills to disk if needed)
Programming model	Map + Reduce only	map, filter, join, groupBy, SQL, ML, streaming
Iterative algorithms	Slow: disk per iteration	Fast: cached RDD, 10–100×
Fault tolerance	Re-execute task from HDFS	Recompute from lineage
Streaming / interactive	Not native / over 30 s	Structured Streaming / sub-second shell

memory cost is prohibitive, or when existing MapReduce code is stable and fast enough.

Languages / learning curve	Java; low (2 functions)	Python, Scala, Java, R; moderate

CALCULATION

K-Means on 50 GB (50,000 MB), 100 iterations. HDFS 500 MB/s (write assumed equal to read), RAM 5,000 MB/s.

	MapReduce	Spark
HDFS I/O	100 × (50 + 50) = 10,000 GB = 10 TB	50 GB (read once)
Time per pass	100 s read + 100 s write = 200 s	first 100 s, then 10 s from RAM
Total time	100 × 200 = 20,000 s ≈ 5.56 h	100 + 99 × 10 = 1,090 s ≈ 18.2 min

Disk I/O ratio 10,000 / 50 = **200×**. Time speedup 20,000 / 1,090 ≈ **18.3×**.

Step by step

1. Given D = 50 GB, k = 100, MapReduce reads and writes D each iteration → MR I/O = 100 × (50 + 50) = 10,000 GB = 10 TB
2. Given Spark reads HDFS only in iteration 1 → Spark HDFS I/O = 50 GB
3. Given 10,000 GB and 50 GB → disk I/O ratio = 10,000 / 50 = 200×
4. Given 50,000 MB at 500 MB/s → one HDFS pass = 100 s; one MR iteration (read + write) = 200 s
5. Given 200 s per iteration → T_MR = 100 × 200 = 20,000 s ≈ 5.56 h
6. Given RAM at 5,000 MB/s → one RAM pass = 10 s; T_Spark = 100 + 99 × 10 = 1,090 s ≈ 18.2 min
7. Given 20,000 s and 1,090 s → speedup = 20,000 / 1,090 ≈ 18.3×

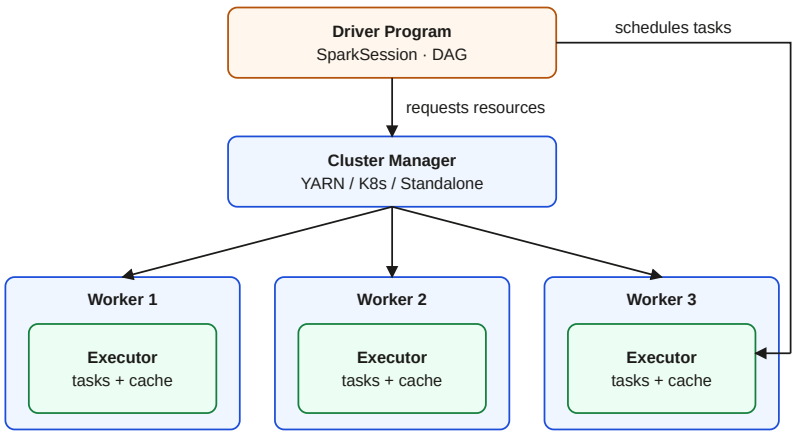
Variants

- D = 20 GB, k = 30, HDFS 400 MB/s, RAM 4,000 MB/s: MR I/O = 30 × 40 = 1,200 GB (60× Spark's 20 GB); T_MR = 30 × 100 = 3,000 s; T_Spark = 50 + 29 × 5 = 195 s; speedup ≈ 15.4×
- Part 2 slide: 10 iterations over 100 GB at 100 MB/s, reads only: 10 × 100,000 / 100 = 10,000 s ≈ 2.8 h (the slide's "≈ 3 hours")
- Per-iteration benchmark (RDD paper): logistic regression 127 s vs 0.96 s → 127 / 0.96 ≈ 132×; K-Means 150 s vs 5 s → 30×

TRAPS

- The 200× is disk I/O, not time. Here RAM is only 10× faster than HDFS, so the time speedup is about 18×.
- Spark iterations 2–100 still read 50 GB each, from RAM. That is 99 RAM passes, not 100.
- If the HDFS write speed is not given, say you assume it equals the read speed.
- Spark does not skip the shuffle. Wide operations like reduceByKey still shuffle.

Spark architecture



The Driver asks the Cluster Manager for resources, then schedules tasks on Executors that run on Worker Nodes and hold the cache.

FACTS

- Three tiers: Driver Program, Cluster Manager, Executors.
- Driver: runs the user's main(), creates SparkContext, builds the DAG (directed acyclic graph of steps) and coordinates execution.
- Cluster Manager: allocates cluster resources. Options: YARN, Kubernetes, Mesos or Spark Standalone.
- Executors: JVM processes on Worker Nodes. They run tasks and store cached RDD partitions in memory.
- Spark 1.x entry point: SparkContext. Spark 2.x+: SparkSession unifies SQL, DataFrames and RDDs.
- In Spark 2.x+ the SparkContext is still reachable as `spark.sparkContext`.

WHY · PROS · LIMITS

- A separate Cluster Manager lets the same Spark program run on YARN, Kubernetes, Mesos or Standalone.
- Executors hold the cache, so data stays in RAM next to the tasks that reuse it.
- The Driver sees the whole DAG, so it can plan stages and know which partitions to rebuild after a failure.
- Cost: every Executor needs memory, about 8 GB minimum.

HOW IT WORKS

1. The Driver starts and creates a SparkSession (and its SparkContext)
2. The Driver asks the Cluster Manager for resources
3. The Cluster Manager launches Executors on the Worker Nodes
4. The Driver turns the program into a DAG, splits it into stages and tasks, and schedules the tasks on Executors
5. Executors run the tasks and keep cached partitions in RAM
6. Results go back to the Driver, or an action writes them to storage
7. If an Executor stops responding, the Driver marks its partitions lost and reschedules the work

COMPARE

	Driver	Executor
How many	One per application	Many, on Worker Nodes
Runs	main(), SparkContext/SparkSession	Tasks sent by the Driver
Job	Builds the DAG, schedules, coordinates	Computes partitions
Holds	The plan (DAG, lineage)	Cached RDD partitions in RAM

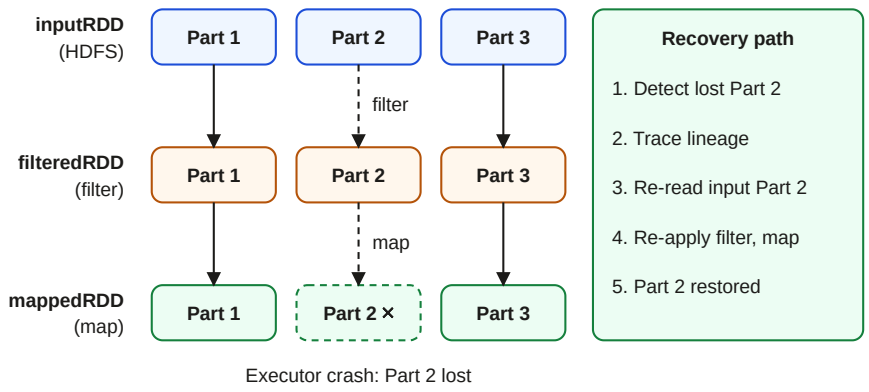
	SparkContext	SparkSession
Version	Spark 1.x entry point	Spark 2.x+ entry point
Covers	RDDs	SQL, DataFrames and RDDs

Access	`SparkContext(...)`	`SparkSession.builder...getOrCreate()` ; context via `spark.sparkContext`

TRAPS

- Spark does not need YARN. Kubernetes, Mesos and Standalone also work.
- Cached data lives in the Executors, not in the Driver.
- SparkContext did not disappear in Spark 2; it sits inside SparkSession.

RDDs and lineage-based fault tolerance



When Part 2 of mappedRDD is lost, Spark follows the lineage back to inputRDD and re-runs filter and map for that one partition.

FACTS

- RDD (Resilient Distributed Dataset): a read-only, partitioned collection of records.
- Created only two ways: by transforming an existing RDD (map, filter) or by loading from stable storage (HDFS, S3).
- Five properties: immutable, partitioned, lazy, lineage, resilient.
- Lineage: the record of transformations that derived an RDD from its parent RDDs.
- Resilient: lost partitions are recomputed from lineage, not replicated. “The computation is the backup.”
- Persistence levels: MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY, MEMORY_ONLY_SER, OFF_HEAP. `cache()` = MEMORY_ONLY.

WHY · PROS · LIMITS

- Replicating in-memory data 3 times would triple RAM use and send every write over the network. Lineage stores only the steps.
- You pay only when a failure happens, and only for the lost partitions.
- Immutability makes replay safe: the same steps on the same input give the same partition.

HOW IT WORKS

1. An Executor crashes and Part 2 of mappedRDD is lost
2. Spark detects the lost partition
3. It traces the lineage: mappedRDD came from filteredRDD, which came from inputRDD
4. It re-reads only Part 2 of the input from HDFS
5. It re-applies filter, then map, to that one partition
6. Part 2 is restored without restarting the whole job

COMPARE

	HDFS replication	RDD lineage
What is kept	3 copies of the data	The list of transformations
Cost with no failure	3× storage and network writes	Almost none
Recovery	Read another copy	Recompute lost partitions from parents

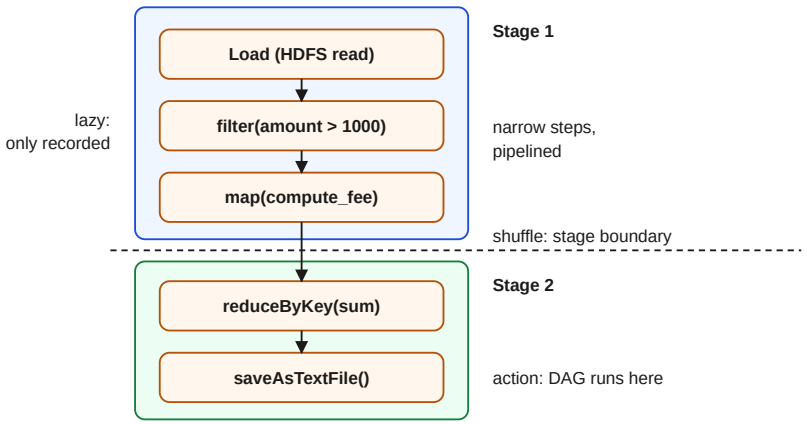
- Limitation: cheap for narrow dependencies, expensive for wide ones. Checkpoint long lineage chains to HDFS.
- Cache an RDD only if more than one action reuses it. Caching single-use RDDs wastes RAM and evicts other data (LRU).

Weak spot	Storage overhead	Wide dependencies and long chains (fix: checkpoint)

TRAPS

- Checkpointing is an optional add-on for long chains. Lineage is the main mechanism.
- Copying each partition to two Executors is replication, which RDDs avoid.
- `cache()` is lazy. Nothing is stored until an action runs.
- Resilient means recomputable, not replicated.

Transformations, actions and lazy evaluation



Transformations are only recorded; the action saveAsTextFile runs the DAG, with narrow steps pipelined in Stage 1 and the reduceByKey shuffle starting Stage 2.

FACTS

- Transformation: builds the DAG and returns a new RDD, without running. map, filter, flatMap, distinct, reduceByKey, join, groupByKey, cache.
- Action: triggers DAG execution and returns a value to the Driver or writes to storage. collect, count, first, take(n), reduce, saveAsTextFile, foreach.
- Rule: no transformation executes until an action is called.
- Lazy evaluation: Spark only records transformations in the DAG and runs them when an action is called.
- Slide example: `rdd.filter(x > 1000).first()` stops after the first matching element.

WHY · PROS · LIMITS

- Spark sees the whole plan before running anything, so it can optimise it.
- Pipelining saves extra passes and intermediate writes.

HOW IT WORKS

1. Each transformation is recorded in the DAG; no data is read
2. An action such as count() or collect() is called
3. Spark analyses the full DAG
4. It applies the Catalyst optimizer (DataFrames) or the DAG scheduler (RDDs)
5. It pipelines compatible steps, e.g. filter and map in one pass over each partition, with no intermediate disk writes
6. It executes only the partitions it needs and returns the result

COMPARE

	Transformation	Action
Runs	Lazily: only recorded	Eagerly: starts the job
Returns	A new RDD	A value to the Driver, or output to storage

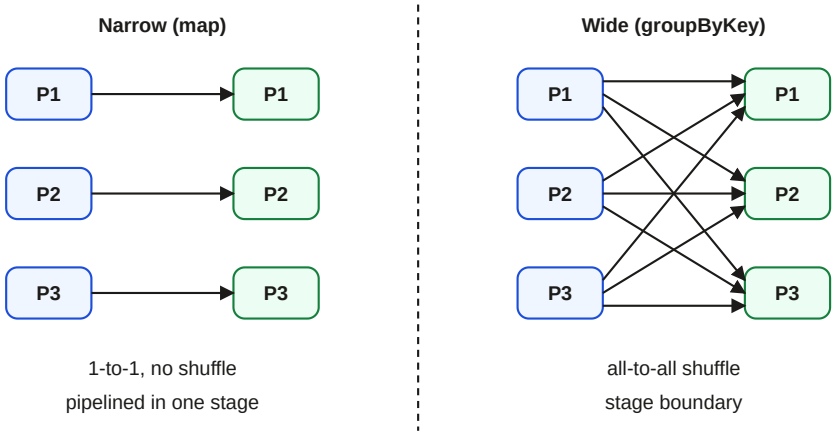
- Early stop: first() and take(n) read only as much as they need.
- The recorded plan is also the lineage that fault recovery replays.

Examples	map, filter, flatMap, reduceByKey, join, cache	count, collect, first, take(n), reduce, saveAsTextFile
Effect on DAG	Adds a node	Executes the DAG

TRAPS

- cache() is a transformation (lazy), not an action.
- reduceByKey is a transformation; reduce is an action.
- map and filter exist in PySpark, not only in Scala.
- A transformation's result is not written to HDFS automatically.

Narrow vs wide dependencies, stages and shuffle



In a narrow dependency each child partition reads one parent partition; in a wide one every child reads from every parent, which forces a shuffle and a new stage.

FACTS

- Narrow dependency: each output partition depends on at most one input partition. map, filter, flatMap, union, mapPartitions, coalesce.
- Wide dependency: an output partition may depend on many input partitions. groupByKey, reduceByKey, aggregateByKey, sortByKey, join, repartition, distinct.
- Wide dependency ⇒ shuffle (data sent across the network) ⇒ stage boundary (a new stage).
- Narrow steps are pipelined in one stage. A lost narrow partition is rebuilt from one parent partition; a wide one needs all parent partitions.
- Prefer reduceByKey over groupByKey: it combines locally before the shuffle, like a MapReduce Combiner.
- Hash partitioner (default): `key.hashCode() % numPartitions`. Co-partitioning two RDDs on the same key removes the join shuffle.

HOW IT WORKS

1. The Driver reads the DAG from the action back to the input
2. Consecutive narrow transformations are grouped into one stage and pipelined
3. Each wide transformation cuts a stage boundary
4. Each stage runs as one task per partition
5. Tasks before the boundary write shuffle output to local disk
6. Tasks after the boundary fetch their keys from every earlier task over the network

WHY · PROS · LIMITS

- A shuffle costs network traffic, disk writes and a barrier where every task must finish. It is Spark's

COMPARE

	Narrow	Wide

main cost.
- Minimising wide dependencies is the main Spark tuning skill.
- reduceByKey sends at most one record per key per partition; groupByKey sends every pair.
- GraySort 2013: Spark sorted 100 TB in 23 min on 206 nodes (Hadoop: 72 min, 2,100 nodes), mainly through a cheaper sort-based shuffle.
- Limitation: some shuffles cannot be avoided, e.g. a global sort or a join on un-partitioned data.

Dependency	1-to-1 (at most one parent partition)	All-to-all (many parent partitions)
Examples	map, filter, flatMap, union, coalesce	groupByKey, reduceByKey, join, repartition, distinct
Data movement	None	Shuffle across the network
Stages	Pipelined in one stage	Starts a new stage
Recovery	Recompute one parent partition	Recompute from all parent partitions

	reduceByKey	**groupByKey**
Local combine	Yes, before the shuffle	No
Records shuffled	At most one per key per partition	Every (key, value) pair
Result	One value per key	All values per key (list)

CALCULATION

A2: input → filteredRDD → mappedRDD → reducedRDD (groupByKey), 10 partitions. The Executor with reducedRDD partitions 3 and 7 crashes.

Case	**Work redone**
All narrow (comparison)	Partitions 3 and 7 only: 2 / 10 = 20%; 2 × 4 = 8 partition computations
Wide, shuffle files survive	Only the 2 Stage-2 tasks, re-fetching from all 10 map outputs
Wide, worst case (shuffle files lost)	Stage 1 again: 10 input + 10 filter + 10 map = 30, plus 2 reduce tasks

Shuffle size: 4 partitions × 1,000 pairs, 50 distinct words each. groupByKey 4,000 records; reduceByKey ≤ 4 × 50 = 200, 95% less.

Step by step

1. Given filter and map are narrow and groupByKey is wide → Stage 1 = input → filter → map; Stage 2 = groupByKey
2. Given partitions 3 and 7 lost with only narrow steps → redo 2 / 10 = 20% of the work
3. Given groupByKey → each lost partition needs data from all 10 mappedRDD partitions
4. Given shuffle files lost → rerun Stage 1: 10 + 10 + 10 = 30 partition computations, then 2 reduce tasks
5. Given 4 partitions × 1,000 pairs → groupByKey shuffles 4,000 records
6. Given 50 distinct words per partition → reduceByKey shuffles at most 4 × 50 = 200, i.e. 1 − 200/4,000 = 95% less
7. Given numPartitions = 4 and a key with hash 17 → 17 % 4 = 1, so partition 1

Variants

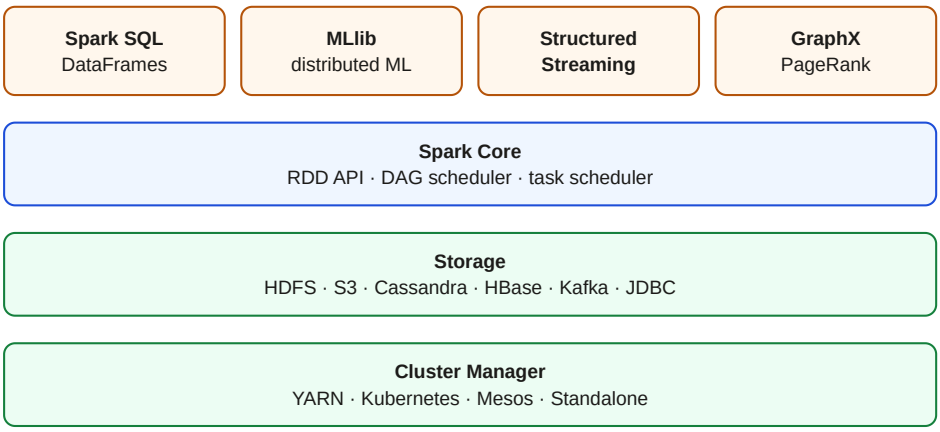
- Hashes 42, 10, 23, 31 with numPartitions = 4 → partitions 2, 2, 3, 3
- 3 of 12 partitions lost, all narrow → 3 / 12 = 25% of the work redone
- 8 partitions × 500 pairs, 20 distinct keys each → groupByKey 4,000; reduceByKey ≤ 160; 96% less

- GraySort ratios: 72 / 23 ≈ 3.1× faster; 2,100 / 206 ≈ 10.2× fewer nodes; (72 × 2,100) / (23 × 206) = 151,200 / 4,738 ≈ 32× better efficiency per node

TRAPS

- reduceByKey is still wide. It shuffles, only less data.
- coalesce is narrow (no shuffle); repartition is wide.
- join is narrow only when both RDDs are co-partitioned on the key.
- Part 2 slide 2 says “3.5× fewer machines”, but 2,100 / 206 ≈ 10.2×, as slide 14 says.

DataFrames, Spark SQL and the ecosystem



Spark SQL, MLlib, Structured Streaming and GraphX all run on Spark Core, which reads from storage and gets resources from a Cluster Manager.

FACTS

- DataFrame: a distributed collection organised into named columns with a schema, like a database table or a distributed Pandas DataFrame.
- Built on RDDs internally; every DataFrame operation compiles to an RDD plan when it runs.
- Catalyst optimizer rewrites query plans: predicate pushdown, join reordering, constant folding.
- DataFrames are 10–100× faster than RDDs for structured data. Use them unless you need low-level control.
- Supports SQL and method chaining. Reads Parquet, ORC, CSV, JSON, Hive tables, JDBC.
- On Spark Core: Spark SQL, MLlib (ML), Structured Streaming, GraphX (graphs). HBase is a storage source, not a component.

WHY · PROS · LIMITS

- The schema tells Catalyst what each column is, so it can optimise the query; raw RDD lambdas are opaque to it.
- One engine covers SQL, ML, streaming and graphs, so there is no separate system per workload.
- MLlib trains iteratively on cached data, which is where Spark is strongest.

HOW IT WORKS

1. Load the data: `df = spark.read.parquet(path)`
2. Register a SQL view: `df.createOrReplaceTempView("health_records")`
3. Run the query: `spark.sql("SELECT ... GROUP BY ...")`
4. Catalyst rewrites the plan, e.g. pushes the WHERE filter down to the scan
5. The optimised plan runs as RDD stages on the Executors
6. Write the result: `result.write.mode("overwrite").csv(path)`

COMPARE

	DataFrame	RDD
Structure	Named columns + schema	Records with no schema
Optimiser	Catalyst	DAG scheduler only
Speed (structured data)	10–100× faster	Baseline

- Limitation: DataFrames give less low-level control than RDDs.

API	SQL + method chaining	Functional (map, filter, lambdas)
Use when	Default for structured data	Low-level control needed

	Does	Example
Spark SQL	Structured queries on DataFrames	District-wise disease counts
MLlib	Distributed ML in memory	k-means, logistic regression, random forest
Structured Streaming	Stream as an ever-growing table; watermarks, exactly-once with Kafka	Real-time events
GraphX	Graph algorithms	PageRank, connected components
HBase	Not Spark: a NoSQL store Spark can read	Data source

TRAPS

- HBase is not a Spark ecosystem component.
- DataFrames do not replace the engine: they compile down to RDDs.
- Catalyst optimises DataFrames/SQL; RDD code goes through the DAG scheduler only.

CHAPTER 3 · WEEK 7

Hive, HiveQL and SQL-on-Hadoop

Hive lets analysts query data on HDFS with SQL-like HiveQL, which it compiles into MapReduce, Tez or Spark jobs, using a Metastore that records every table's schema and location. The chapter covers how Hive runs a query, schema-on-read, partitioning and bucketing for speed, Presto for interactive queries, and Spotify's Discover Weekly pipeline as a case study.

Exam question pool

21 questions from the slides · click a question for its answer

MCQ **Q1.** In Hive, where is table schema metadata (column names, types, partition info) stored?

(a) HDFS NameNode
(b) Hive Metastore (relational DB such as MySQL)
(c) HBase
(d) ZooKeeper

(b) Hive Metastore (relational DB such as MySQL)
The NameNode holds HDFS block metadata, not table schemas.

MCQ **Q2.** What does "schema-on-read" mean in the context of Hive?

(a) Schema is validated when data is inserted into the table
(b) Schema is applied when data is queried, not when it is stored on HDFS
(c) Schema is stored separately from data in HDFS metadata
(d) Schema is inferred automatically from column names in the file header

(b) Schema is applied when data is queried, not when it is stored on HDFS
(a) describes schema-on-write, how MySQL works.

MCQ **Q3.** Which Hive optimization technique physically organises data into separate HDFS sub-directories by a column's value to avoid full table scans?

(a) Bucketing
(b) Indexing
(c) Partitioning
(d) Compression

(c) Partitioning
Bucketing splits files inside a partition; it does not skip directories.

MCQ **Q4.** Presto's primary performance advantage over Hive+MapReduce for interactive queries is:

(a) Presto supports more SQL functions than HiveQL
(b) Presto has better native integration with HBase and Cassandra
(c) Presto executes queries entirely in memory without writing intermediate results to disk
(d) Presto has native Python UDF support

(c) Presto executes queries entirely in memory without writing intermediate results to disk
Hive+MR writes to HDFS between every stage.

TRUE/FALSE **1.** HiveQL supports full ACID transactions on all table types, just like MySQL.

False. Corrected: HiveQL supports ACID only on ACID-enabled transactional ORC tables; MySQL (InnoDB engine) gives full ACID: INSERT, UPDATE, DELETE and ROLLBACK on any row.

TRUE/FALSE **2.** Partitioning in Hive reduces query time by eliminating irrelevant data partitions from the HDFS scan.

True. A WHERE filter on the partition key makes Hive read only the matching sub-directories (partition pruning).

TRUE/FALSE **3.** Hive directly executes a HiveQL query on HDFS without any intermediate translation step.

False. Corrected: Hive compiles HiveQL (parse, analyse, plan, optimise) into MapReduce, Tez or Spark jobs, which then read HDFS.

TRUE/FALSE **4.** Bucketing in Hive is particularly useful for optimising JOIN operations between two large tables.

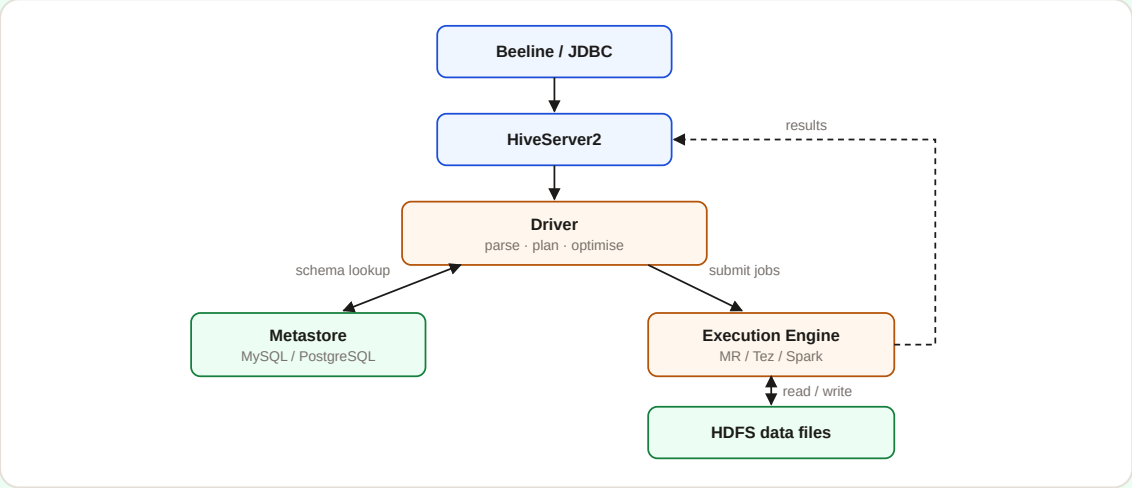
True. Both tables bucketed on the join key with the same N let Hive join bucket i with bucket i, with no full shuffle.

DESCRIPTIVE **D1.** Explain the Hive architecture: what is the role of HiveServer2, the Driver, and the Metastore? How is a HiveQL query compiled into executable jobs? Draw the component diagram.

- **HiveServer2**: JDBC/ODBC front door; authenticates users, accepts HiveQL from Beeline, BI tools, JDBC.
- **Driver**: Parser → Semantic Analyser → Logical Planner → Optimizer → Physical Plan Generator; manages the query lifecycle.
- **Metastore**: MySQL/PostgreSQL catalogue of schemas, types, partitions, HDFS paths.

Compilation: parse to AST → check names and types in the Metastore → logical plan (Filter, Join, GroupBy, OrderBy) → optimise (partition pruning) → MR/Tez/Spark jobs → submit to YARN; results to HDFS and client.

Diagram:



DESCRIPTIVE **D2.** Differentiate between partitioning and bucketing in Hive. When would you choose each? Describe a scenario in Bangladesh e-commerce data where both are used together.

	Partitioning	Bucketing
Unit	Directory per value	Fixed N files, hash(col) % N
Benefit	Pruning for WHERE filters	Bucket map join
Column	Low cardinality (date, division)	High cardinality key (customer_id)

Choose partitioning when queries filter on a few-valued column; bucketing when large tables are joined on a many-valued key.

Scenario: a Bangladeshi e-commerce `orders` table, `PARTITIONED BY (order_date, division)` and `CLUSTERED BY (customer_id) INTO 32 BUCKETS`. The Dhaka daily report reads one folder; the join with `customers` (also 32 buckets on customer_id) pairs bucket i with i.

DESCRIPTIVE **D3.** What is "schema-on-read"? What are its advantages and disadvantages compared to "schema-on-write" in a traditional RDBMS such as MySQL? Give one example where schema-on-read causes a data quality problem.

Schema-on-read: Hive applies column types at query time; loading is a raw file copy. MySQL (schema-on-write) checks each row at INSERT.

Advantages	Disadvantages
Fast load, no validation	Slower queries (parse every time)
Load before the schema is known; many schemas on the same files; no ETL at ingest	Bad data silent: becomes NULL
Open formats (ORC, Parquet, CSV, JSON, Avro)	Unfit for OLTP (banking, orders)

Example: `abc` in a DOUBLE `amount` column. MySQL rejects the INSERT; Hive stores it and returns NULL, so `SUM(amount)` under-reports revenue with no warning. Fix: Great Expectations or dbt tests downstream.

ANALYTICAL

A1. Partitioning & Bucketing Design: You have a 5 TB e-commerce transaction table with columns: (txn_id, user_id, product_id, amount, country, txn_date). Design a complete partitioning and bucketing strategy to optimise: – Daily revenue reports filtered by country and txn_date – JOINs between this table and a 10 M-row user profile table (keyed on user_id). Write the complete CREATE TABLE DDL. Justify why you chose a specific number of buckets and which column(s) you partitioned on.

Partition on `txn_date, country` (low cardinality, used in report filters). **Bucket** on `user_id` (high-cardinality join key), same N on both tables for a bucket map join. Never partition on user_id or txn_id.

Bucket count (assume 3 years, 5 countries):

Partitions	3 × 365 × 5 = 5,475
Per partition	5,000 GB / 5,475 ≈ 0.913 GB
8 buckets	≈ 114 MB/file (near 128 MB block), 43,800 files
32 buckets	≈ 28.5 MB/file, 175,200 files: too small

```
CREATE TABLE transactions (
  txn_id BIGINT, user_id BIGINT,
  product_id BIGINT, amount DOUBLE)
PARTITIONED BY (txn_date STRING, country STRING)
CLUSTERED BY (user_id) SORTED BY (user_id) INTO 8 BUCKETS
STORED AS ORC;

CREATE TABLE user_profile (
  user_id BIGINT, name STRING, city STRING)
CLUSTERED BY (user_id) SORTED BY (user_id) INTO 8 BUCKETS
STORED AS ORC;
```

Result: a one-day, one-country report reads 1 of 5,475 partitions (≈ 0.018%); the join pairs bucket i with i.

ANALYTICAL

A2. Query Compilation to MapReduce: Write HiveQL to find the top 5 products by total revenue in 2023. Then trace how Hive compiles this query into MapReduce stages: what is emitted by the Map phase, what is the shuffle key, and what does the Reduce phase compute? How would switching the execution engine from MapReduce to Tez change the execution plan?

```
SELECT product_id, SUM(amount) AS total_revenue
FROM transactions
WHERE YEAR(txn_date) = 2023
GROUP BY product_id
ORDER BY total_revenue DESC
LIMIT 5;
```

	Job 1: aggregate	**Job 2: sort + limit**
Map emits	(product_id, amount) for 2023 rows; combiner gives partial sums	(total_revenue, product_id); each mapper keeps its top 5
Shuffle key	product_id	total_revenue DESC, to one reducer
Reduce	SUM(amount) per product, written to HDFS	First 5 rows

Tiny check: (P1,100), (P2,50), (P1,30), (P3,80), (P2,20) → P1 130, P3 80, P2 70.

Tez: one DAG, Map 1 → Reducer 2 → Reducer 3. No HDFS write between the jobs, one submission, container reuse. Same result, faster.

DISCUSSION **1.** Spotify's co-occurrence query runs GROUP BY song_a, song_b over 2 billion playlist rows. Estimate the output size if there are 30M songs. Propose a partitioning + bucketing scheme for the playlist_songs Hive table that would reduce the shuffle size for this query. [CLO4 – Apply]

Claim: output is far below the 30M-song bound. Bound: 30M × (30M − 1) / 2 ≈ 4.5 × 10^14 pairs. Realistic (assume 20 songs per playlist): 10^8 playlists × 190 = 1.9 × 10^10 pairs, ≤ 304 GB at 16 B/row, much less after HAVING > 5.

Scheme: `PARTITIONED BY (week)`, `CLUSTERED BY (playlist_id) SORTED BY (playlist_id, song_id) INTO 1024 BUCKETS`. **For:** the self-join runs inside each bucket with no shuffle, and weekly partitions cut 8 h to 2 h. **Against:** it does not shrink the GROUP BY shuffle on song pairs; rely on map-side aggregation there.

DISCUSSION **2.** The ALS collaborative filtering model is a matrix factorisation over a 75M-user × 30M-song matrix. At 4 bytes per float and rank k = 50, calculate the model size and explain whether it fits in a 5,000-node YARN cluster's aggregate RAM. [CLO4 – Analyse]

Size: (75M + 30M) × 50 × 4 B = **21 GB** (users 15 GB + songs 6 GB).

Fits? Assume 64 GB/node: 5,000 × 64 GB = 320 TB; 21 GB ≈ 0.0066%, about 4.2 MB per node. **For:** it fits easily, even on one large server; the dense 9 PB matrix would not. **Against:** training also holds the interaction data, shuffled factor blocks and JVM overhead, several times the model size, but still well within the cluster.

DISCUSSION **3.** Spotify uses Luigi while other companies use Apache Airflow. Compare the two on: dependency declaration style, backfill capability, and UI/observability. Which would you choose for a 100-step daily pipeline? [CLO3 – Evaluate]

Claim: choose Airflow for a 100-step daily pipeline. Luigi: target-based `requires()/output()`, skips finished outputs, backfill by date parameters, simple UI. Airflow: DAG with a scheduler, built-in catchup/backfill, rich UI with logs, retries and alerts.

For: at 100 steps you need to see which step failed, read its logs and re-run past days, which Airflow gives out of the box. **Against:** Luigi is lighter and its skip-if-output-exists model fits file-based Hadoop pipelines like Spotify's.

LIVE EXERCISE

Task. Task (25 minutes, pairs): Simulate a Hive session using PySpark's SQL interface (which reads the same Hive Metastore). Dataset: a 10,000-row synthetic play_events CSV (generate with the skeleton below).

```
from pyspark.sql import SparkSession
import random, datetime

spark = SparkSession.builder \
        .appName("HiveDemo") \
        .getOrCreate()
spark.setLogLevel("WARN")

# Generate synthetic data
rows = []
for i in range(10_000):
    rows.append({
      "user_id":    random.randint(1, 500),
      "song_id":    random.randint(1, 200),
      "playlist_id":random.randint(1, 1000),
      "dt":         "2024-10-01"
    })

df = spark.createDataFrame(rows)
df.createOrReplaceTempView("play_events")

# Co-occurrence query
result = spark.sql("""
    SELECT a.song_id AS song_a,
           b.song_id AS song_b,
           COUNT(*) AS co_count
    FROM play_events a
    JOIN play_events b
      ON a.playlist_id = b.playlist_id
     AND a.song_id < b.song_id
    GROUP BY a.song_id, b.song_id
    HAVING COUNT(*) > 2
    ORDER BY co_count DESC
""")
result.show(10)
```

Write the data as a real partitioned table, then time and inspect the queries.

```
import time
random.seed(42)  # rows as in the skeleton, dt from 2 dates
df.write.mode("overwrite").partitionBy("dt") \
  .format("parquet").saveAsTable("play_events_part")
t0 = time.time()
spark.sql(cooc_sql).show(10)      # skeleton query on play_events_part, with a.dt = b.dt = '2024-10-01'
print(time.time() - t0)
songs = spark.createDataFrame(
  [(i, "genre" + str(i % 10)) for i in range(1, 201)], ["song_id", "genre"])
songs.createOrReplaceTempView("songs")   # 200-row dimension table
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
hint_sql = """SELECT /*+ BROADCAST(s) */ s.genre, COUNT(*)
  FROM play_events_part p JOIN songs s ON p.song_id = s.song_id
  WHERE p.dt = '2024-10-01' GROUP BY s.genre"""
spark.sql(hint_sql).explain(mode="formatted")
```

Expected with the single-date skeleton: about 10 rows per playlist, about 50,000 pair rows, about 9,000 pairs kept by HAVING > 2, top count about 11 to 14 (simulated; varies with the data).

LIVE EXERCISE

1. Create a partitioned Spark table (partition on dt).

`df.write.partitionBy("dt").saveAsTable("play_events_part")` writes one sub-directory per date, e.g. `play_events_part/dt=2024-10-01/`, like Hive's `year=2024/`. Expected `SHOW PARTITIONS`: one row per date.

LIVE EXERCISE **2.** Run the co-occurrence query and time it.

Wrap an action (`show` , `count`) in `time.time()` : Spark is lazy, so `spark.sql()` alone runs nothing. Expected: a few seconds on a laptop, mostly start-up and the join/GROUP BY shuffle. The self-join grows with the square of playlist length (10 songs give 45 pairs).

LIVE EXERCISE **3.** Add a broadcast join hint for the songs dimension table and compare timings.

`/*+ BROADCAST(s) */` copies the 200-row songs table to every executor, so the big table is not shuffled (Hive's map-side join). The plan changes from SortMergeJoin to BroadcastHashJoin. Expected: slightly faster on 10,000 rows. Spark broadcasts tables under 10 MB automatically, so set the threshold to -1 to compare.

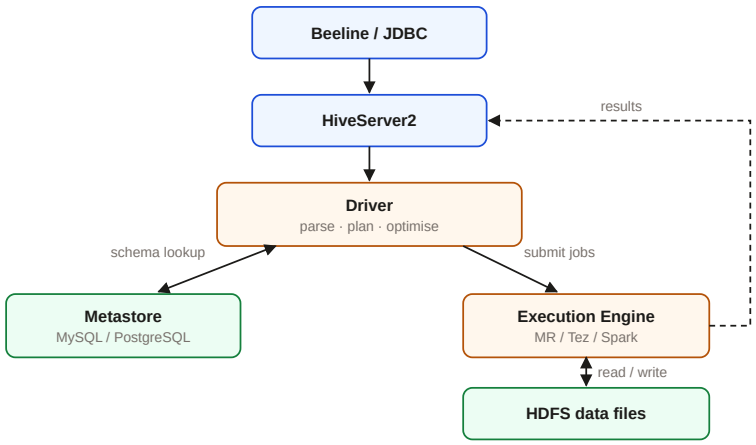
LIVE EXERCISE **4.** Use EXPLAIN to print the physical plan and identify where partition pruning occurs.

Run `EXPLAIN FORMATTED` or `df.explain(mode="formatted")` . Pruning shows in the FileScan node as `PartitionFilters: [isnotnull(dt), (dt = 2024-10-01)]` : only that folder is read. Filters on ordinary columns appear as `PushedFilters` . A temp view built in memory shows no PartitionFilters.

Topic cards

6 topics · everything the pool tests

Hive architecture and query compilation



A HiveQL query enters through HiveServer2; the Driver looks up the schema in the Metastore and submits jobs to the execution engine, which reads and writes HDFS.

FACTS

- Apache Hive (Facebook, 2008) is a data warehouse layer on Hadoop. You write HiveQL, a SQL-like language; Hive turns it into distributed jobs.
- **HiveServer2**: the JDBC/ODBC server. It authenticates users and accepts HiveQL from Beeline, BI tools (Tableau, Power BI) and JDBC drivers.
- **Driver**: the query pipeline Parser → Semantic Analyser → Logical Planner → Query Optimizer → Physical Plan Generator.
- **Metastore**: a relational database (MySQL/PostgreSQL) holding table schemas, column types, partition values with HDFS paths, bucket counts, SerDe, statistics, privileges.
- **SerDe** (Serializer/Deserializer): turns raw bytes on HDFS (ORC, Parquet, CSV, JSON, Avro) into

HOW IT WORKS

1. The client (Beeline, JDBC, Tableau) sends HiveQL to HiveServer2, which authenticates the user.
2. The parser turns the query text into an abstract syntax tree (AST), a tree of the query's parts.
3. Semantic analysis looks up every table and column in the Metastore and checks types (e.g. user_id is BIGINT, dt is a partition column).
4. The logical planner builds operators: Filter → Join → GroupBy → OrderBy.
5. The optimizer adds partition pruning and map-side aggregation; the physical plan generator turns the plan into MR, Tez or Spark jobs.
6. The execution engine submits the jobs to YARN in dependency order; they read and write HDFS.

Hive rows and back.

- **Execution engine** is pluggable: MapReduce (legacy default), Tez (DAG of tasks, faster) or Spark (`SET hive.execution.engine=spark`). HDFS holds data: table = directory.

WHY · PROS · LIMITS

- Before Hive, a simple GROUP BY was a 100+ line Java MapReduce program and an answer took days. Analysts know SQL, not Java.
- Declare, don't program: the analyst writes SELECT and Hive writes the distributed job.
- The Metastore keeps the schema apart from the files, so Spark, Presto/Trino and Impala read the same tables. It is Hive's most lasting contribution.
- If the Metastore is lost, no table can be queried even though HDFS data is intact. Production runs it highly available on replicated MySQL.
- Limitation: on MapReduce every stage writes to HDFS, so even simple queries take minutes.

7. The result goes back to the client through HiveServer2.

COMPARE

	MapReduce	Tez
Plan shape	Chain of separate MR jobs	One DAG of tasks
Between stages	Writes to HDFS, next job reads it back	Passes data directly
Top-5 products query	2 job submissions	1 DAG: Map 1 → Reducer 2 → Reducer 3
Start-up cost	New containers per job	Containers can be reused
Result	Same	Same; only plan and speed change

	Hive Metastore	HDFS NameNode
Stores	Table schemas, types, partitions, HDFS paths, SerDe	File-to-block map and block locations
What it is	Relational DB (MySQL/PostgreSQL)	HDFS master daemon
If lost	No table can be queried	No file can be found

CALCULATION

P2 slide 10: `SELECT country, COUNT(*) FROM logs GROUP BY country`, 50 million rows, 200 countries. Without map-side aggregation the mappers emit one (country, 1) row per input row: 50,000,000. With it they emit one partial count per country: 200. Reduction = 50,000,000 / 200 = **250,000×** less shuffle data.

Step by step

1. Given 50,000,000 rows and no combiner → shuffle rows = 50,000,000
2. Given 200 countries and one mapper with map-side aggregation → shuffle rows = 200
3. Given 50,000,000 and 200 → reduction = 50,000,000 / 200 = 250,000×
4. Given M mappers that each see all 200 countries → shuffle = M × 200 rows, reduction = 50,000,000 / (M × 200)

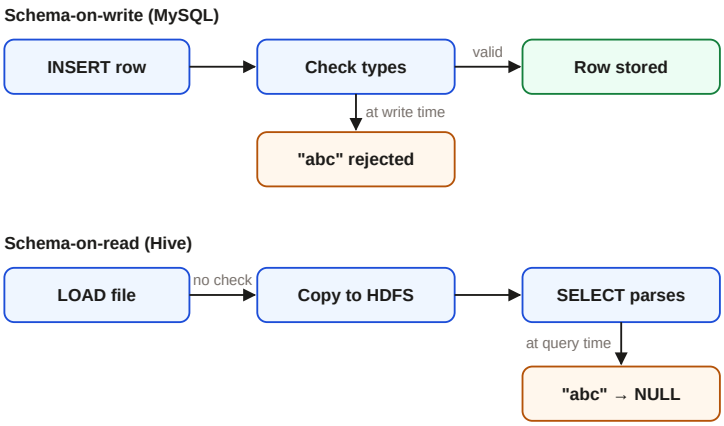
Variants

- 120 million rows, 400 regions: 120,000,000 / 400 = 300,000×
- 50 million rows, 10 mappers each seeing all 200 countries: shuffle = 2,000 rows, reduction = 25,000×

TRAPS

- Hive never runs HiveQL directly on HDFS. It compiles the query to MR, Tez or Spark jobs first.
- Schemas live in the Metastore, not the NameNode. The NameNode only knows files and blocks.
- Switching to Tez changes the plan and the speed, never the answer.
- ORDER BY needs one reducer for a global order, so the final sort stage is a bottleneck.

Schema-on-read and HiveQL vs SQL



Schema-on-write checks types at INSERT and rejects bad rows; schema-on-read copies the file unchecked and turns a bad value into NULL at query time.

FACTS

- **Schema-on-read**: Hive applies column types at query (SELECT) time. Loading just copies raw files to HDFS.
- **Schema-on-write**: an RDBMS such as MySQL checks each row at INSERT time and rejects bad rows.
- Under schema-on-read, text `abc` in a DOUBLE `amount` column is stored, and the query returns NULL.
- HiveQL ACID is limited: UPDATE/DELETE work only on ACID-enabled transactional ORC tables.
- Hive has no B-tree indexes (it uses partitions and buckets) and supports only non-correlated sub-queries.
- Latency: SQL milliseconds; Hive minutes on MR, seconds on Tez/Spark. `COUNT(*)` on 1 TB can take 5 to 10 minutes.

WHY · PROS · LIMITS

- Fast loading: a plain file copy, no validation cost.
- Flexible: load raw web logs before the final schema is known, and define several tables over the same files. No ETL at ingest.
- Open file formats: ORC, Parquet, CSV, JSON, Avro.
- Limitation: every query parses and checks the data again, so queries are slower.
- Limitation: bad data stays silent. Add downstream checks (Great Expectations, dbt tests).

HOW IT WORKS

1. CREATE TABLE writes column names, types and the SerDe to the Metastore. No data is checked.
2. LOAD DATA moves the file into the table's HDFS directory unchanged.
3. At SELECT time the SerDe reads each record and splits it into fields.
4. Each field is cast to its declared type. A failed cast becomes NULL.
5. Aggregates such as SUM skip NULLs, so a wrong total comes back with no error.

COMPARE

	Schema-on-write (MySQL)	Schema-on-read (Hive)
Schema enforced	At INSERT / LOAD	At SELECT
Invalid data	Rejected immediately	Stored; NULL at query
Write speed	Slower (validation)	Fast (raw file copy)
Query speed	Faster (pre-validated)	Slower (parse each query)
Schemas per data	One per table	Many over the same files
Use case	OLTP: orders, banking	OLAP: log analysis, batch reports

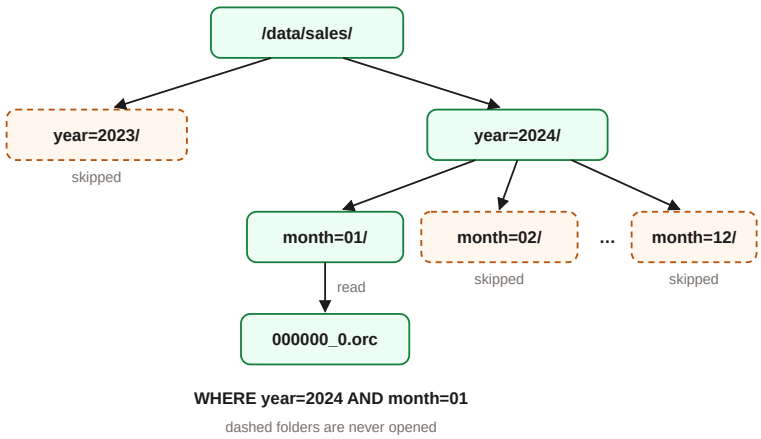
	SQL (RDBMS)	HiveQL

ACID	Full: INSERT, UPDATE, DELETE, ROLLBACK	Only ACID-mode ORC tables
UPDATE / DELETE	Any row, any time	Only transactional tables
Indexing	B-tree, hash, bitmap	Partitions + buckets
Latency	Milliseconds	Minutes (MR) / seconds (Tez, Spark)
Scale	Terabytes	Petabytes
Workload	OLTP (real-time reads/writes)	OLAP (batch analytics)

TRAPS

- Schema-on-read does not mean the schema is inferred. You still write CREATE TABLE (MCQ 2 option d).
- The schema is stored separately under both models. The key word is *when* it is applied.
- HiveQL looks like SQL but runs as batch jobs: minutes, not milliseconds.
- Hive ACID is not on all tables, only transactional ORC tables.

Partitioning and partition pruning



With WHERE year=2024 AND month=01, Hive reads only the month=01/ folder and never opens the other partition directories.

FACTS

- **Partitioning** stores table data in separate HDFS sub-directories by column value, e.g. `/data/sales/year=2024/month=01/`.
- **Partition pruning**: when WHERE filters on the partition key, Hive reads only the matching sub-directories and skips the rest.
- Good key: a low-cardinality column (few distinct values) used often in WHERE: date, country, region, status.
- Never partition on high-cardinality columns (user_id, txn_id): 100 M users means 100 M directories, crashing the NameNode.
- DDL: `PARTITIONED BY (year INT, month INT)`. Partition columns are not repeated in the column list.

HOW IT WORKS

1. Create the table with `PARTITIONED BY (year INT, month INT)`.
2. Load: a static insert names the values, `PARTITION (year=2024, month=1)`; a dynamic insert takes them from the last SELECT columns.
3. Hive writes each partition's files to its own sub-directory and records the value and path in the Metastore.
4. A query arrives with `WHERE year=2024 AND month=1`.
5. The optimizer checks the Metastore's partition list and keeps only matching paths.
6. Mappers read only `year=2024/month=01/`; every other directory is skipped.

- Dynamic insert needs `hive.exec.dynamic.partition=true` and `mode=nonstrict`; partition columns go last in SELECT.

WHY · PROS · LIMITS

- Avoids a full table scan, so data read and job time drop in proportion.
- Pruning uses the partition list in the Metastore, so Hive need not list every file.
- Limitation: it helps only when the query filters on the partition column.
- Limitation: too many partitions create millions of directories and small files that overload the NameNode.

COMPARE

	Static partition insert	Dynamic partition insert
Partition values	Written in PARTITION (year=2024, month=1)	Taken from the last SELECT columns
Settings	None	dynamic.partition=true, mode=nonstrict
Partitions per statement	One	All at once (e.g. 36 months)
Typical slip	Wrong value typed	Columns out of order: values mis-assigned silently

CALCULATION

P2 slide 11: `play_events` partitioned by dt (365 days) and country (200). P_total = 365 × 200 = 73,000. The query reads dt='2024-10-01' for BD, IN, US: P_read = 3. Fraction read = 3 / 73,000 ≈ **0.0041%**; 72,997 partitions skipped. P1 slide 20: one month of 36 = 1/36 ≈ 2.78%.

Step by step

1. Given 365 dates and 200 countries → P_total = 365 × 200 = 73,000
2. Given dt = one day and country IN (BD, IN, US) → P_read = 1 × 3 = 3
3. Given 3 of 73,000 (equal-size partitions) → fraction = 3 / 73,000 ≈ 0.0041%
4. Given 73,000 total and 3 read → skipped = 72,997

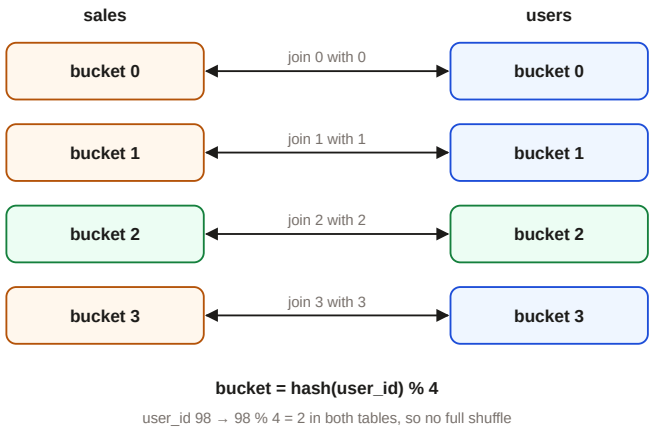
Variants

- 36 months, query one month: 1 / 36 ≈ 2.78% read
- 24 months × 50 countries, one month for 2 countries: 2 / 1,200 ≈ 0.167% read

TRAPS

- Partitioning splits by directory; bucketing splits into files inside a partition. Only partitioning skips directories.
- In a dynamic insert, partition columns must be last in SELECT and in PARTITION order, or Hive assigns values wrongly without error.
- Pruning needs a filter on the partition column itself. A filter on any other column scans every partition.
- The fraction formula assumes equal-size partitions. Say so.

Bucketing and bucket map join



Both tables bucketed on user_id into 4 buckets, so bucket i joins only bucket i (bucket map join) with no full shuffle.

FACTS

- **Bucketing** splits each partition into a fixed number N of files (buckets) by hashing a chosen column.
- A row goes to bucket `hash(column) % N`. For an integer key the hash is the value itself.
- DDL: `CLUSTERED BY (user_id) INTO 32 BUCKETS STORED AS ORC`. Add `SORTED BY` for a sort-merge bucket join.
- **Bucket map join**: both tables bucketed on the join key with the same N; Hive joins bucket i with bucket i, no full shuffle.
- Good bucket column: a high-cardinality join key such as user_id.
- The Metastore stores the bucket count per table; Hive uses it to decide whether a join can skip the shuffle.

WHY · PROS · LIMITS

- Joining two large tables no longer shuffles both of them, a large speed-up.
- Works on high-cardinality keys, where partitioning would explode into millions of directories.
- Limitation: both tables need the same key and the same bucket count, or the benefit is lost.
- Limitation: too many buckets give small files. Aim for files near the 128 MB HDFS block.

HOW IT WORKS

1. Declare `CLUSTERED BY (user_id) INTO N BUCKETS` on both tables, with the same N.
2. On insert, Hive computes `hash(user_id) % N` for each row.
3. The row is written to bucket file k, where k is that remainder.
4. The same key gives the same k in both tables, so matching rows sit in matching files.
5. At join time Hive reads the bucket counts from the Metastore and pairs bucket 0 with 0, 1 with 1, and so on.
6. Each pair is joined locally, with no full sort-merge shuffle.

COMPARE

	Partitioning	Bucketing
Physical unit	Directory per value	Fixed number of files
Decided by	Column value	hash(column) % N
Main benefit	Skips data for WHERE filters (pruning)	Faster joins (bucket map join)
Good column	Low cardinality: date, country	High cardinality join key: user_id
Risk	Too many directories on high-cardinality keys	Mismatched key or bucket count kills the benefit
DDL	`PARTITIONED BY (year INT)`	`CLUSTERED BY (user_id) INTO 32 BUCKETS`

CALCULATION

Bucket number (P1 slide 24, 4 buckets on user_id): user_id 98 → 98 % 4 = **2**, in both sales and users.

File size (A1, assume 3 years and 5 countries): P = 3 × 365 × 5 = 5,475 partitions; 5 TB / 5,475 ≈ 0.913 GB each; with 8 buckets ≈ **114 MB** per file (near a 128 MB block), 5,475 × 8 = 43,800 files.

Step by step

1. Given user_id = 98, N = 4 → bucket = 98 % 4 = 2
2. Given 3 years, 365 days, 5 countries → P = 1,095 × 5 = 5,475
3. Given 5 TB = 5,000 GB and P = 5,475 → ≈ 0.913 GB per partition
4. Given 0.913 GB and N = 8 → ≈ 114 MB per bucket file
5. Given P = 5,475 and N = 8 → files = 43,800

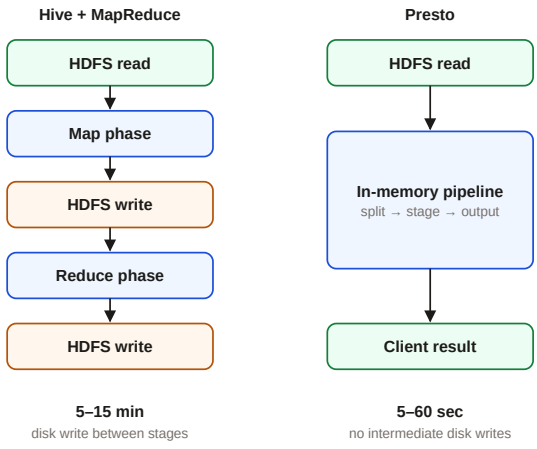
Variants

- user_id 1,234,567 with 32 buckets: 1,234,567 % 32 = 7
- N = 32 in the 5 TB design: ≈ 28.5 MB per file and 175,200 files: too small
- 2 TB, 730 daily partitions, 16 buckets: 2,000 GB / 11,680 ≈ 171 MB per file

TRAPS

- Bucketing does not skip directories. It cuts join cost.
- Buckets are numbered from 0: with 4 buckets the files are 0 to 3.
- Same key and same bucket count on both tables, or no bucket map join.
- The 5 TB question gives no years or countries. State your assumptions before sizing.

Presto and the SQL-on-Hadoop engines



Hive+MapReduce writes to HDFS between stages and takes minutes; Presto streams data through an in-memory pipeline and answers in seconds.

FACTS

- **Presto** (Meta, 2013; fork: Trino) runs interactive SQL over petabytes on HDFS: 5 to 60 seconds versus Hive+MR's 5 to 15 minutes.
- No intermediate disk writes: data flows through an in-memory pipeline (split → stage → output).
- Vectorized execution: processes column chunks of 1,024 rows at a time, not row by row.

HOW IT WORKS

1. Hive+MR reads HDFS and runs the map phase.
2. It writes the map output to disk.
3. It runs the reduce phase and writes the result to HDFS, again for every stage.
4. Presto reads HDFS splits into worker memory.
5. It streams the data through its stages in memory, in batches of 1,024 rows.

- Cost-based optimizer: uses table statistics to pick join order and join strategy.
- Connector architecture: one query can read HDFS, Hive, Cassandra, MySQL, Kafka and S3 together.
- Other engines: Impala (Cloudera, 2012, native C++ MPP), Spark SQL (2014, Catalyst optimizer), Drill (MapR, 2012, schema-free JSON).

6. It returns the result to the client with no intermediate disk write.

WHY · PROS · LIMITS

- Disk writes between stages are the main cost of Hive+MR; Presto removes them.
- Dashboards and ad-hoc questions need seconds; Hive's minutes are too slow.
- Hive still suits large ETL and batch jobs, where MR/HDFS give it fault tolerance.
- Presto reads the same Hive Metastore, so it swaps the engine and keeps the tables.

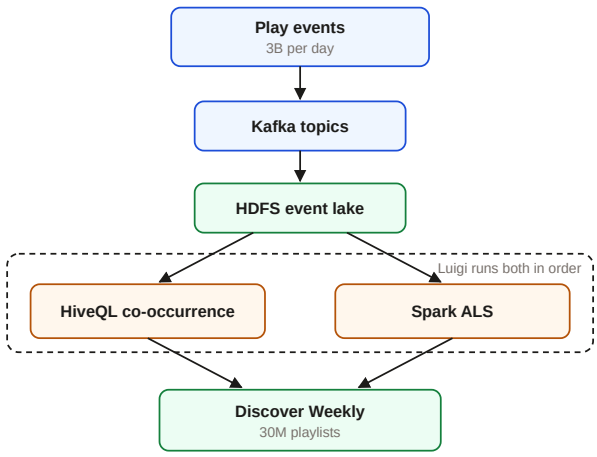
COMPARE

Tool	Execution model	Latency	Best for
Hive (Facebook, 2008)	MR / Tez / Spark, compiled	Minutes	Batch analytics, large ETL
Presto/Trino (Meta, 2013)	In-memory pipeline	Seconds	Interactive BI dashboards
Impala (Cloudera, 2012)	Native C++ MPP	Seconds	Low-latency SQL on HDFS/S3
Spark SQL (Databricks, 2014)	Catalyst optimizer, in-memory	Seconds	Unified batch + streaming SQL
Drill (MapR, 2012)	Schema-free, JSON-native	Seconds	Exploratory SQL on JSON/Parquet

TRAPS

- Presto's speed comes from no intermediate disk writes, not from extra SQL functions, connectors or Python UDFs.
- Presto replaces the engine, not the catalogue: it reads the Hive Metastore.
- The slide title says 10× faster; the ranges (5–15 min vs 5–60 s) give 5× to 180×.

Case study: Spotify Discover Weekly



Play events flow through Kafka to HDFS, where Luigi runs the HiveQL co-occurrence and Spark ALS jobs that build Discover Weekly.

FACTS

- Scale (2015): about 3 billion events/day, 30 million songs, 2 billion playlists, 75M users;

HOW IT WORKS

1. Apps log plays, skips, pauses and playlist adds as Kafka events.

output 30M personalised playlists.

- Data path: Kafka events → HDFS → HiveQL (analytics, features) and Spark MLlib (collaborative filtering).
- **Co-occurrence**: songs found together in more than N playlists are similar: `GROUP BY song_a, song_b HAVING COUNT(*) > 5`.
- **ALS** (Alternating Least Squares) factorises the user × song matrix into a user matrix and a song matrix of k factors each.
- **Luigi** (Spotify, Python, 2012) chains HiveQL and Spark jobs. Each task declares its output path and inputs; finished tasks are skipped.
- Partitioning play_events by week cut the co-occurrence query from 8 hours to 2 hours.

2. Kafka consumers land the events on HDFS.
3. HiveQL self-joins playlist_songs on playlist_id to form song pairs, then GROUP BY counts each pair.
4. Spark ALS learns user and song factor matrices from the interactions.
5. Luigi runs the steps in dependency order each week and skips finished outputs.
6. The scores produce 30M Discover Weekly playlists.

WHY · PROS · LIMITS

- Implicit feedback (2 billion playlists) was a richer signal than explicit star ratings.
- SQL suits aggregation; iterative ML (ALS) suits Spark. Luigi joins them in one pipeline.
- Schema design decides speed: the right partition column changes runtime by hours.
- ALS stores (users + songs) × k numbers instead of users × songs, so the model fits in memory.
- Limitation: cold start. A new song with no playlist appearances needs audio features instead.

COMPARE

	Luigi	Airflow
Dependency style	Target-based: requires(), output(), run(); pulls back from the final task	DAG file of tasks and edges (t1 >> t2), run on a schedule
Backfill	Pass past dates; finished outputs skipped; usually cron-triggered	Built in: catchup and `airflow dags backfill`
UI	Simple status and dependency graph page	Rich UI: graph/grid views, logs, retries, alerts
Best fit	Small, file-based Hadoop pipelines	Large pipelines (e.g. 100 steps)

CALCULATION

ALS model size (Discussion 2): (75M + 30M) × 50 × 4 B = 2.1 × 10^10 B = **21 GB**. The full matrix would be 75M × 30M × 4 B = 9 × 10^15 B = 9 PB. **Speed-up** (lesson 3): 8 h / 2 h = **4×**, 75% time saved.

Step by step

1. Given 75M users, k = 50, 4 B → user factors = 75 × 10^6 × 50 × 4 = 15 GB
2. Given 30M songs → song factors = 30 × 10^6 × 50 × 4 = 6 GB
3. Given 15 GB + 6 GB → model = 21 GB
4. Given 5,000 nodes × 64 GB (assumed) → 320 TB RAM; 21 GB ≈ 0.0066% of it: fits
5. Given 8 h before, 2 h after → speed-up = 4×, saved = (8 − 2) / 8 = 75%
6. Given a playlist of L = 20 songs → pairs = 20 × 19 / 2 = 190

Variants

- 10M users, 1M songs, k = 100: 11M × 100 × 4 B = 4.4 GB
- Dense matrix 9 PB vs 320 TB RAM: about 28× too big, does not fit
- 2B (playlist, song) rows at 20 songs each: 10^8 playlists × 190 = 1.9 × 10^10 pairs

TRAPS

- ALS size uses users *plus* songs, not users *times* songs.
- The slides give no RAM per node. State the assumption (e.g. 64 GB).

- Bucketing playlist_songs by playlist_id speeds the self-join, not the GROUP BY on song pairs.

CHAPTER 4 · WEEK 8

Data integration: Kafka, Lambda/Kappa, data lakes

How data moves from the systems that create it to the systems that analyse it: batch tools (Sqoop, Flume) and the Kafka event log. How Lambda and Kappa architectures combine batch and stream processing, and how a data lake stays usable instead of turning into a swamp.

Exam question pool

23 questions from the slides · click a question for its answer

MCQ **Q1.** Which tool is specifically designed to transfer data in bulk between an RDBMS (e.g., MySQL) and HDFS?

(a) Apache Flume
(b) Apache Kafka
(c) Apache Sqoop
(d) Apache NiFi

(c) Apache Sqoop
Sqoop (SQL-to-Hadoop) bulk-copies tables between a database and HDFS; Flume is for logs.

MCQ **Q2.** Apache Kafka's data model is based on which fundamental storage abstraction?

(a) B-tree indexes for fast point lookups
(b) Column-family storage (like HBase)
(c) Append-only immutable distributed logs (partitioned topics)
(d) Document storage with secondary indexes

(c) Append-only immutable distributed logs (partitioned topics)
Kafka is a distributed, partitioned, replicated commit log.

MCQ **Q3.** In the Lambda architecture, which layer handles **real-time**, **low-latency** query views?

(a) Batch Layer
(b) Serving Layer
(c) Speed Layer
(d) Ingestion Layer

(c) Speed Layer
The Serving layer only merges views; the Speed layer computes the seconds-old views.

MCQ **Q4.** The Kappa architecture differs from the Lambda architecture primarily by:

(a) Adding a third processing layer for machine learning
(b) Using only relational databases for both batch and stream storage
(c) Eliminating the batch layer and using Kafka log replay for all reprocessing
(d) Replacing Hadoop with Spark in the batch layer while keeping the speed layer

(c) Eliminating the batch layer and using Kafka log replay for all reprocessing
Kappa removes the whole batch layer; (d) is the common misconception.

TRUE/FALSE **1.** Apache Sqoop is designed for real-time streaming ingestion from transactional systems.

False. Sqoop runs batch MapReduce imports. Corrected: Sqoop is designed for bulk batch transfer between an RDBMS and HDFS; real-time ingestion uses Kafka.

TRUE/FALSE **2.** Kafka retains messages for a configurable period, allowing consumers to replay events from any past offset.

True. Messages stay for the retention period (e.g. 7 days) after being read, so a consumer can reset its offset and replay.

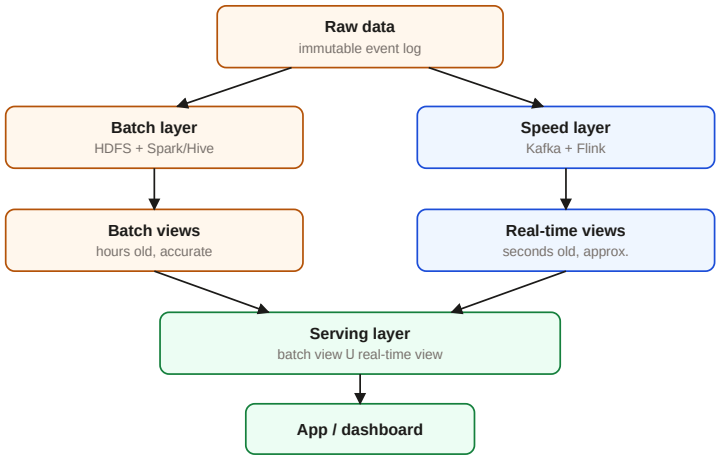
TRUE/FALSE **3.** The Lambda architecture requires maintaining two separate code paths implementing the same business logic.

True. The same logic is written for the batch layer (Spark/Hive) and the speed layer (Spark Streaming) and must be kept in sync.

TRUE/FALSE **4.** Apache Flume is the preferred tool for collecting and aggregating server log data into HDFS.

True. Flume moves logs into HDFS through Source, Channel and Sink with fan-in from many servers.

DESCRIPTIVE **D1.** Explain the **Lambda architecture** with a labelled diagram. What are its three layers? What is its primary engineering disadvantage compared to the Kappa architecture?



- **Batch:** master dataset on HDFS, recomputed hourly/nightly; accurate, 1–24 h stale.
- **Speed:** stream processing of data since the last batch; approximate, results within seconds.
- **Serving:** query = batch view UNION real-time view.

Disadvantage vs Kappa: two code paths for the same logic, kept in sync (30–40% of effort), plus two clusters and a runtime merge. Kappa has one stream code path.

DESCRIPTIVE **D2.** Describe how Apache Kafka achieves **high throughput and fault tolerance**. Explain the roles of topics, partitions, offsets, brokers, and replication factor in the Kafka data model.

Term	Role
Topic	Named append-only log for one event type
Partition	Ordered slice of a topic; P partitions run in parallel; same key → same partition
Offset	Message position; each consumer tracks its own
Broker	Server storing partitions
Replication factor R	1 leader + R − 1 followers per partition (default 3)

Throughput: sequential writes, zero-copy sendfile(), batching, partition parallelism.

Fault tolerance: a follower takes over when a leader's broker fails; retention lets a crashed consumer resume from its offset. Example: bkash-txns, 3 partitions, RF = 2 survives 1 broker failure.

DESCRIPTIVE **D3.** What is a **data lake**? How does it differ from a data warehouse in terms of schema enforcement, storage format, and typical use cases? What is a “data swamp” and how is it prevented?

Data lake: central store of raw data in any format, no upfront schema.

	Lake	Warehouse
Schema	On read	On write
Format	Raw files, any type	Cleaned relational tables
Use	ML, data science, exploration	BI reports, dashboards

Data swamp: a lake with no governance: no catalogue, stale vs current unknown, unversioned copies, PII mixed in.

Prevention: catalogue at ingest, owner per dataset, access control, quality checks, Delta Lake/Iceberg for ACID.

ANALYTICAL

A1. Pipeline Design for a Bank: Design a data integration pipeline for a Bangladeshi commercial bank that must:

- (a) Synchronise 10 million customer records nightly from Oracle to Hadoop for analytics
- (b) Detect fraudulent card-swipe events in real time with a decision within 200 ms
- (c) Power a risk management dashboard with data no older than 5 minutes

Specify which tool (Sqoop, Flume, Kafka, Spark Streaming) you would use at each stage. Justify each choice with reference to latency, throughput, and fault-tolerance requirements. Which architecture (Lambda or Kappa) fits best, and why?

Need	Tool	Latency	Throughput	Fault tolerance
(a) 10 M rows nightly	Sqoop (incremental import)	Hours is fine: batch	Parallel map tasks by key range, GB–TB/h	Failed map task retried; HDFS replicated
(b) Fraud < 200 ms	Kafka topic (keyed by card no.) + Spark Streaming	Milliseconds	Partitions scale consumers	RF = 3; restart from last offset
(c) Dashboard < 5 min	Second consumer group on same topic + Spark Streaming aggregates	Minutes	Same topic, no extra ingest	Replay from offset

Flume is not used: it is for logs, not tables or card events.

Architecture: Lambda. The bank has a real batch need (nightly sync, regulatory batch reports), which the slide lists as a reason Kappa is not suitable. Batch = Sqoop + HDFS + Hive; Speed = Kafka + Spark Streaming; Serving = dashboard. Cut the dual-path cost by making Kafka the single source both layers read.

ANALYTICAL

A2. Lambda to Kappa Migration: A team using Lambda architecture complains that maintaining two code paths (batch + stream) doubles development cost. Their CTO suggests migrating to Kappa. Analyse: (a) what must be true about the stream processing engine for Kappa to be viable; (b) what are the risks of migrating; (c) under what conditions is Kappa **not suitable** (give at least two concrete scenarios from the Bangladesh financial or healthcare sector)?

(a) Conditions: the engine reprocesses history about as fast as live data; full history sits in a replayable log (long Kafka retention or Iceberg on S3; default is only 7 days); the logic fits a stream job; exactly-once processing so replays match.

(b) Risks: TB+ replay may take days; batch throughput falls to stream-engine speed; storage cost of long retention; every Hive/Spark batch job must be rewritten and run in parallel during the switch; staff must learn stream SQL.

(c) Not suitable:

1. **Regulatory reconciliation:** Bangladesh Bank requires a daily end-of-day batch reconciliation; the batch layer must stay.
2. **TB+ history:** a national health-records system recomputing 10 years of disease statistics after a coding change; one Spark batch job is far faster than replay.
3. **Batch SQL preferred:** yearly credit-risk model training written in Hive/Spark SQL over full history.

Verdict: move streaming workloads to one Kappa path; keep batch where regulation or huge history demands it.

DISCUSSION

L1. Kafka as the single source of truth: all events written to Kafka first; the speed and batch layers are both consumers of the same topic — eliminating the data inconsistency problem of classic Lambda (two separate ingestion paths)

Claim: Netflix writes each event to Kafka once; Flink (speed, < 1 min) and the S3 sink + Spark/Iceberg (batch, ~1 day) read the same topic, so both layers see the same data. **For:** no mismatch between layers; new consumers just subscribe. **Against:** Kafka becomes a single critical dependency (2,000+ brokers to keep up), and the two code paths still exist; only ingestion is unified.

DISCUSSION

L2. Retention enables reprocessing: 7-day retention lets Netflix replay any event window when a Flink job has a bug or a Spark job fails

Claim: reading does not delete events, so a fixed job resets its offset and recomputes the last week. **For:** bugs are fixed after the fact without asking devices to resend. **Against:** only 7 days; a bug found on day 8 needs the S3/Iceberg copy. At 100 billion events/day, 7 days is 700 billion events of storage; tiered storage (Kafka 3.6) moves old segments to S3.

DISCUSSION **L3. Kappa simplification**: Netflix increasingly moved batch jobs to Flink stream processing (processing historical S3 data as a bounded stream), reducing pipeline complexity

Claim: a bounded stream (data with a start and an end, such as last month's S3 files) can be read by the same Flink code as live data, so one code path covers both. **For:** less duplicate code, fewer systems. **Against:** Netflix still keeps Spark + Iceberg for ML training and billing, where batch is easier or cheaper; the move is gradual.

DISCUSSION **L4. Schema Registry is non-negotiable at scale**: 4,000 topics × evolving schemas ⇒ consumer breakage without schema governance

Claim: a schema is a message's field names and types; if a producer renames a field, consumers break. Schema Registry stores the Avro/Protobuf schema per topic so consumers can read any past producer version. **For:** 4,000+ topics and 150+ Flink jobs cannot be coordinated by hand. **Against:** another service to run, and teams must follow compatibility rules. It is the streaming form of the data-swamp lesson.

DISCUSSION **L5. Backpressure is the real production problem**: during Super Bowl commercials, event rates spike 5×; Kafka's durable log absorbs the burst; Flink consumers catch up within minutes

Claim: backpressure is data arriving faster than consumers can process. If the 5× spike landed on the 1.3 M/s peak, that would be 6.5 M/s; Kafka appends the excess to disk and Flink catches up later. **For:** no data lost, no need to size consumers for the rare peak. **Against:** results lag by minutes during the spike (the 5-minute A/B latency slips), and brokers need spare disk.

LIVE EXERCISE

Task. Simulate a Kafka producer/consumer pipeline using the **kafka-python** library with a local single-broker cluster (Docker Compose provided below).
Quick-start: `pip install kafka-python` ; `docker run -p 9092:9092 apache/kafka:3.7.0`

```
import json, random, time
from kafka import KafkaProducer, KafkaConsumer

TOPIC   = "play-events"
BROKER  = "localhost:9092"
CONTENT = [f"show_{i}" for i in range(20)]

# --- Producer ---
producer = KafkaProducer(
  bootstrap_servers=BROKER,
  value_serializer=lambda v:
      json.dumps(v).encode())

for _ in range(1_000):
    event = {
      "user_id":    random.randint(1, 500),
      "content_id": random.choice(CONTENT),
      "action":     random.choice(
                      ["play","pause","seek"]),
      "ts":         int(time.time())
    }
    producer.send(TOPIC, event)
producer.flush()
print("Produced 1,000 events")

# --- Consumer ---
consumer = KafkaConsumer(
  TOPIC,
  bootstrap_servers=BROKER,
  group_id="cse4345-group",
  auto_offset_reset="earliest",
  value_deserializer=lambda m:
      json.loads(m.decode()))

counts = {}
for msg in consumer:
    cid = msg.value["content_id"]
    counts[cid] = counts.get(cid, 0) + 1
    if sum(counts.values()) ⩾ 1_000:
        break
print(sorted(counts.items(),
             key=lambda x: -x[1])[:5])
```

The producer sends 1,000 JSON events (user 1–500, one of 20 shows, play/pause/seek); `flush()` waits until queued messages are sent. The consumer (group `cse4345-group` , `auto_offset_reset="earliest"`) reads from the start, counts events per `content_id` , stops after 1,000 and prints the top 5 shows. Broker: `docker run -p 9092:9092 apache/kafka:3.7.0` .

Replay (goal 3): `consumer.seek_to_beginning(*consumer.assignment())` . New group (goal 4): change `group_id` .

LIVE EXERCISE

1. Run the producer — send 1,000 synthetic Netflix play events to topic `play-events` .

Expected output: `Produced 1,000 events` . With one partition the messages sit at offsets 0–999. Running it again appends offsets 1,000–1,999; nothing is overwritten.

LIVE EXERCISE

2. Run the consumer — read all messages, count plays per `content_id` .

Expected: 1,000 events read. The slide's consumer counts every event per `content_id` (play, pause and seek alike), so each of the 20 shows gets about 1,000 / 20 = **50**; the top-5 print shows the largest. To count plays only, add `if msg.value["action"] == "play"` : about 1,000 / 3 ≈ 333 plays, about 16.7 per show. Exact counts change each run because the producer is random.

LIVE EXERCISE **3.** Reset the consumer offset to the *beginning* and re-consume — verify idempotent results.

After `seek_to_beginning` the same group re-reads all 1,000 events and gets identical counts. Reading did not delete anything, so the result is idempotent (same answer when repeated). If the producer ran again in between, there are 2,000 events: that is new data, not a broken replay.

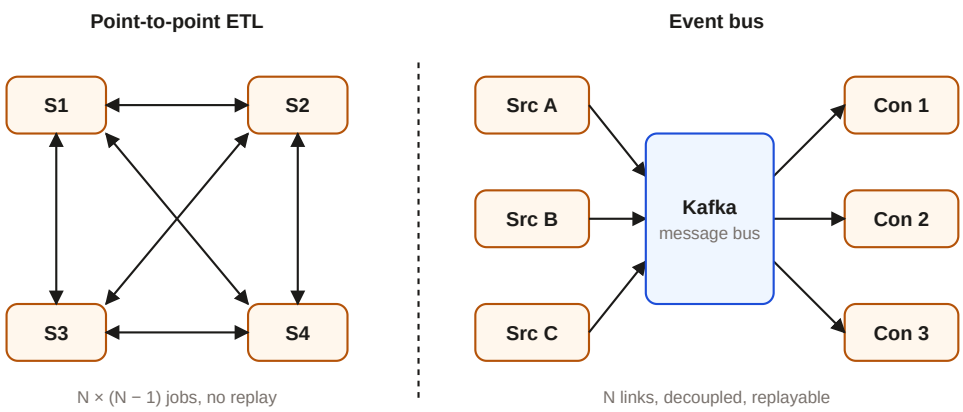
LIVE EXERCISE **4.** Change the consumer group ID and observe that a new group starts from offset 0.

A new group has no committed offset, so with `auto_offset_reset="earliest"` it reads all 1,000 events from offset 0. The old group's offset is untouched. Trap: kafka-python's default is "latest", which makes a new group read 0 events.

Topic cards

6 topics · everything the pool tests

From point-to-point ETL to event pipelines



Point-to-point ETL wires every pair of systems (N × (N − 1) jobs); an event bus needs one link per system and lets consumers replay.

FACTS

- ETL (Extract, Transform, Load) is a job that copies data from one system into another.
- Point-to-point ETL joins every pair of systems with its own job: O(N²) connections, 2,450 jobs at 50 systems.
- Four faults of point-to-point ETL: brittle (schema change breaks jobs), no replay, no real time (latency ≥ 24 h), no decoupling.
- Event-driven pipeline: one central message bus (Kafka). Sources publish events, consumers subscribe.
- Adding a new consumer to a bus needs zero changes to any source system.
- Tool landscape: Sqoop = bulk DB sync, Flume = log collection, Kafka = real-time events, NiFi = visual no-code routing, Spark Streaming = processing streams.

HOW IT WORKS

1. Each source system publishes its events to the bus once
2. The bus stores the events durably, in order
3. Each consumer subscribes to the topics it needs and reads at its own pace
4. A new consumer subscribes and can replay stored history; no source changes

WHY · PROS · LIMITS

- N systems on a bus need N connections instead of N × (N − 1) jobs.
- Decoupling: a source does not know or care who reads its events.

COMPARE

	Point-to-point ETL	Event-driven pipeline
Connections	N × (N − 1), O(N²)	N (one per system, to the bus)

- Replay: stored events let a late or crashed consumer catch up; nightly ETL loses them.
- Real time: events flow in seconds, not after a 24-hour batch.
- Cost: the bus is one critical system; if it is down, every flow stops.

Latency	≥ 24 hours (nightly batch)	Seconds
Replay	None; offline consumer loses data	Yes, from the retained log
Schema change	Breaks the jobs on that pair	Only affects that topic's readers
New consumer	New job per source	Subscribe; zero source changes

Tool	Direction	Use it for
Sqoop	RDBMS ↔ HDFS, batch	Nightly bulk copy of database tables
Flume	Logs → HDFS, batch/micro-batch	Collecting server logs
Kafka	Any → any, real-time	Central event bus: fraud events, clickstream, IoT
NiFi	Any → any, visual	Drag-and-drop routing, IoT edge to cloud
Spark Streaming	Kafka → HDFS/DB	Stateful aggregation, enrichment, joins on streams

CALCULATION

Slide: 50 systems. Point-to-point jobs = 50 × 49 = **2,450**. With a bus: **50** connections.

Step by step

1. Given N = 50 systems -> each system sends to N − 1 = 49 others
2. Given 50 senders × 49 targets -> 2,450 jobs (ordered pairs)
3. Given 2,450 ordered jobs -> one job per pair = 2,450 / 2 = 1,225
4. Given a bus -> connections = N = 50

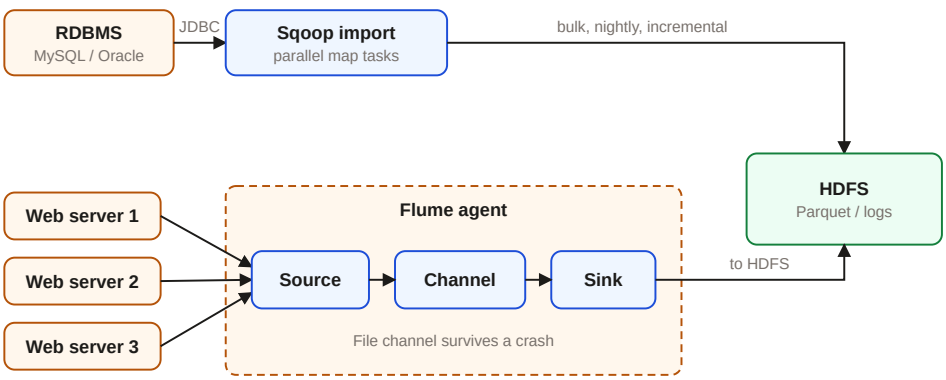
Variants

- N = 10: 10 × 9 = 90 jobs (45 if each pair is counted once), bus 10
- N = 20: 20 × 19 = 380 jobs, bus 20
- N = 30: 30 × 29 = 870 jobs, bus 30

TRAPS

- The slide's 2,450 counts A→B and B→A as separate jobs. If you count each pair once, say so and divide by 2.
- Spark Streaming processes streams; it does not ingest from a database. Do not list it as an ingestion tool.
- NiFi is the visual tool, not Flume.

Batch ingestion: Sqoop and Flume



Sqoop bulk-copies database tables to HDFS through parallel map tasks; Flume carries web-server logs to HDFS through an agent's Source, Channel and Sink.

FACTS

- Sqoop (SQL-to-Hadoop) is a command-line tool for bulk transfer between an RDBMS (MySQL, Oracle, PostgreSQL, SQL Server) and HDFS, Hive or HBase.
- Sqoop import: RDBMS → HDFS as CSV, Avro, Parquet or SequenceFile. Export: HDFS → RDBMS.
- Sqoop incremental import copies only rows newer than a last-value checkpoint on a timestamp or auto-increment column.
- Flume moves semi-structured logs (web server, application, IoT) into HDFS.
- A Flume agent has three parts: Source (reads), Channel (buffers), Sink (writes).
- Flume channel types: Memory (fast, lost on crash) or File (on disk, survives agent restart).

HOW IT WORKS

1. Sqoop client connects to the database over JDBC
2. It submits a MapReduce job; each map task reads one primary-key range of rows in parallel
3. Rows land in HDFS (or Hive/HBase) as Parquet, CSV or Avro
4. Next run, incremental mode reads only rows past the last-value (for example txn_date > 2024-06-12)
5. Flume: a Source reads log lines (Exec tail, Syslog, HTTP, Avro)
6. The Channel buffers the events
7. The Sink writes them to HDFS (or Kafka, HBase)

WHY · PROS · LIMITS

- Sqoop lets Hive analytics run on HDFS without touching the live OLTP database.
- Parallel map tasks give GB–TB/hour; a failed map task is retried by MapReduce.
- Flume fan-in (many sources → one agent) and fan-out (one source → many sinks) aggregate logs from many servers.
- File channel trades speed for durability: events survive an agent crash.
- Limitation: both are batch. Real-time needs Kafka; modern stacks often replace Flume with Kafka producers.

COMPARE

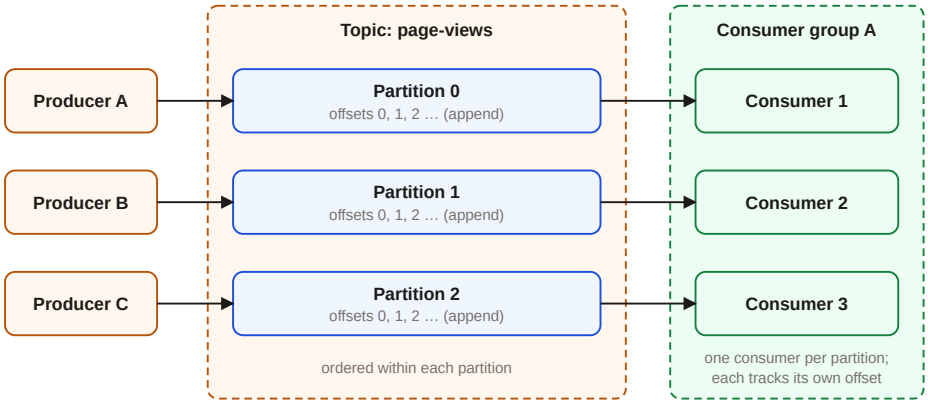
	Sqoop	Flume	Kafka
Source data	Database tables	Log files	Any events
Mode	Batch (nightly)	Batch / micro-batch	Real-time stream
Target	HDFS, Hive, HBase (and back)	HDFS (also Kafka, HBase)	Any consumer
Replay	Re-run the import	No	Yes, from offset

TRAPS

- Sqoop is batch, not real-time streaming (pool T/F 1 is False).
- Memory channel is faster but loses buffered events on a crash; File channel survives.

- Flume is for logs, not database tables. Database → HDFS is Sqoop.

Kafka: the append-only log



Producers append to a topic's partitions; each partition is read by exactly one consumer in a group, which tracks its own offset.

FACTS

- Kafka is a distributed, partitioned, replicated commit log, built at LinkedIn by Kreps, Narkhede and Rao (2011).
- Topic: a named, append-only log for one event type. Partition: an ordered slice of a topic; P partitions run in parallel.
- Offset: the integer position of a message in a partition. The consumer tracks its own offset (in the __consumer_offsets topic).
- Broker: a Kafka server. Each partition has 1 leader and R − 1 follower replicas (default R = 3).
- Consumer group: consumers that share a topic; each partition goes to exactly one consumer in the group.
- Retention: messages stay for a set time (default 7 days) or size quota, even after being read.

WHY · PROS · LIMITS

- Append-only gives durability, replay and decoupling for free (the founding insight).
- Same key → same partition, so order is kept per key (for example per driver_id).
- Replicas on other brokers let the cluster survive R − 1 broker failures.
- The broker does not track consumers, so one log serves any number of consumer groups.
- Limit: a consumer group cannot use more consumers than partitions; extras sit idle.

HOW IT WORKS

1. A producer sends a message to a topic; the key's hash (or round-robin) picks the partition
2. The partition leader appends it at the end and gives it the next offset
3. Followers on other brokers copy it
4. Each consumer in a group pulls from its assigned partitions and advances its own offset
5. To replay, a consumer resets its offset to an earlier value (for example 0)
6. Old segments are deleted after the retention period
7. If a leader's broker fails, a follower becomes leader (ZooKeeper before Kafka 3.0, KRaft from 3.0)

COMPARE

	Traditional queue	Kafka log
After a read	Message deleted	Message kept until retention ends
Late consumer	Misses data	Reads from offset 0
Who tracks position	Broker	Consumer (its own offset)
Many readers	Message goes to one	Every group reads every message

CALCULATION

Slide: topic bkash-txns, P = 3 partitions, RF = 2. Copies = 3 × 2 = **6**; followers per partition = 1; survives **1** broker failure. Pathao driver-location: P = 10; a group of 4 gets 3, 3, 2, 2 partitions.

Step by step

1. Given P and R -> total partition copies = P × R
2. Given R -> followers per partition = R − 1
3. Given R -> broker failures survived = R − 1
4. Given P partitions and C consumers in one group -> active = min(P, C), idle = C − P if C > P
5. Given P = 10, C = 4 -> 10 / 4 = 2.5, so two consumers get 3 and two get 2

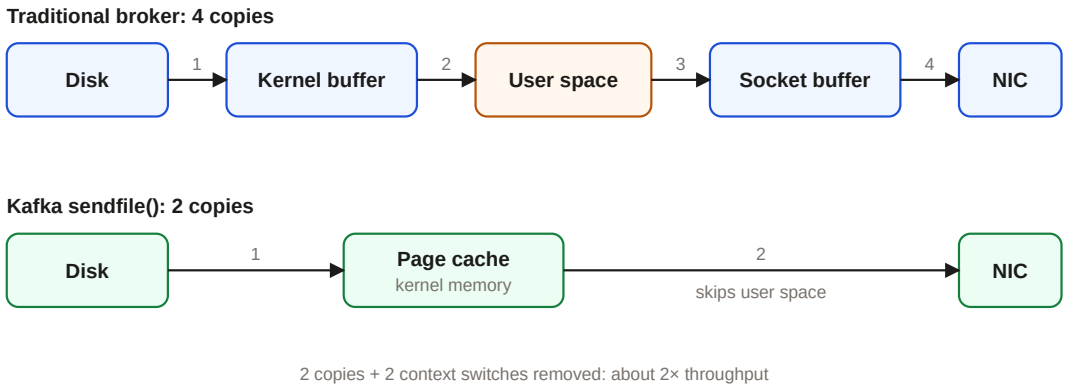
Variants

- page-views, 3 partitions, group of 5: 3 active, 2 idle
- P = 10, default R = 3: 30 copies, 20 followers, survives 2 failures
- P = 6, C = 4: two consumers get 2, two get 1

TRAPS

- Reading does not delete a message; retention decides deletion.
- Order is guaranteed only within a partition, not across a whole topic.
- Two different consumer groups each get every message; within one group a message goes to one consumer.
- The broker does not track consumer position; the consumer does.

Why Kafka is fast



A traditional broker copies data four times through user space; Kafka's sendfile() zero-copy sends it from the page cache to the NIC in two.

FACTS

- Four throughput tricks (P1): sequential disk writes, zero-copy transfer, batching, partition parallelism.
- Sequential writes: about 600 MB/s vs about 100 MB/s random (paper slide); P1 quotes < 1 MB/s random.
- Zero-copy: the sendfile() system call sends data disk → kernel buffer → NIC: 2 copies instead of 4.
- Batching: many messages per network round trip, set by batch.size and linger.ms; batches are compressed 5–10×.

HOW IT WORKS

1. Producer groups messages into a compressed batch
2. Broker appends the batch to the end of the active segment file (sequential write)
3. OS page cache holds recent pages and prefetches the next ones
4. A consumer pulls from an offset; the .index file maps offset to byte position
5. Broker calls sendfile(): data goes from page cache to the network card, skipping user space
6. More partitions let more producers and consumers work at once

- Paper benchmark (100-byte messages, one broker): producer Kafka 50.5 MB/s, RabbitMQ 14.7, ActiveMQ 4.3.
- Consumer benchmark: Kafka 22.0 MB/s, RabbitMQ 8.2, ActiveMQ 4.0.

WHY · PROS · LIMITS

- Disk heads do not jump when writing only at the end, so spinning disks get near-SSD throughput.
- Zero-copy removes 2 copies and 2 context switches, so about 2× throughput with CPU near zero.
- The broker acknowledges a batch, not each message, cutting network overhead.
- Consumers pull and track their own offsets, so the broker keeps no per-consumer queue.
- Old MOM systems (ActiveMQ, RabbitMQ) were built for a few thousand transactional messages per second.

COMPARE

	Kafka	ActiveMQ / RabbitMQ
Built for	Log aggregation, 100 MB/s per broker	Transactional messaging, few thousand msgs/s
Storage	Append-only log, kept for days	Queue, deleted after delivery
Acknowledgement	Per batch	Per message
Producer MB/s (paper)	50.5	4.3 / 14.7
Replay	Yes	No

CALCULATION

Paper, producer, 100-byte messages: 50.5 MB/s ÷ 100 B = 50,500,000 / 100 = **505,000 msgs/s**. Kafka vs ActiveMQ = 50.5 / 4.3 ≈ **11.7×**; vs RabbitMQ = 50.5 / 14.7 ≈ **3.4×**. Disk: 600 / 100 = **6×**. Zero-copy: 4 / 2 = **2×**.

Step by step

1. Given MB/s -> bytes/s = MB/s × 1,000,000
2. Given bytes/s and message size -> msgs/s = bytes/s ÷ size
3. Given two throughputs -> speedup = Kafka ÷ other
4. Given sequential and random MB/s -> advantage = sequential ÷ random
5. Given 4 copies (traditional) and 2 (sendfile) -> 2 copies saved per message, about 2×

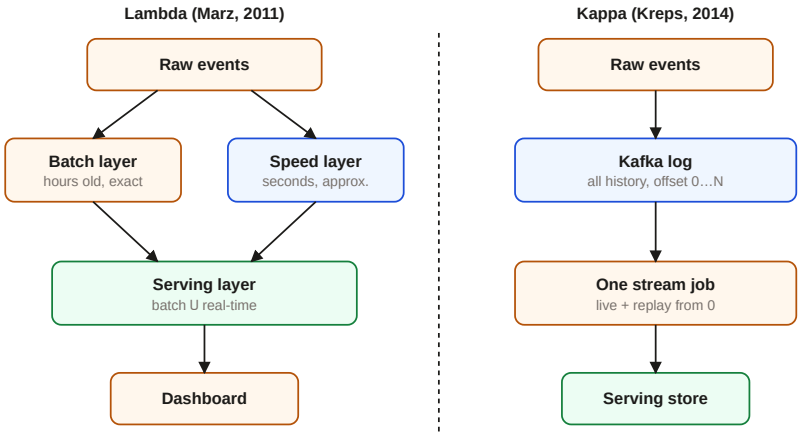
Variants

- Consumer: 22.0 MB/s ÷ 100 B = 220,000 msgs/s; vs ActiveMQ 22.0 / 4.0 = 5.5×; vs RabbitMQ ≈ 2.7×
- 1,000-byte messages at 50.5 MB/s: 50,500 msgs/s
- Netflix: 100 billion events/day ÷ 86,400 ≈ 1,157,407 events/s average; if the 5× Super Bowl spike hit the 1.3 M/s peak, that would be 6.5 M/s
- Netflix 7-day retention holds 100 B × 7 = 700 billion events

TRAPS

- Zero-copy removes copies through user space; it does not mean no copies (2 remain).
- Two different random-disk numbers appear: < 1 MB/s (P1) and about 100 MB/s (paper). Quote the one the question gives.
- The benchmark table treats 1 MB as 1,000,000 bytes: 50.5 MB/s of 100-byte messages = 505,000 msgs/s.

Lambda and Kappa architectures



Lambda runs a batch and a speed layer merged by a serving layer; Kappa keeps one stream job over a replayable Kafka log.

FACTS

- Lambda (Nathan Marz, 2011): Batch layer, Speed layer, Serving layer.
- Batch layer: immutable master dataset on HDFS, recomputed by Spark/Hive; accurate but 1–24 hours stale.
- Speed layer: Kafka + Spark Streaming/Flink on data since the last batch; approximate but only seconds old.
- Serving layer: merges views; query = batch view UNION real-time view. It does no processing.
- Kappa (Jay Kreps, 2014): two parts only, log storage (Kafka or Iceberg on S3) and one stream engine (Flink, Spark Structured Streaming).
- Lambda's weakness: the same logic is coded twice; 30–40% of engineering effort goes on keeping them in sync.

WHY · PROS · LIMITS

- Lambda strengths: accuracy (batch recomputes from raw), low latency, fault tolerance, separation of concerns.
- Lambda costs: two code paths, two clusters, a runtime merge, harder debugging.
- Kappa works only if the stream engine can reprocess history as fast as it handles live data.
- Kappa is NOT suitable when: TB+ history is too slow to replay; batch SQL (Hive, Spark) is preferred; regulation demands a separate batch reconciliation.
- Netflix mixes both: Kafka is the single source both layers read, removing two ingestion paths.

HOW IT WORKS

1. Lambda: raw events go to both the batch and the speed layer
2. Batch layer recomputes accurate views hourly or nightly; speed layer covers the gap since
3. Serving layer unions both views to answer a query
4. Kappa reprocessing: deploy a new version of the stream job
5. Start it at Kafka offset 0
6. It processes all history, then catches up to live events
7. When caught up, swap it into production and retire the old job

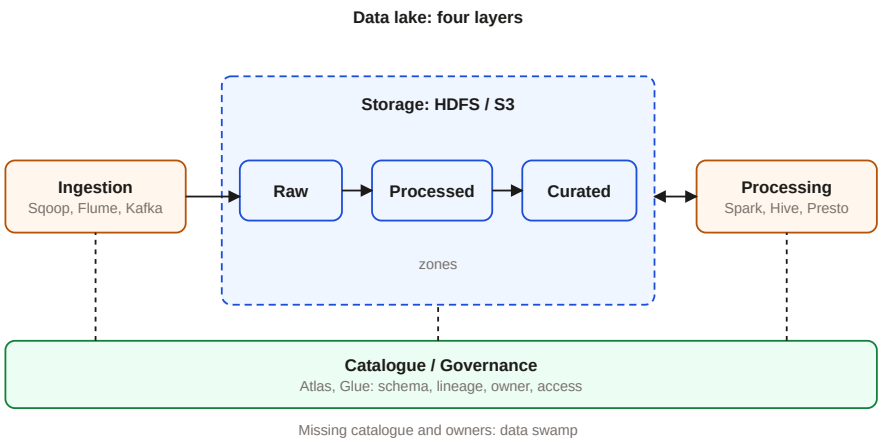
COMPARE

	Lambda	Kappa
Code paths	Two (batch + stream), kept in sync	One (stream only)
Reprocessing	Re-run batch job on HDFS	Replay Kafka log from offset 0
Latency	Low (speed) + high (batch)	Low throughout
Complexity	High: two systems + merge	Lower: one pipeline
Batch throughput	Very high (Spark/Hive)	Limited by stream engine
Use when	Complex logic, batch ML training, regulatory batch reports	Stream engine fast enough; one code path worth the storage cost

TRAPS

- The Serving layer does not process data; real-time views come from the Speed layer (pool MCQ 3).
- Kappa is not 'Spark instead of Hadoop'; it removes the whole batch layer (pool MCQ 4, option d).
- There is no Ingestion layer in Lambda; that is a data-lake layer.
- Kappa keeps a serving store; what it drops is the batch layer.

Data lakes and the data swamp



A data lake's four layers: ingestion, zoned storage, processing, and the catalogue that keeps it from becoming a swamp.

FACTS

- Data lake: a central store for raw data at any scale and in any format, with no schema defined up front.
- Four layers: Ingestion (Sqoop, Flume, Kafka, NiFi), Storage (HDFS, S3, ADLS), Processing (Spark, Hive, Presto), Catalogue/Governance (Atlas, Glue, Hive Metastore).
- Storage is organised in zones: raw → processed → curated.
- Data swamp: a lake without governance, where files pile up with no metadata, owners or quality guarantees.
- Swamp symptoms: no catalogue, unclear current vs stale files, unversioned copies, PII mixed with other data.
- Prevention: catalogue at ingest, owner per dataset, access control, automated quality checks, table format (Delta Lake, Iceberg) for ACID.

WHY · PROS · LIMITS

- Cheap, fast ingest: nothing is rejected for not fitting a schema.
- Keeps full raw history for data science and ML.
- Schema-on-read: structure is applied only when a query reads the data.
- Risk: without the catalogue layer the lake becomes a swamp nobody can use or trust.

HOW IT WORKS

1. Ingestion tools pull data from every source and write raw files
2. Files land in the raw zone of HDFS or object storage
3. Processing engines transform them into processed and curated zones on demand
4. The catalogue records schema, lineage, owner and access policy for each dataset

COMPARE

	Data lake	Data warehouse
Schema	Schema-on-read (applied at query time)	Schema-on-write (must fit before load)
Storage format	Raw files of any type on cheap storage	Cleaned, structured relational tables
Users / use	Data science, ML, exploration, raw history	Fixed reports, BI dashboards, SQL by business users

- Mixed PII without access control breaks privacy rules.

Data quality	Varies; needs governance	High, enforced at load

TRAPS

- A lake still needs a schema; it is applied on read, not removed.
- The swamp is caused by missing governance, not by data volume.
- Delta Lake and Iceberg are table formats that add ACID; they are not separate lakes.

CHAPTER 5 · WEEK 9

Data visualization

How to draw big data so people see the truth in it: Tufte's rules for clean, honest charts, Shneiderman's rule for interactive ones, and picking the chart from the question. Then the three problems scale adds (overplotting, latency, streaming), the tools that solve them, and the NYT COVID tracker as a worked case.

Exam question pool

19 questions from the slides · click a question for its answer

MCQ **Q1.** Edward Tufte's “data-ink ratio” principle states that a visualization should:

(a) Use more colours to highlight different categories of data
(b) Maximise the fraction of ink devoted to representing actual data, removing all decorative elements
(c) Use ink sparingly to reduce printing costs
(d) Always include data labels using a different ink colour from the chart body

(b) Maximise the fraction of ink devoted to representing actual data, removing all decorative elements
The ratio is about what the ink encodes, not saving ink.

MCQ **Q2.** Which visualization type is most appropriate for showing the distribution and spread of a continuous variable?

(a) Pie chart
(b) Line chart
(c) Histogram
(d) Treemap

(c) Histogram
Bins along the number line show shape and spread.

MCQ **Q3.** Shneiderman's visualization mantra begins with which step?

(a) Details first, then overview
(b) Overview first, zoom and filter, then details on demand
(c) Filter first, then provide an overview
(d) Details on demand first, then zoom out

(b) Overview first, zoom and filter, then details on demand
Filtering only makes sense after the overview shows what to filter.

MCQ **Q4.** Which visualization tool is best suited for a real-time production operations dashboard monitoring server metrics and application error rates with 5-second refresh intervals?

(a) Tableau
(b) D3.js
(c) Power BI
(d) Grafana

(d) Grafana
Native time-series DB integration with 5 s auto-refresh; D3 would need custom code.

TRUE/FALSE **1.** A pie chart is a recommended choice for comparing more than 5 categories.

False. People judge angles poorly. Corrected: use a horizontal bar chart; pies only for ≤ 4 categories.

TRUE/FALSE **2.** Hexbin plots are an effective technique for reducing overplotting when visualising millions of data points.

True. Counting points per hexagon turns 10 million dots into ≈ 10,000 coloured cells.

TRUE/FALSE **3.** D3.js generates static PNG or PDF image files as its primary output format.

False. Corrected: D3.js binds data to DOM elements and produces interactive SVG in the browser.

TRUE/FALSE **4.** Choropleth maps are used to visualise the geographic distribution of a quantitative variable.

True. Regions are shaded by a value, best a rate such as cases per 100,000.

DESCRIPTIVE **D1.** Explain Tufte's data-ink ratio principle. Provide a “before and after” example: describe a cluttered bar chart (what chart-junk elements are present) and then describe the clean version after applying the principle.

Principle: data-ink ratio = ink encoding data ÷ total ink; maximise it. If erasing an element loses no information, erase it.

Before (quarterly sales)	After
Title "CHART TITLE!!!"	Plain short title
Legend for one series	No legend; Q1-Q4 on the x-axis
3-D bars with shadows	Flat 2-D bars, one colour
Gradient fill, border	Removed
Heavy dark gridlines	None, or one light line; y starts at 0

Same data, but almost all remaining ink is data, so the trend reads at once.

DESCRIPTIVE **D2.** What is overplotting? Describe three techniques to address it when visualising millions of data points in a scatterplot. For each technique, explain what information it preserves that the original overplotted chart lost.

Overplotting: millions of dots overlap into a blob that only says "data exists here".

Technique	How	Preserves
Alpha transparency	Each point α = 0.02; stacks look darker	Relative density (fails at very high density)
Hexbin	Count points per hexagon, colour by count; 10 M → ≈ 10,000 cells	Actual count per area
KDE contour	Smooth density, contour lines	Shape, modes (peaks), tails

DESCRIPTIVE **D3.** What is a data cube? How does pre-aggregating a billion-row fact table into a data cube enable interactive visualization at sub-second latency? What is the trade-off of this approach?

Data cube: pre-computed aggregates (SUM, COUNT) of a fact table for combinations of dimensions, e.g. district × date × product × channel.

- A nightly Spark/Hive job builds it once.
- It holds millions of rows, not billions.
- Tableau/Superset filters read the cube, so they answer in under 1 s instead of minutes.

Trade-off: staleness up to 24 h; only pre-built dimensions can be sliced; extra storage. Alternative: AQP (sampling) trades accuracy for speed.

ANALYTICAL **A1.** Public Health Dashboard Design: A DGHS public health dashboard needs to communicate: (a) COVID-19 confirmed case counts by district over 24 months (b) Real-time hospitalisation rate updated every 5 minutes (c) Demographic breakdown (age group, gender) of confirmed cases For each sub-requirement: specify the appropriate visualization type, justify using Shneiderman's data-type taxonomy, identify which big data visualization challenge applies (if any), and name the tool you would use. Apply Tufte's data-ink principle to state one specific chart-junk element to eliminate.

Req.	Data type	Chart	Challenge	Tool
(a) Cases by district, 24 months	Temporal (+ 2-D map)	Small multiples of line charts, same scale; choropleth of rate per 100,000	Latency: pre-aggregate to daily counts per district	Tableau / Power BI
(b) Hospitalisation every 5 min	Temporal (live)	Stat panel + sparkline, 24 h line chart	Streaming	Grafana on InfluxDB/Prometheus
(c) Age × gender	Categorical / multi-dimensional	Grouped horizontal bar	None (small table)	Tableau / Power BI

Sizes: (a) 730 days × 64 districts = **46,720** rows after aggregation. (b) 24 × 60 ÷ 5 = **288** points per day, so 5 s polling is ample.

Mantra: national total → district dropdown and date slider → hover tooltip.

Chartjunk to remove: 3-D effect on the demographic bars (also heavy gridlines on the line charts).

ANALYTICAL

A2. Streaming Sentiment Visualization: Design a complete visualization system for a streaming Twitter (X) sentiment analysis pipeline processing 10,000 tweets per second. Specify: (a) which Kafka aggregation window you would use and why; (b) which chart types best convey sentiment drift over time and topic clustering; (c) how you would handle the velocity challenge at the visualization layer without overloading the browser; (d) which tools form the pipeline from Kafka to browser render.

Givens: 10,000 tweets/s → **600,000** per minute, **864,000,000** per day.

(a) 1-minute tumbling window per topic: count, mean sentiment, % positive/neutral/negative. One point replaces 600,000 tweets, smooths noise, still shows drift. Optional 15-min sliding window for a smoother trend.

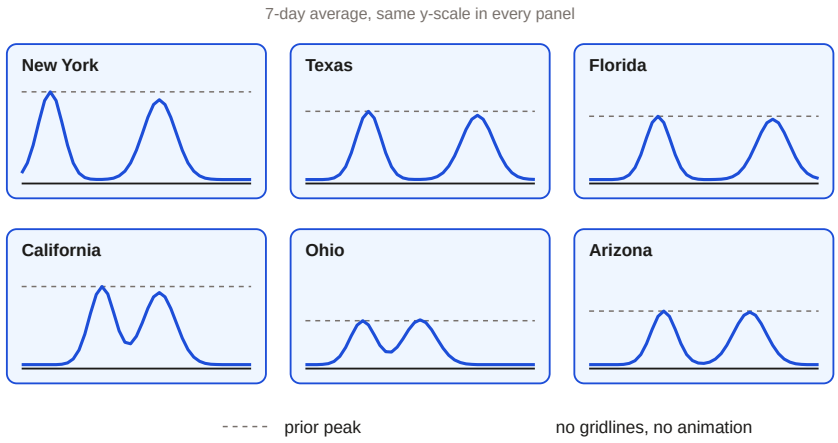
(b) Drift: line chart (or 100% stacked area) per topic as small multiples. Clustering: t-SNE/UMAP scatter drawn as hexbin or KDE to avoid overplotting.

(c) Send only window aggregates (one refresh of 5 s replaces 50,000 tweets); keep a rolling buffer of the last 60 points; throttle redraws; pre-bin the cluster view server-side.

(d) X API → Kafka → Spark Structured Streaming/Flink → InfluxDB/Prometheus or Elasticsearch → Grafana (5 s poll) or Kibana; WebSocket → D3.js for sub-second views.

DISCUSSION

Lesson 1. Data quality is a visualisation problem: the NYT's biggest engineering effort was cleaning data, not drawing charts; retroactive state corrections forced D3 charts to support negative bars (days where cases were revised down)



Illustrative NYT-style small multiples: one 7-day-average chart per state on a shared y-scale, no gridlines, one light reference line at the prior peak.

Claim: a chart is only as right as its data.

For: most NYT work was normalising 50 state schemas; downward revisions made daily changes negative, so chart design had to show negative bars.

Against: cleaning is really a data-engineering task, and no chart design can fix bad source data. Even so, the chart must show corrections honestly (negative bars) rather than hide them.

DISCUSSION

Lesson 2. Accessibility is non-negotiable at 6M readers/day: every chart had a data table alternative for screen readers; colour scales were tested for colour-blindness (deuteranopia, protanopia)

Claim: at 6 M readers a day, even a small share of blind or colour-blind readers is many people.

For: a screen reader cannot describe the shapes of a chart, so a data table gives blind readers the same facts; avoiding red/green scales serves red-green colour-blind readers of public-health data.

Against: tables and testing cost time on daily deadlines and restrict palettes; the NYT judged it worth paying.

DISCUSSION **Lesson 3.** Small multiples beat animation: the 50-state grid (Tufte small multiples) was more actionable than an animated map of spreading cases

Claim: showing every state at once beats one frame at a time.

For: a shared scale lets readers compare states side by side and find their own without waiting or remembering frames.

Against: animation grabs attention and shows geographic spread vividly; 50 small charts can be dense on a phone. Grid for analysis, animation for a headline.

DISCUSSION **Lesson 4.** Rolling averages are editorial decisions: choosing 7-day vs. 14-day average changes what story the chart tells; these are journalistic choices with ethical weight

Claim: the window length shapes the story readers see.

For 7-day: removes weekend reporting dips and reacts to a real turn within days.

For 14-day: smoother, fewer false alarms, but hides a turning point longer.

Readers made health decisions from it, so the choice should be stated on the chart.

DISCUSSION **Lesson 5.** GitHub as data infrastructure: publishing the raw CSV on GitHub enabled 1,000+ derivative dashboards worldwide within days

Claim: open data multiplies the value of expensive cleaning.

For: others reused it for 1,000+ dashboards; version history makes corrections traceable and numbers checkable.

Against: GitHub is not a database; others can misread the data, and the publisher must keep files consistent as corrections arrive.

LIVE EXERCISE

Task. Live Exercise — D3.js Bar Chart with Tufte Styling Task (25 minutes, pairs) Build a minimal D3.js bar chart that follows Tufte's data-ink principles: 1. No gridlines (or only a single light reference line) 2. y-axis starts at 0 3. No fill gradient — use a flat colour 4. Tooltip on hover (Shneiderman: details-on-demand) 5. Add a sparkline for each bar showing the 7-day trend Dataset: weekly Kafka message counts for 5 topics (generate 7×5 random values). Extension: add a dropdown filter to select one topic and zoom into its 7-day line chart.

```
<!DOCTYPE html>
<meta charset="utf-8">
<style>
  .bar { fill: steelblue; }
  .bar:hover { fill: #b5651d; }
  .axis path, .axis line {
    stroke: #ccc; stroke-width: 0.5px; }
  .tooltip {
    position: absolute; background: #fff;
    border: 1px solid #ccc; padding: 6px;
    font-size: 11px; pointer-events: none; }
</style>
<body>
<script src=
  "https://d3js.org/d3.v7.min.js"></script>
<script>
const topics = ["orders","clicks",
                "errors","logins","search"];
const data = topics.map(t => ({
  topic: t,
  count: Math.floor(Math.random()*50000)
         + 10000
}));
const svg = d3.select("body").append("svg")
  .attr("width", 500).attr("height", 300);
const x = d3.scaleBand()
  .domain(data.map(d=>d.topic))
  .range([50,480]).padding(0.3);
const y = d3.scaleLinear()
  .domain([0, d3.max(data,d=>d.count)])
  .range([260, 20]);
svg.append("g").attr("transform","translate(0,260)")
  .call(d3.axisBottom(x));
svg.append("g").attr("transform","translate(50,0)")
  .call(d3.axisLeft(y).ticks(4));
svg.selectAll(".bar").data(data).join("rect")
  .attr("class","bar")
  .attr("x",d=>x(d.topic))
  .attr("y",d=>y(d.count))
  .attr("width",x.bandwidth())
  .attr("height",d=>260-y(d.count));
</script>
```

Key code (added to the skeleton; 7 daily values per topic, bar = weekly sum; SVG height raised to 340 for the sparklines; CSS `.ref { stroke: #ddd; stroke-dasharray: 3 3; }`):

```
const data = topics.map(t => { const days = d3.range(7)
  .map(() => Math.floor(Math.random()*50000) + 10000);
  return {topic: t, days, count: d3.sum(days)}; });
svg.selectAll("g").attr("class","axis");   // skeleton forgot this
const mean = d3.mean(data, d => d.count);  // one light line at the mean
svg.append("line").attr("class","ref").attr("x1",50).attr("x2",480).attr("y1",y(mean)).attr("y2",y(mean));
const tip = d3.select("body").append("div").attr("class","tooltip").style("opacity",0);
svg.selectAll(".bar").on("mouseover",(e,d)⇒tip.style("opacity",1)
      .style("left",(e.pageX+8)+"px").style("top",(e.pageY-28)+"px")
      .html(d.topic+": "+d.count))
    .on("mouseout",()⇒tip.style("opacity",0));
const sx = d3.scaleLinear().domain([0,6]).range([0,x.bandwidth()]);
data.forEach(d => { const sy = d3.scaleLinear()
  .domain(d3.extent(d.days)).range([325,290]);
```

```
svg.append("path").attr("fill","none").attr("stroke","#555").attr("transform",`translate(${x(d.topic)},0)`)
    .attr("d", d3.line().x((v,i)⇒sx(i)).y(v⇒sy(v))(d.days)); });
```

1. **Gridlines:** D3 axes draw none; add only one dashed light line at the mean.
2. **y from 0:** domain `[0, max]`; a higher start would push the lie factor above 1.
3. **Flat colour:** `fill: steelblue`, no gradient.
4. **Tooltip:** mouseover shows topic and count; mouseout hides it. This is details on demand.
5. **Sparkline:** 7-point line, no axes, under each bar, own min-max scale (shape, not size).

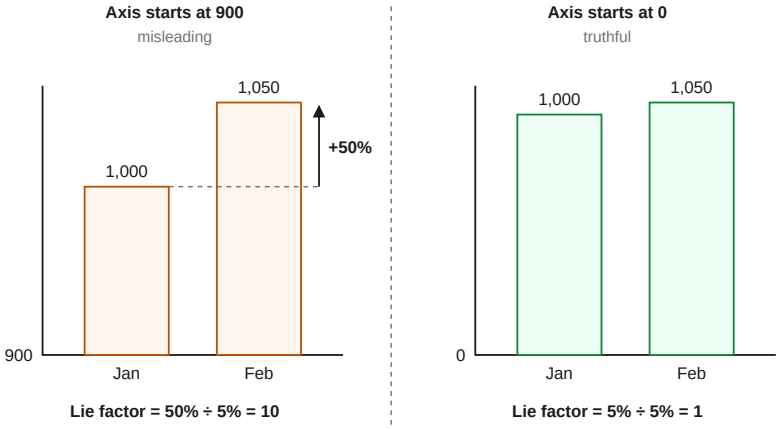
Extension: a `select` of the 5 topics; on change, fade other bars and draw that topic's 7-day line chart below.

Expected: 5 flat bars; tallest reaches y = 20 (240 px); bar width ≈ 56.8 px (step 430 ÷ 5.3 ≈ 81.1); weekly totals between 70,000 and 419,993.

Topic cards

6 topics · everything the pool tests

Tufte: data-ink, chartjunk, lie factor



A y-axis starting at 900 makes a 5% rise (1,000 → 1,050) look like a 50% jump, lie factor 10; starting at 0 keeps it at 1.

FACTS

- Edward Tufte, *The Visual Display of Quantitative Information* (1983). Motto: "Above all else, show the data."
- **Data-ink ratio** = ink encoding data ÷ total ink in the chart. Goal: make it as high as possible (maximum 1).
- **Chartjunk**: ink that carries no data. Slide list: 3-D effects, gradient fills, redundant legends, heavy gridlines, decorative borders, vibrations, ducks.
- **Lie factor** = size of effect shown in graphic ÷ size of effect in data. 1 = truthful, above 1 overstates, below 1 understates.
- **Small multiples** (trellis or facet chart): the same small chart repeated per subset (district, year), all on the same axis scale.
- **Sparkline**: a word-sized trend line placed inside text or a table cell, next to the number it describes.

HOW IT WORKS

1. List every element in the chart: bars, axes, gridlines, legend, title, borders, shading
2. For each element ask: if I erase it, does the chart lose information?
3. Erase it if not: 3-D depth, gradients, borders, a legend for a single series
4. Lighten what must stay: thin light axes, at most one light reference line
5. Label data directly on the axis instead of in a legend
6. Check the y-axis starts at 0 so the lie factor stays near 1

WHY · PROS · LIMITS

COMPARE

- Non-data ink competes with the data for the reader's attention, so the trend takes longer to see.
- 3-D depth and perspective distort bar heights: never use 3-D for 2-D data.
- A cut y-axis inflates differences; the lie factor puts a number on that distortion.
- Small multiples let the eye compare all panels at once; a shared scale makes the comparison valid.
- Limitation: the ratio is about what ink shows, not how little ink is used. One thick bar for one key value still scores high.

	Data-ink	Chartjunk
What it is	Ink that encodes a value	Ink that encodes nothing
Examples	Bar length, data point, trend line	3-D shading, gradient, heavy grid, single-series legend
Erase test	Erasing loses information	Erasing loses nothing
Effect on ratio	Raises it	Lowers it

	Small multiples	Sparkline	Animation
Shows	Same chart per subset, grid	Tiny trend inside text/table	One frame at a time
Comparison	All subsets at once, same scale	Trend shape next to one number	Viewer must remember earlier frames
Axes	Shared scale	None; shows shape and direction only	Changes over time

CALCULATION

Lie factor (P2 slide 7). y-axis starts at 50 instead of 0. Data rises +10%, but the bar doubles in height (+100%). Lie factor = 100 ÷ 10 = **10**.

Data-ink ratio. 1,000 units of ink, 300 draw bars: 300 ÷ 1,000 = **0.3**. Remove 500 units of junk: 300 ÷ 500 = **0.6**.

Step by step

1. Given old and new data values → effect in data = (new − old) ÷ old
2. Given old and new drawn heights → effect shown = (new height − old height) ÷ old height
3. Given both effects → lie factor = shown ÷ data; compare with 1
4. Given data-ink and total ink → ratio = data-ink ÷ total ink
5. Given ink removed as junk → new total = old total − removed; data-ink unchanged; recompute ratio

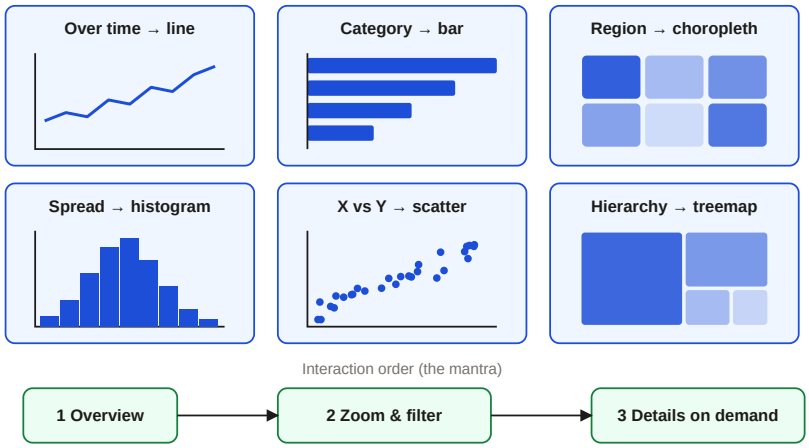
Variants

- Revenue 100 → 110 (+10%); bars drawn 40 px → 60 px (+50%). Lie factor = 50 ÷ 10 = **5** (overstates)
- 400 data-ink + 600 decoration: ratio = 400 ÷ 1,000 = **0.4**; delete all decoration: 400 ÷ 400 = **1.0**
- Data +20%, drawn +10%: lie factor = 10 ÷ 20 = **0.5** (understates)

TRAPS

- Data-ink ratio is not about saving ink or printing cost (MCQ option (c)).
- Lie factor uses percentage changes, not raw heights: divide the two changes, not the two bars.
- Small multiples need the same scale on every panel, or cross-panel comparison is invalid.
- Lie factor below 1 is also a distortion (understating), not "extra honest".

Shneiderman's mantra and choosing the chart



Pick the chart from the question and data type, then build the interaction in the mantra's order: overview, zoom and filter, details on demand.

FACTS

- Ben Shneiderman, "The Eyes Have It" (1996). Mantra: **"Overview first, zoom and filter, then details on demand."**
- **Seven data types**: 1-D (linear), 2-D (map), 3-D (world), temporal, multi-dimensional, tree, network.
- **Seven tasks** (P2 slide 9), the same for every data type: overview, zoom, filter, details-on-demand, relate, history, extract. P1 lists only the first five.
- **Treemap** (Shneiderman, 1991): nested rectangles, area ∝ size. **Dynamic queries**: sliders and buttons that redraw the chart in under 100 ms (ancestor of Tableau's filter shelf).
- **Anscombe's Quartet** (1973): same mean x 9.0, mean y 7.5, variance x 11.0, r 0.816, fit y = 3 + 0.5x; four different shapes.
- **Pie chart**: only for ≤ 4 categories and the question "what fraction of the whole?" People judge angles worse than lengths.
- **Choropleth**: regions shaded by a rate or proportion. For absolute counts use a proportional bubble map.

HOW IT WORKS

1. Overview: show the whole dataset first (national total, whole map)
2. Zoom: narrow to a region or time range of interest
3. Filter: drop items you do not need (dropdown, slider)
4. Details on demand: hover or click to get the raw record of one item
5. Relate: cross-highlight the same item across linked charts
6. To pick each chart: write the question, classify the data type, take the recommended chart, check the avoid list

WHY · PROS · LIMITS

- The user needs global context before knowing what to filter; filtering first hides where to look.
- Details for every item at once would swamp the screen, so they appear only on request.
- Equal summary statistics can hide different shapes (Anscombe), so plot before trusting a mean: visualization is the first quality check.
- Bar lengths on a common baseline are read more accurately than angles or areas.

COMPARE

Data type	Example	Best chart
1-D linear	Ranked list	Bar chart, histogram
2-D map	Districts	Choropleth, bubble map
3-D world	MRI, CT volumes	3-D scatter, volume render
Temporal	Time series	Line, area, calendar heatmap

- Choropleths of raw counts just show where people live; rates remove population size.

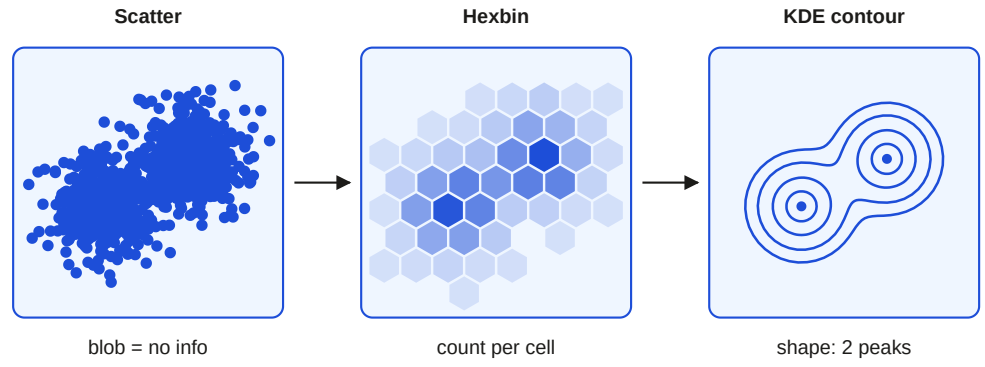
Multi-dimensional / tree	Census (10+ vars) / file system	Scatterplot matrix, parallel coords / treemap, sunburst
Network	Social graph	Force-directed graph, chord diagram

Question	Use	Avoid
How did X change over time?	Line, area, calendar heatmap	Pie chart
Which category is largest?	Horizontal bar (more than 5 cats), grouped bar	3-D pie
How is X spread across regions?	Choropleth (rate), bubble map (count)	Plain bar
What is the spread of X?	Histogram, box plot, violin	Line chart
Does X predict Y?	Scatterplot, scatterplot matrix	Bar chart
Pattern across more than 3 variables?	Parallel coordinates, t-SNE/UMAP	3-D bar

TRAPS

- Overview comes first. "Filter first" is the classic wrong option.
- A pie chart for more than 5 categories is never recommended (T/F trap).
- A histogram shows spread; a treemap shows hierarchy, not spread.
- The same r = 0.816 does not mean the same relationship (Anscombe).
- Seven data types (1-D … network) and seven tasks (overview … extract) are different lists; do not mix them.

Overplotting: alpha, hexbin, KDE



Same 1,500 points drawn three ways

The same points as an overplotted scatter (a blob), a hexbin (count per hexagon) and KDE contours (two peaks revealed).

FACTS

- **Overplotting**: with millions of points, dots cover each other and the scatterplot becomes a solid blob.
- Three fixes in increasing sophistication: alpha transparency, hexbin aggregation, KDE density contours.

HOW IT WORKS

1. Spot the blob: the scatter shows only that data exists
2. Try alpha: set opacity low so stacked points darken the area
3. If dense areas all turn black, bin instead: divide the plane into hexagons

- **Alpha transparency**: each point nearly see-through (slide: α = 0.02), so dense areas look darker.
- **Hexbin**: hexagonal grid, count points per hex, colour by count. 10 million points become about 10,000 cells.
- **KDE** (kernel density estimation): a smooth density surface drawn as contour lines; shows shape, modes and tails.

4. Count the points in each hexagon and colour it by the count
5. For shape, estimate a KDE and draw contours at chosen probability levels

WHY · PROS · LIMITS

- Alpha keeps relative density, but stacked points saturate to full black at very high densities.
- Hexbin keeps the count per area and draws thousands of cells instead of millions of dots, so it stays interactive.
- KDE keeps the shape of the distribution: peaks (modes), clusters and tails.
- Limitation of binning: individual points and outliers inside a cell are lost.

COMPARE

	Alpha	Hexbin	KDE contour
How	Low opacity per point	Count per hexagon, colour by count	Smooth density, contour lines
Preserves	Relative density	Exact count per area	Shape, modes, tails
Weakness	Saturates at high density	Hides points inside a cell	Smooths away detail; needs a bandwidth
Marks drawn	All N points	≈ number of cells	A few contour lines

CALCULATION

Hexbin (P1 slide 17): 10,000,000 points → ≈ 10,000 hex cells. Reduction = 10,000,000 ÷ 10,000 = **1,000×** fewer shapes.

Alpha saturation (standard compositing, explains "fails at very high densities"): opacity after n stacked points = $1 - (1 - \alpha)^n$. At α = 0.02, n = 100 gives **0.867**; n = 200 gives **0.982**.

Step by step

1. Given points and cells → reduction = points ÷ cells
2. Given α → single-point transparency = 1 − α
3. Given n stacked points → combined opacity = $1 - (1 - \alpha)^n$
4. Given target opacity p → n = ln(1 − p) ÷ ln(1 − α)

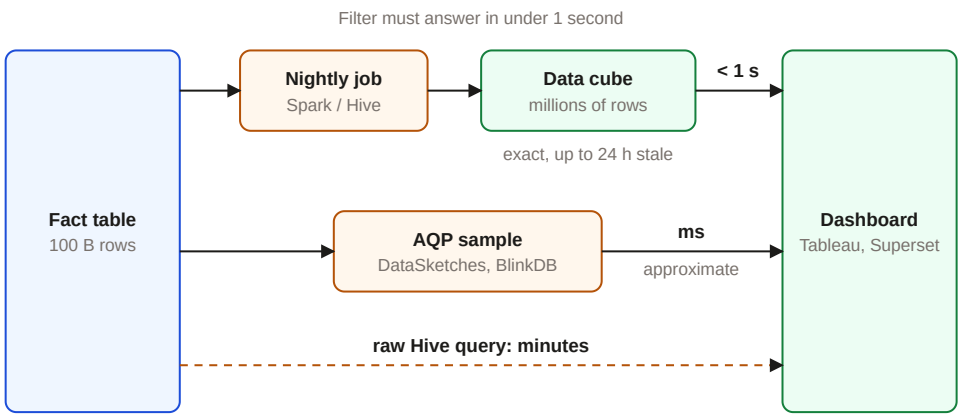
Variants

- 2,000,000 GPS pings into 500 hexes: 2,000,000 ÷ 500 = **4,000×**
- α = 0.02, n = 10: $1 - 0.98^{10}$ = **0.183**
- Points for 50% opacity at α = 0.02: ln 0.5 ÷ ln 0.98 ≈ 34.3, so **35** points

TRAPS

- Alpha is the simplest fix but still fails at very high density; do not call it the best.
- Hexbin shows counts, KDE shows shape; do not swap what each preserves.
- Hexbins are True for reducing overplotting (T/F).

Latency: data cubes and AQP



A nightly job pre-aggregates the fact table into a cube read in under 1 s (exact but up to 24 h stale); AQP answers from a sample in milliseconds (approximate); a raw Hive query takes minutes.

FACTS

- Analysts expect a dashboard filter to answer in under 1 second; a Hive query over 100 billion rows takes minutes.
- **Fact table**: one row per event (each sale). **Data cube**: pre-computed aggregates for combinations of dimensions, e.g. district × date × product × channel.
- The cube is built by a scheduled (nightly) Spark/Hive job and holds millions of rows, not billions.
- **AQP** (approximate query processing): answer from a sample in milliseconds. Tools: Apache DataSketches, BlinkDB.
- Cube trade-off: staleness up to 24 h for speed. AQP trade-off: accuracy for speed.

HOW IT WORKS

1. A nightly Spark or Hive job scans the fact table once
2. It writes SUM/COUNT aggregates for each combination of dimensions
3. Tableau or Superset points at the cube, not the raw table
4. A filter such as "Q1 2024 only" reads millions of pre-summed rows
5. The answer returns in under 1 second; the cube refreshes on the next run

WHY · PROS · LIMITS

- Reading millions of rows is far faster than scanning billions on every click.
- Staleness: the cube is only as fresh as its last build (up to 24 h).
- Fixed questions: only pre-computed dimensions can be sliced; a new one needs a rebuild.
- Storage and batch compute grow with the number of dimension combinations.
- AQP is fine for dashboards but not for financial reconciliation, where exact totals are required.

COMPARE

	Raw Hive query	Data cube	AQP
Reads	All 100 B rows	Millions of pre-summed rows	A sample
Latency	Minutes	Under 1 s	Milliseconds
Accuracy	Exact, current	Exact, but stale	Approximate
Cost	Slow every click	Nightly build, storage	Error margin
Unfit for	Interactive use	Live data, new dimensions	Financial reconciliation

CALCULATION

Slide: 100 billion fact rows; cube of millions of rows. Taking 5 million: 100,000,000,000 ÷ 5,000,000 = **20,000×** fewer rows to read; the cube is **99.995%** smaller.

Step by step

1. Given distinct values per dimension → finest cube rows ≤ $d_1 \times d_2 \times \ldots \times d_k$
2. Given fact rows and cube rows → reduction = fact rows ÷ cube rows
3. Given k dimensions → group-by combinations = 2^k (including the grand total)

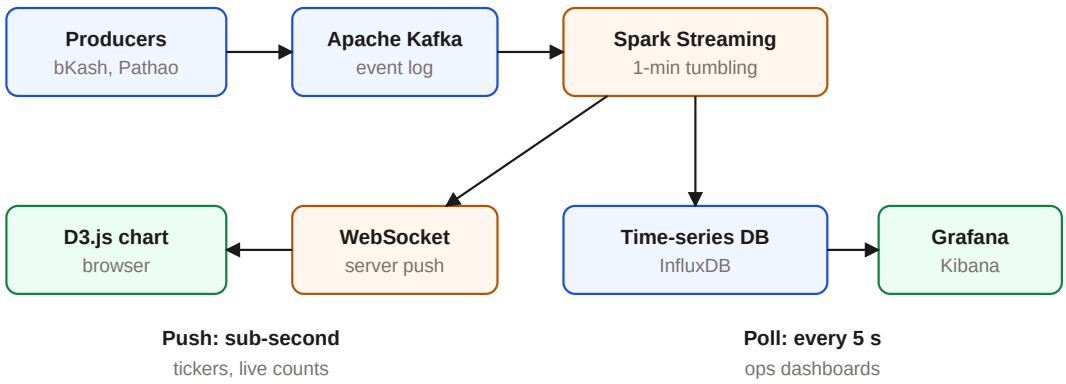
Variants

- 365 dates × 64 districts × 50 products × 4 channels = **4,672,000** rows at most
- 100 billion ÷ 4,672,000 ≈ **21,404×** fewer rows
- k = 4 dimensions → 2^4 = **16** group-by combinations

TRAPS

- The cube's trade-off is staleness (and fixed dimensions), not wrong answers; AQP's trade-off is accuracy.
- The cube is built in batch ahead of time, not when the user clicks.
- Finest cube size is an upper bound: only combinations that occur are stored.

Streaming dashboards and tools



Kafka events are window-aggregated by Spark Streaming, then either stored in a time-series DB that Grafana/Kibana polls every 5 s, or pushed over a WebSocket to a D3.js chart for sub-second updates.

FACTS

- Pipeline: producers → **Kafka** (durable event log) → **Spark Structured Streaming** or Flink window aggregation → time-series DB → **Grafana** or Kibana.
- Slide window: **1-minute tumbling** (fixed, non-overlapping) window. Time-series DBs: InfluxDB, Prometheus, Elasticsearch.
- **Poll**: Grafana re-queries the DB on a schedule (5 s refresh). **Push**: for sub-second updates, a WebSocket server pushes to a D3.js chart.
- Tools: business users → Tableau/Power BI; custom web charts → D3.js; real-time ops → Grafana; data-lake BI → Superset; logs → Kibana.

HOW IT WORKS

1. Events arrive at a Kafka topic in real time
2. Spark Structured Streaming or Flink aggregates over a window (e.g. 1-minute tumbling)
3. Aggregates are written to a time-series DB
4. Grafana or Kibana polls the DB every few seconds and redraws
5. For sub-second needs, the server pushes each aggregate over a WebSocket to a D3.js chart

- **D3.js** (Data-Driven Documents) binds data to DOM elements, so output is interactive SVG in the browser, not PNG/PDF files.

WHY · PROS · LIMITS

- A browser cannot redraw a chart for each of thousands of raw events per second; windows turn them into a few points per minute.
- Grafana reads time-series DBs natively and auto-refreshes, so it suits ops monitoring with no custom code.
- WebSocket push avoids polling delay, needed for live election counts or stock tickers.
- Tableau and Power BI support only limited scheduled refresh; D3 can do real time but needs custom development.

COMPARE

Tool	Best for	Real-time
Tableau	Business dashboards, drag-and-drop	Limited
Power BI	Microsoft ecosystem, executive reports	Limited
D3.js	Custom interactive web charts	Yes (WebSocket)
Grafana	Real-time metrics, ops dashboards	Yes (5 s poll)
Superset	Open-source BI on data lakes (Hive, Spark SQL)	Limited
Kibana	Log search on Elasticsearch	Yes

	Poll (Grafana)	Push (WebSocket + D3)
Who starts	Browser asks every few seconds	Server sends on each new value
Latency	Seconds	Sub-second
Effort	Configuration	Custom code

CALCULATION

Pool A2: 10,000 tweets/s with a 1-minute tumbling window → 10,000 × 60 = **600,000** tweets summarised into one point per topic. Per 5 s refresh: **50,000** tweets. Per day: 10,000 × 86,400 = **864,000,000**.

Step by step

1. Given event rate and window length → events per window = rate × seconds
2. Given refresh interval → events between refreshes = rate × interval
3. Given update interval → updates per day = 86,400 s ÷ interval

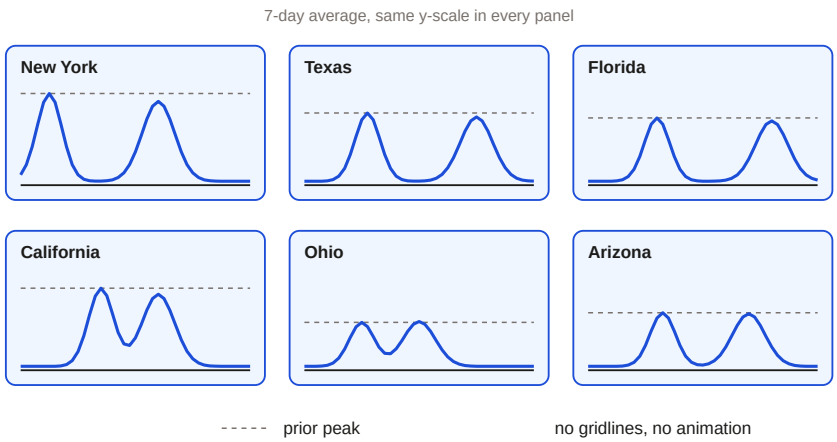
Variants

- Update every 5 minutes (A1(b)): 24 × 60 ÷ 5 = **288** points per day
- 1,000 events/s, 1-min window: **60,000** events per point

TRAPS

- Tumbling windows do not overlap (each event counted once); sliding windows overlap.
- D3 can do real time, but the 5 s ops dashboard answer is Grafana: native time-series support, no custom code.
- D3.js output is SVG in the DOM, not static images (T/F).
- Send aggregates, never raw events, to the browser.

Case study: NYT COVID-19 tracker



Illustrative NYT-style small multiples: one 7-day-average chart per state on a shared y-scale, no gridlines, one light reference line at the prior peak.

FACTS

- NYT Graphics Desk: ≈ 50 journalists, designers and engineers; 6 million readers per day. Mike Bostock built D3.js there.
- Data: Johns Hopkins CSSE, CDC, 50 state health departments, with inconsistent schemas, missing values and retroactive corrections.
- Pipeline: Python scrapers → pandas cleaning → GitHub CSV → D3 charts updated every hour.
- Design: 50-state small-multiples grid, no gridlines (one light line at the prior peak), 7-day rolling averages, county choropleths.
- Mantra: national total (overview) → state dropdown and date slider (filter) → hover tooltip (details).

HOW IT WORKS

1. Scrape data from many sources
2. Normalise schemas and fix missing values in pandas
3. Publish the clean CSV to GitHub
4. Compute 7-day rolling averages
5. Render D3 charts hourly, with data-table alternatives for screen readers

WHY · PROS · LIMITS

- The hardest work was data cleaning across 50 states, not drawing charts.
- Corrections made some daily changes negative, so charts had to allow negative bars.
- A 7-day average removes day-of-week reporting dips; the window choice changes the story.
- Small multiples let readers compare all states at once, unlike an animated map.
- At 6 M readers a day, accessibility (tables, colour-blind-safe palettes) reaches many people.

COMPARE

	7-day	14-day
Removes weekly cycle	Yes	Yes
Reacts to a real turn	Within days	Later
Noise	More	Less

TRAPS

- The lesson is that cleaning, not rendering, was the biggest effort.
- A rolling average is an editorial choice, not a neutral fix.
- Deuteranopia and protanopia are red-green colour-blindness: avoid red/green scales.

CHAPTER 6 · WEEK 10

Big data in real applications

This chapter applies the course's stack (Kafka, Spark, HDFS, Hive, Grafana) to three domains: social media graphs, time series forecasting and healthcare data. A flu surveillance case study shows what big data signals can and cannot replace.

Exam question pool

22 questions from the slides · click a question for its answer

MCQ **Q1.** Facebook's Prophet library is primarily designed for:

(a) Graph analytics and community detection in social networks
(b) Real-time stream processing of events
(c) Time series forecasting with decomposable trend, seasonality, and holiday components
(d) Text sentiment classification using deep learning

(c) Time series forecasting with decomposable trend, seasonality, and holiday components
Prophet fits $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$.

MCQ **Q2.** Which algorithm is most commonly used for community detection in large-scale social network graphs?

(a) K-Means clustering
(b) PageRank
(c) Louvain algorithm
(d) ARIMA

(c) Louvain algorithm
It maximises modularity in O(n log n); PageRank ranks importance, not groups.

MCQ **Q3.** HIPAA in the United States primarily regulates the privacy and security of:

(a) Financial transaction data and bank records
(b) Healthcare data including patient records and medical information
(c) Social media user data and behavioural profiles
(d) Government data retention and public records

(b) Healthcare data including patient records and medical information
HIPAA is the Health Insurance Portability and Accountability Act; financial data falls under GLBA.

MCQ **Q4.** DeepMind's AlphaFold used big data and deep learning to solve:

(a) Real-time financial fraud detection at bank-card transaction scale
(b) Social network influence maximisation over billion-node graphs
(c) The protein structure prediction problem, predicting 3D structure from amino acid sequence
(d) Time series anomaly detection in IoT sensor streams

(c) The protein structure prediction problem, predicting 3D structure from amino acid sequence
Trained on 170,000 PDB structures with MSAs from 250M sequences; 2024 Nobel Prize in Chemistry.

TRUE/FALSE **1.** Google Flu Trends successfully replaced traditional CDC flu surveillance and has been used as the primary source ever since.

False. GFT over-predicted flu by 2× in 2012-2013 and was discontinued.
Corrected: Google Flu Trends failed, and CDC still relies on clinical reporting, with big data signals only as a supplement.

TRUE/FALSE **2.** Time series data is order-sensitive: randomly shuffling the records in a time series changes the analysis result.

True. Autocorrelation, trend and seasonality exist only in time order; shuffling destroys them.

TRUE/FALSE **3.** Social network graphs are typically sparse, with most nodes connected to only a small fraction of all possible neighbours.

True. Real social networks are scale-free (power-law degrees): most nodes have few links and a few hubs have millions. Twitter

had 1.47B edges among 41.7M users, about 35 follows each out of millions possible.

TRUE/FALSE **4.** Medical imaging data such as MRI and CT scans is classified as structured data because it is stored in files.

False. Being in a file does not make data structured.
Corrected: MRI and CT scans are unstructured binary DICOM files (200 MB to 2 GB), analysed by deep learning, not SQL.

DESCRIPTIVE **D1.** Describe three types of analysis commonly performed on social media data (e.g., community detection, sentiment analysis, influence maximisation). For each, state one specific ethical risk that arises when the analysis is deployed at scale.

Analysis	What it does	Ethical risk at scale
Community detection	Louvain finds groups more densely linked inside than outside, by maximising modularity.	Profiling: group labels expose political or religious ties users never shared.
Sentiment analysis	VADER scores each post in [−1, +1]; > 0.05 positive, < −0.05 negative.	Suppressing dissent: flagging people who criticise a government policy.
Influence maximisation	Picks k seed users for widest spread; greedy gives ≥ 63% of optimal.	Manipulation: spreading misinformation or microtargeting vulnerable users.

Safeguard for all three: transparency and purpose limitation.

DESCRIPTIVE **D2.** What is the difference between ARIMA and Facebook Prophet for time series forecasting? Under what conditions would you prefer Prophet over ARIMA? When might ARIMA still be the better choice?

ARIMA(p, d, q) predicts from past p values and past q errors after differencing d times to make the series stationary. **Prophet** adds up trend with changepoints, Fourier seasonality and a holiday term: $y = g + s + h + \varepsilon$.

Prefer Prophet when the series has several seasonalities, holiday effects (Eid, Puja), trend shifts, missing data or outliers, or when thousands of series must be fitted automatically (5,000 SKUs in 12 minutes with applyInPandas).

ARIMA is still better for a short, stationary series with no strong seasonality and little data, where an expert can tune one important series closely.

DESCRIPTIVE **D3.** Identify three data integration and privacy challenges specific to healthcare big data analytics. For each, describe one technical mitigation strategy (e.g., differential privacy, federated learning, de-identification).

Challenge	Mitigation
Records carry names, DOB, NID, phone; HIPAA bars research use as-is	**De-identification:** safe harbour removes the 18 identifiers, or expert determination certifies low risk
Models need 1M+ records, but data cannot leave each hospital (consent, PDPA)	**Federated learning:** train at each hospital, share only model updates
Published counts for small areas can reveal one patient	**Differential privacy:** add calibrated noise to aggregates

Integration itself: FHIR, Parquet genomics, Kafka wearables and DICOM images meet in one HDFS/S3 lake with a Hive Metastore.

ANALYTICAL **A1.** Flu Outbreak Early-Warning System for Bangladesh: Design a big data system to detect flu outbreaks in Bangladesh using Twitter/Facebook posts and pharmacy purchase records from major pharmacy chains. (a) How would you collect and ingest these two data sources? (Specify tools: Kafka, Sqoop, Flume, REST API) (b) What features would you extract from each source for a prediction model? (Name specific NLP features and pharmacy signal metrics) (c) What model would you use, and how would you validate it against ground-truth CDC/DGHS surveillance data? (d) What are the ethical risks of this system? How would you mitigate them?

Design: sources → Kafka → Spark Streaming → alert engine + Grafana (the CDC pattern).

Part	Social posts	Pharmacy sales
(a) Ingest	REST APIs (X API v2, Facebook Pages), keyword filter in Bangla and English, producer writes to Kafka topic `social_raw`	Sqoop for nightly bulk import from the chains' databases; Flume ships POS logs to Kafka topic `pharmacy_sales`

(b) Features	Daily flu-keyword post count per district; share of symptom terms (fever, cough, 'jor'); first-person illness flag; VADER/Bangla sentiment; ratio to 4-week baseline	Daily paracetamol and cough-syrup units per district; week-on-week % change; z-score vs same week last year

(c) Model: Prophet per district on DGHS weekly ILI counts, yearly seasonality, Eid holidays, social and pharmacy features as extra regressors; run districts in parallel with applyInPandas. **Validate** with a rolling backtest against DGHS counts: error (MAE), lead time, hit rate, false alarms, and test on an unseen season (GFT had $r^2 = 0.97$ in-sample and still failed).

(d) Ethics: privacy (keep district counts only, drop user IDs, add differential-privacy noise); panic from false alarms (an officer confirms before any announcement); bias against rural and older people who post less (weight with pharmacy and DGHS data); misuse for watching individuals (purpose limitation, access logs, PDPA consent).

ANALYTICAL

A2. Retail Demand Forecasting Pipeline: A large Bangladeshi supermarket chain needs 90-day demand forecasts for 5,000 store–product SKU combinations. Design the end-to-end big data pipeline: (a) data ingestion from POS systems; (b) storage and partitioning strategy in HDFS/S3; (c) forecasting model choice (ARIMA vs. Prophet vs. LSTM) and justification; (d) how you parallelise model fitting across 5,000 SKUs using Spark; (e) how forecasts are served to the inventory management dashboard. Identify the primary bottleneck in your pipeline.

Stage	Design
(a) Ingest	Tills send sale events to Kafka topic `pos_sales`; Sqoop bulk-loads history; Spark aggregates to (sku_id, date, units)
(b) Storage	Raw Parquet on HDFS/S3 partitioned by date; daily aggregate Hive table partitioned by date, bucketed by sku_id; forecasts partitioned by run_date
(c) Model	**Prophet:** weekly and yearly seasonality, holidays (Eid, Puja, Pohela Boishakh), automatic changepoints, tolerates gaps. ARIMA needs hand tuning per series; LSTM needs much data, GPUs and is a black box
(d) Parallel fit	`daily.groupBy("sku_id").applyInPandas(fit, schema)`; each group goes to one task, Prophet fits inside the Pandas UDF
(e) Serve	Write Parquet to Hive/S3; load latest run into a serving table; dashboard (Superset/Grafana) shows yhat with bands and reorder alerts

Numbers: 8 h = 480 min sequential; 12 min on 20 executors; speedup = 480 / 12 = **40×**; 5.76 s per model; 250 models per executor.

Bottleneck: model fitting (Prophet via Stan, the 8-hour step). Skewed SKUs with long histories finish last; cap history and add cores.

DISCUSSION

1. Kwak et al. found Twitter's average shortest path is 4.12 hops over 41.7M users. If a misinformation tweet is retweeted by a high-PageRank account, estimate how many users it reaches within 3 hops assuming an average out-degree of 45. What graph property makes this estimate an overestimate? [CLO4 — Apply]

Estimate: $45 + 45^2 + 45^3 = 45 + 2{,}025 + 91{,}125 =$ **93,195 users** (0.22% of 41.7M).

For an overestimate: clustering. Followers overlap, so the tree counts the same people again; most viewers also do not retweet.

Against: a high-PageRank account may have over a million followers, so hop 1 alone can beat 45.

DISCUSSION

2. A Prophet model for weekly hospital admissions has strong annual and weekly seasonality but misses a sudden COVID-19 wave onset. Identify which Prophet component should detect this changepoint and explain what prior hyperparameter controls the model's sensitivity to new changepoints. [CLO4 — Analyse]

Component: the trend g(t). A wave is a structural break, not a repeating cycle or a listed holiday.

Hyperparameter: `changepoint_prior_scale`, the scale of the sparse prior on the slope changes δ. Default 0.05 keeps the trend stiff and smooths over the wave; raising it (e.g. 0.5) lets the trend bend.

Against raising it: the trend then chases noise and the forecast band widens, so tune on held-out weeks.

DISCUSSION **3.** Design a dengue fever surveillance system for Bangladesh using BioSense-style ED data, Twitter signals, and Prophet forecasting. Specify: data sources, pipeline architecture (Kafka + Spark), model inputs, and how you would validate the system before policy use. [CLO4 — Design]

- **Sources:** ED visits (FHIR) as main signal, NS1 lab results, dengue tweets as leading signal, rainfall.
- **Pipeline:** Kafka topics → Spark Streaming de-identifies and aggregates per district → Parquet → nightly Prophet per district via applyInPandas → alerts + Grafana.
- **Inputs:** weekly dengue-like visits, yearly (monsoon) seasonality, Eid holidays, tweets and rainfall as regressors.
- **Validate:** backtest on past seasons and an unseen season; compare with and without Twitter.

For: earlier warning. **Against:** tweets skew urban and young, so ED data stays primary.

LIVE EXERCISE **Task.** Live Exercise — Prophet Flu Forecasting in Python. Task (25 minutes, pairs): Fit a Prophet model to synthetic weekly flu incidence data and produce a 12-week forecast. Install: pip install prophet matplotlib pandas

```
import pandas as pd
import numpy as np
from prophet import Prophet
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
dates = pd.date_range("2021-01-04",
                      periods=156, freq="W")
# Annual flu seasonality + noise
ili = (5 + 3*np.sin(
          2*np.pi*np.arange(156)/52
          - np.pi/2)
       + rng.normal(0, 0.5, 156)).clip(0)
df = pd.DataFrame({"ds": dates, "y": ili})

# Optional: add Eid holiday
holidays = pd.DataFrame({
   "holiday": "Eid",
   "ds":       pd.to_datetime([
                 "2021-05-13","2022-05-03",
                 "2023-04-22"]),
   "lower_window": 0,
   "upper_window": 3,
})

m = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    changepoint_prior_scale=0.05,
    holidays=holidays)
m.fit(df)
future = m.make_future_dataframe(
    periods=12, freq="W")
forecast = m.predict(future)
m.plot(forecast)
m.plot_components(forecast)
plt.show()
```

Run the skeleton, then refit with the changes below.

```
m = Prophet(yearly_seasonality=True,
            weekly_seasonality=False,
            changepoint_prior_scale=0.05,   # try 0.5
            holidays=holidays,
            interval_width=0.95)            # 95% band
m.fit(df)
fc = m.predict(m.make_future_dataframe(periods=12, freq="W"))
m.plot(fc); m.plot_components(fc)
```

Expected: dates run 2021-01-10 to 2023-12-31 (weeks end Sunday); the forecast covers 2024-01-07 to 2024-03-24 and rises from the yearly low (true curve 2.0) towards about 4.3.

LIVE EXERCISE **1.** Generate 3 years of synthetic weekly ILI data with annual seasonality + noise.

The skeleton does it: 156 weekly dates (156 / 52 = 3 years), a sine with period 52 weeks running from 2 to 8 around a mean of 5, plus noise with SD 0.5. Columns must be named `ds` and `y`.

LIVE EXERCISE **2.** Fit Prophet with yearly and weekly seasonality.

`yearly_seasonality=True` fits the annual Fourier wave (P = 365.25). Keep `weekly_seasonality=False`: with one point per week, always a Sunday, there is no within-week pattern to learn.

LIVE EXERCISE **3.** Plot the forecast with 95% uncertainty intervals.

Pass `interval_width=0.95` (default is 80%), then `m.plot(forecast)`. Expected: data dots, the yhat line and a band from yhat_lower to yhat_upper that widens into the future.

LIVE EXERCISE **4.** Add a holiday for “Eid” (spike in healthcare visits) and observe the effect.

Expected effect is near zero: the synthetic data has no Eid spike, and the 2022 window (05-03 to 05-06) contains no Sunday, so it matches no point. Add +3 to the Eid weeks and the components plot shows an Eid bar near +3.

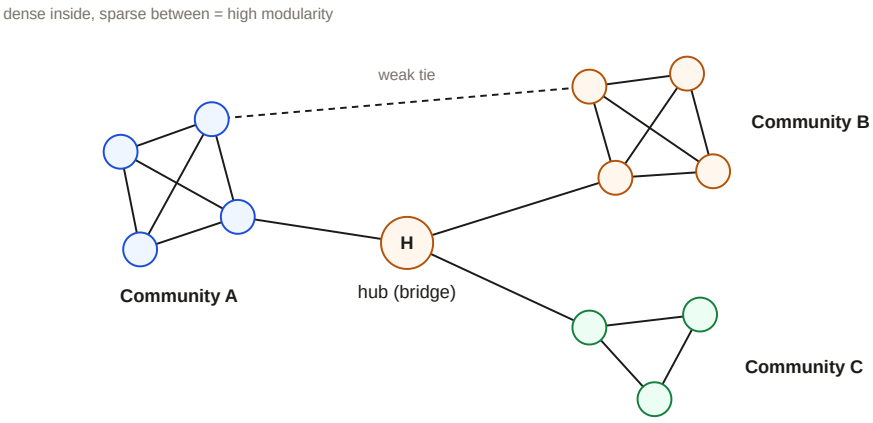
LIVE EXERCISE **5.** Change changepoint_prior_scale from 0.05 to 0.5 and compare trend flexibility.

At 0.05 the trend stays nearly flat around 5 (the true trend). At 0.5 it wiggles, absorbs noise and part of the seasonal wave, and the future band widens. Higher reacts faster to real breaks but overfits stable data.

Topic cards

6 topics · everything the pool tests

Social media as a graph: communities, influence, Twitter's shape



Communities are dense inside and sparsely linked between, which is what Louvain's modularity rewards; a hub bridges them.

FACTS

- Four data dimensions of social media: graph-structured, temporal, text (NLP), multimedia.
- The graph (who follows or retweets whom) is the main object of analysis, not the single post.
- Community: nodes more densely linked to each other than to the rest of the graph. Found with Louvain (Blondel 2008), O(n log n).
- Modularity: fraction of edges inside communities minus the fraction expected by chance. Louvain

HOW IT WORKS

1. Louvain start: every node is its own community
2. Move each node into the neighbour community that gives the biggest modularity gain
3. Collapse each community into one super-node (edges between communities become weighted edges)
4. Repeat the move and collapse steps on the smaller graph

maximises it.

- Influence maximisation: choose k seed users so a message spreads furthest. NP-hard; Kempe's greedy method guarantees (1 − 1/e) ≈ 63% of optimal.
- Kwak 2010: 41.7M users, 1.47B directed edges, 22.1% mutual follows, power law α ≈ 2.276, average shortest path 4.12 hops, 85% of trends are news.
- Kwak 2010 impact: seeding a campaign at high-PageRank users gives measurably different spread from seeding at high-follower users, so follower count is not the same as influence.

WHY · PROS · LIMITS

- Louvain is greedy and local, so it scales to billion-node graphs in Spark GraphX.
- Modularity compares against chance, so a big group is not called a community just for being big.
- Exact influence maximisation is NP-hard; greedy gives a proven 63% floor in reasonable time.
- Twitter is sparse: 1.47B edges among 41.7M users is about 35 follows each, a tiny share of possible links.
- Low reciprocity and fast news trends make Twitter a news medium, which is why health agencies watch it.

5. Stop when no move raises modularity (gain is zero)

COMPARE

	Louvain	PageRank	K-Means
Answers	Which group is a node in?	How important is a node?	Which points are close in space?
Input	Graph links	Graph links	Feature vectors
Output	Community labels	One score per node	Cluster labels
Goal	Maximise modularity	Score by random-walk visit probability (no optimising)	Minimise distance to centroid

	Twitter (Kwak 2010)	Facebook
Edge type	Directed (follow)	Undirected (friendship)
Reciprocity	22.1% mutual	≈100% (both must agree)
Behaves like	News medium	Social network

CALCULATION

Discussion Q1 (P2 slide 15): out-degree d = 45, h = 3 hops, 41.7M users.

Hop	New users
1	45
2	45^2 = 2,025
3	45^3 = 91,125
Total	**93,195** (0.22% of 41.7M)

Step by step

1. Given d = 45 -> hop 1 reaches 45 users
2. Given hop 1 = 45 and d = 45 -> hop 2 = 45 × 45 = 2,025
3. Given hop 2 = 2,025 -> hop 3 = 2,025 × 45 = 91,125
4. Given hops 45, 2,025, 91,125 -> total = 93,195
5. Given total 93,195 and 41.7M users -> share = 93,195 / 41,700,000 = 0.22%

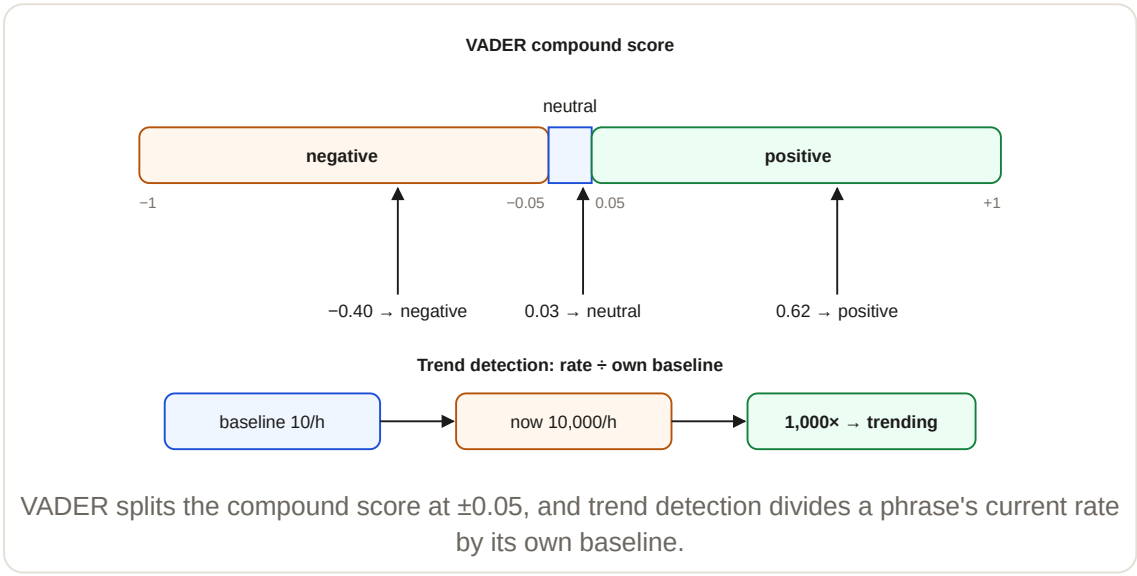
Variants

- d = 30, h = 3: 30 + 900 + 27,000 = 27,930 users.
- d = 40, h = 2: 40 + 1,600 = 1,640 users.
- d = 45, h = 4: adds 45^4 = 4,100,625, total 4,193,820.
- Greedy seeding, optimal reach 10,000: guaranteed at least 10,000 × (1 − 1/e) ≈ 6,321 users.

TRAPS

- PageRank measures importance, not community membership; K-Means clusters vectors, not graph links.
- The hop estimate is an upper bound: it counts shared followers again (clustering, overlapping neighbourhoods).
- Kempe's result: the problem is NP-hard, the greedy answer is approximate (63%), not exact.
- Sparse means few edges per node, not few nodes.

Sentiment (VADER), trend detection and ethics



VADER splits the compound score at ±0.05, and trend detection divides a phrase's current rate by its own baseline.

FACTS

- VADER: Valence Aware Dictionary for Sentiment Reasoning (Hutto & Gilbert 2014), lexicon-and-rule based, tuned for social media text.
- VADER handles capitalisation, punctuation (!!!), degree words (very, kind of), negation (not good) and emojis.
- Compound score in [−1, +1]: > 0.05 positive, < −0.05 negative, otherwise neutral.
- Trend detection: Locality-Sensitive Hashing (LSH) on character n-grams finds phrases whose frequency jumps against their own baseline.
- Slide example: 10 per hour to 10,000 per hour is trending regardless of absolute volume. #QurbaniEid: 1,200× spike in 6 hours, 92% positive.
- Ethical risks: suppressing dissent, microtargeting vulnerable users. Safeguards: transparency and purpose limitation.

HOW IT WORKS

1. Read tweets into a Spark DataFrame (from Kafka)
2. Apply VADER to each tweet with a UDF to get a compound score
3. Label each tweet positive, neutral or negative with the 0.05 cut-offs
4. Aggregate scores by time window and topic
5. In parallel, count each phrase per hour and divide by its historical baseline to flag spikes
6. Push the aggregates to the Grafana streaming dashboard (Week 9)

WHY · PROS · LIMITS

COMPARE

- Rules and a lexicon need no labelled training data, so VADER works out of the box on short, noisy posts.
- Comparing to a phrase's own baseline catches a small topic that suddenly explodes, which raw counts would miss.
- Sentiment on a spike tells organic celebration from coordinated activity (#QurbaniEid was 92% positive).
- Limitation: an English lexicon misreads Bangla, slang and sarcasm, so it needs a Bangla lexicon added.
- The same scores that track public mood can flag critics, so purpose limits are needed.

	Does	Ethical risk at scale
Community detection	Finds densely linked groups	Profiling: exposes political or religious ties
Sentiment analysis	Scores posts positive or negative	Suppressing dissent: flags critics of a policy
Influence maximisation	Picks seed users for spread	Manipulation: spreads misinformation, microtargets the vulnerable
Trend detection	Flags sudden frequency spikes	Amplifies coordinated campaigns if not checked

CALCULATION

P1 slide 12: a phrase goes from 10/hour to 10,000/hour.

Spike ratio = current / baseline = 10,000 / 10 = **1,000×**, so it is trending.

VADER on three posts: 0.62, 0.03, −0.40 → positive, neutral, negative.

Step by step

1. Given baseline 10/hour and current 10,000/hour -> ratio = 10,000 / 10 = 1,000×
2. Given a compound of 0.62 -> 0.62 > 0.05, positive
3. Given a compound of 0.03 -> between −0.05 and 0.05, neutral
4. Given a compound of −0.40 -> < −0.05, negative

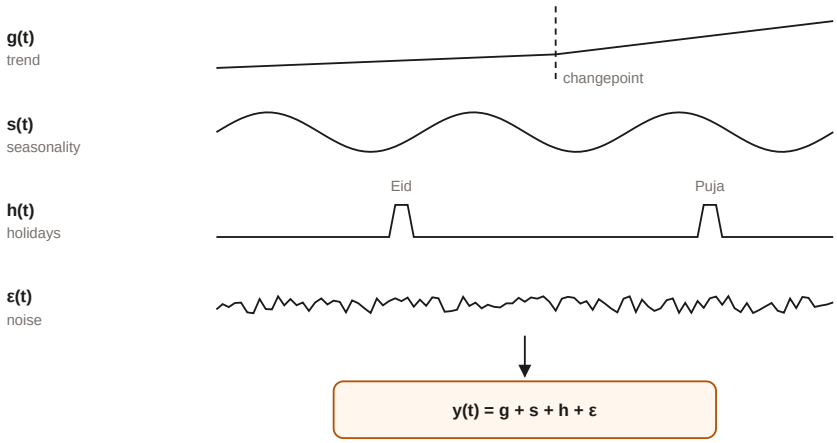
Variants

- #QurbaniEid spike 1,200× on a baseline of 50 posts/hour: peak = 50 × 1,200 = 60,000 posts/hour.
- 20/hour to 6,000/hour: 6,000 / 20 = 300×.
- Compound exactly 0.05: not greater than 0.05, so neutral. −0.06: negative.

TRAPS

- The cut-offs are strict: 0.05 itself is neutral.
- Trending depends on the ratio to baseline, not on absolute volume.
- VADER is rule-based, not a deep-learning classifier.
- LSH here is for detecting trends, not for sentiment.

Time series, ARIMA and Prophet at scale



Prophet adds four parts: a trend with changepoints, Fourier seasonality, listed holidays and noise.

FACTS

- Time series properties: order matters, autocorrelation, seasonality, trend, non-stationarity (mean and variance change over time).
- ARIMA(p, d, q): AR uses past p values, I differences d times for stationarity, MA uses past q forecast errors.
- Prophet (Taylor & Letham 2018): y(t) = g(t) + s(t) + h(t) + ε(t): trend, seasonality, holidays, noise.
- g(t) is piecewise linear or logistic with automatic changepoints; s(t) is a Fourier series, P = 365.25 (yearly) or 7 (weekly).
- changepoint_prior_scale sets trend flexibility: 0.05 default (stiff), 0.5 flexible. Default uncertainty band is 80%.
- Spark: groupBy(sku_id).applyInPandas fits 5,000 Prophet models in parallel: 8 hours to about 12 minutes on 20 executors (40×).

WHY · PROS · LIMITS

- Shuffling a time series destroys trend and seasonality, so standard i.i.d. regression is invalid.
- ARIMA needs stationarity, struggles with several seasonal cycles and needs an expert to pick p, d, q, so it cannot be hand-fitted for 10,000 series.
- Prophet's parts are readable, so analysts (not statisticians) can add holidays and adjust trend flexibility.
- A sparse prior on the slope changes δ keeps most changepoints near zero, which avoids overfitting.
- Each series is fitted independently, so the job is embarrassingly parallel in Spark.

HOW IT WORKS

1. Load all SKU series into one Spark DataFrame, one row per day per SKU
2. Group by sku_id, so each group is one independent series
3. applyInPandas sends each group to one task as a Pandas DataFrame
4. Inside the function, fit Prophet on columns ds and y and predict 90 days ahead
5. All 5,000 models fit in parallel across the cluster
6. Write forecasts to Hive/S3 as Parquet and serve them to the dashboard

COMPARE

	ARIMA	Prophet	LSTM
Kind	Classical statistics	Additive decomposable model	Neural network for sequences
Input	Must be stationary	Handles trend and changepoints	Needs lots of data
Seasonality	Struggles with several	Several at once (Fourier)	Learned implicitly
Holidays	Not built in	h(t) term	Must be fed as features
Tuning	Expert picks p, d, q	Automatic, readable knobs	Heavy, black box

Best for	Short stationary series, small data	Thousands of business series	Long complex series with GPU budget

CALCULATION

P1 slide 19: 5,000 SKUs, 8 hours sequential, about 12 minutes on 20 executors.

Quantity	Working	Result
Speedup	480 min / 12 min	**40×**
Time per model	28,800 s / 5,000	**5.76 s**
Models per executor	5,000 / 20	**250**

Prophet sum: Eid week with g = 100, s = +15, h = +30, ε = −2 gives y = **143**.

Step by step

1. Given 8 hours -> 8 × 60 = 480 minutes
2. Given 480 min sequential and 12 min parallel -> speedup = 480 / 12 = 40×
3. Given 8 h = 28,800 s and 5,000 models -> 5.76 s per model
4. Given 5,000 models and 20 executors -> 250 models per executor
5. Given g = 100, s = 15, h = 30, ε = −2 -> y = 143

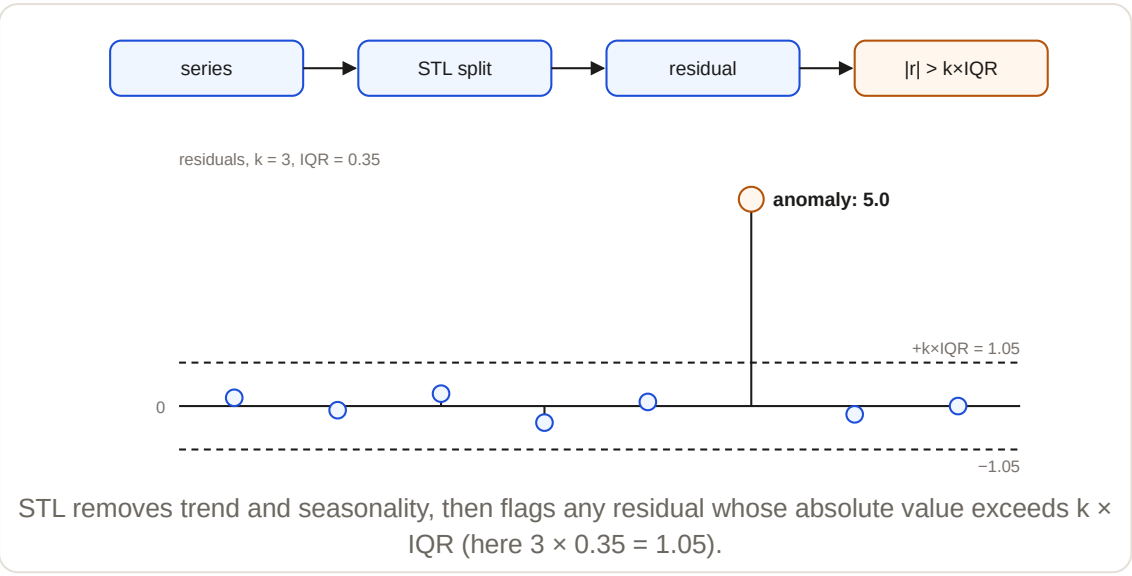
Variants

- 10,000 SKUs at 5.76 s, 40 models at once: sequential 57,600 s = 16 h; parallel 1,440 s = 24 min; speedup still 40×.
- 400 wards at 2 s each, 20 in parallel: 800 s sequential, 40 s parallel, 20×.
- Non-holiday week, y = 120, g = 110, ε = 0: s = +10.
- Changepoint at t = 10: k = 0.5, m = 1, δ = 0.3, γ = −3. g(10) = 6 both sides, g(20) = 0.8 × 20 + 1 − 3 = 14.

TRAPS

- A sudden wave is a trend changepoint, not seasonality; h(t) only covers dates you list in advance.
- Weekly seasonality on weekly data learns nothing: every point is the same weekday.
- Prophet's default interval is 80%; set interval_width=0.95 for 95%.
- ARIMA's d is the number of differences, not the number of seasons.

Anomaly detection: STL and Isolation Forest



STL removes trend and seasonality, then flags any residual whose absolute value exceeds k × IQR (here 3 × 0.35 = 1.05).

FACTS

- Anomaly: a data point that does not conform to the expected pattern (outlier, change point or contextual deviation).
- STL: Seasonal-Trend decomposition using LOESS. Splits a series into trend, seasonal and residual.
- STL rule: flag a point if |residual| > k × IQR of the residuals (IQR = Q3 − Q1).
- Isolation Forest: an ensemble of random trees; an anomaly needs fewer splits to isolate, so its path is shorter.
- Isolation Forest assumes no data distribution and scales to millions of points in Spark MLlib.
- Slide example: bKash agent jumps from 10 to 10,000 transactions per hour, flagged as possible money laundering.

HOW IT WORKS

1. STL: decompose the series into trend, seasonal and residual parts
2. Compute Q1, Q3 and IQR of the residuals
3. Flag points with |residual| above k × IQR
4. Isolation Forest: build many random trees, each splitting on a random feature at a random value
5. Measure how many splits each point needs to be isolated
6. Short average path length means anomaly

WHY · PROS · LIMITS

- Removing trend and seasonality first stops a normal winter peak from being flagged.
- IQR ignores extreme values, so one huge spike does not hide itself by inflating the threshold.
- STL is interpretable and works well when seasonality is regular.
- Isolation Forest needs no distribution assumption and handles many features, such as per-agent volumes.
- Odd points sit alone, so random cuts separate them quickly.

COMPARE

	STL	Isolation Forest
Idea	Large leftover residual	Easy to isolate (short path)
Needs	Regular seasonality in one series	No distribution assumption
Strength	Interpretable	Scales to millions in Spark MLlib
Slide use	Seasonal series	bKash agent volumes

CALCULATION

Residuals: 0.2, −0.1, 0.3, −0.4, 0.1, 5.0, −0.2, 0.0; k = 3.

1. Sorted: −0.4, −0.2, −0.1, 0.0, 0.1, 0.2, 0.3, 5.0

2. Q1 = −0.125, Q3 = 0.225 (inclusive interpolation)
3. IQR = 0.35; threshold = 3 × 0.35 = 1.05
4. Only |5.0| > 1.05: the 6th point is the **anomaly**

Step by step

1. Given the residuals -> sort them
2. Given the sorted list -> Q1 = −0.125, Q3 = 0.225
3. Given Q1 and Q3 -> IQR = 0.225 − (−0.125) = 0.35
4. Given IQR 0.35 and k = 3 -> threshold 1.05
5. Given threshold 1.05 -> compare each |residual|; only 5.0 exceeds it

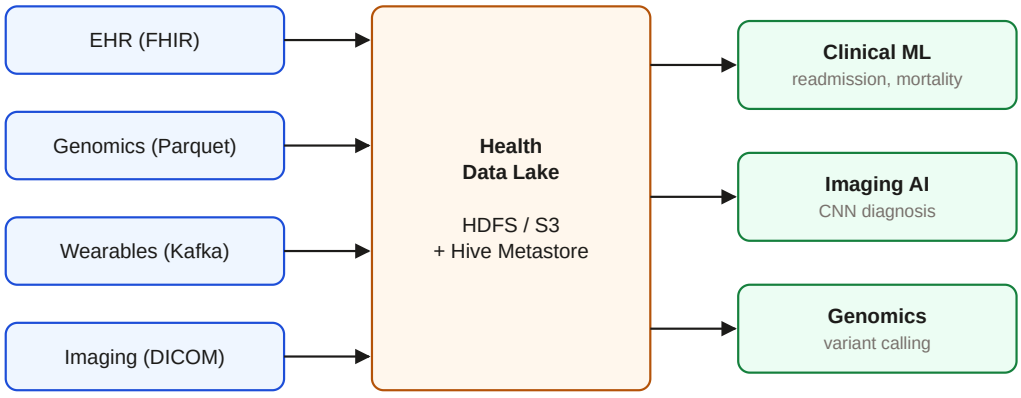
Variants

- Same data, k = 1.5: threshold 0.525; largest normal |−0.4| = 0.4, so still only 5.0 is flagged.
- IQR = 2 and k = 3: threshold 6, a residual of −7 is flagged (absolute value).

TRAPS

- Use the absolute value of the residual: big negative residuals are anomalies too.
- The threshold uses the IQR of residuals, not of the raw series.
- In Isolation Forest, anomalies have shorter paths, not longer.

Healthcare big data, privacy and AlphaFold



Four health data types land in one HDFS/S3 lake with a Hive Metastore and feed clinical, imaging and genomics models.

FACTS

- Four modalities: EHR (ICD-10, notes, FHIR), genomics (≈200 GB per genome; 10,000 patients = 2 PB), wearables (per-second), imaging (DICOM, 200 MB-2 GB, unstructured).
- EHR ML beats physician rules: ICU mortality (random forest, AUROC > 0.90), 30-day readmission (20% fewer), sepsis 6 hours earlier. Needs 1M+ records.
- HIPAA (USA): de-identify 18 PHI identifiers before research; consent. GDPR (EU): minimisation, purpose limitation, right to erasure.

HOW IT WORKS

1. EHR arrives as FHIR resources, genomics as Parquet, wearables through Kafka, images as DICOM
2. All land in a health data lake on HDFS/S3 with a Hive Metastore catalogue
3. De-identify records before research use (remove the 18 identifiers)
4. Train clinical ML models: readmission, mortality, imaging CNNs, variant calling
5. Where records cannot move, train with federated learning; publish aggregates with differential

- PDPA Bangladesh: under development, GDPR-based; NID-linked records need explicit consent.
- Protection: safe harbour, expert determination, differential privacy (calibrated noise on aggregates), federated learning (train at each hospital, data stays).
- AlphaFold 2: 3D protein structure from amino-acid sequence; 170,000 PDB structures, MSAs over 250M UniProt sequences; 200M structures released 2022; 2024 Nobel.

WHY · PROS · LIMITS

- Images are binary DICOM files, so they need deep learning (CNN, ViT), not SQL.
- Reliable clinical models need 1M+ patient records, which forces pooling data across hospitals.
- Federated learning gets that scale without moving raw records, which consent and law forbid.
- Small-area counts can identify one patient; calibrated noise blocks that inference.
- AlphaFold is a big data problem because each query searches 250M sequences to build its MSA.

privacy

COMPARE

	What it does	Protects against
Safe harbour	Removes 18 named identifiers	Direct identification
Expert determination	Statistician certifies low re-identification risk	Indirect identification
Differential privacy	Adds calibrated noise to aggregates	Inferring one person from statistics
Federated learning	Trains at each site, shares only model updates	Moving raw records between hospitals

	Scope	Key rule
HIPAA	US health data	Remove 18 PHI identifiers; consent
GDPR	EU personal data	Minimisation, purpose limitation, erasure
PDPA	Bangladesh, draft	Explicit consent for NID-linked records

CALCULATION

P1 slide 21: 10,000 patients × 200 GB per genome = 2,000,000 GB = **2 PB** (1 PB = 1,000,000 GB).

Step by step

1. Given 200 GB per genome and 10,000 patients -> 2,000,000 GB
2. Given 2,000,000 GB -> divide by 1,000,000 = 2 PB
3. Given 2 PB and HDFS replication 3 -> 6 PB of raw disk

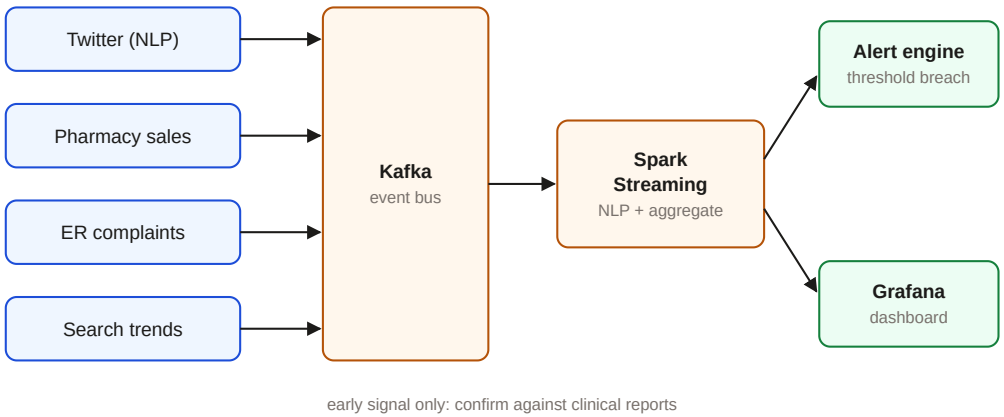
Variants

- 2,500 patients with replication 3: 2,500 × 200 = 500 TB; × 3 = 1.5 PB.
- 5,000 patients: 5,000 × 200 = 1,000,000 GB = 1 PB.

TRAPS

- Stored in a file does not mean structured: DICOM images are unstructured.
- HIPAA is for health data; US financial data falls under GLBA.
- Federated learning moves the model, not the data; differential privacy adds noise, it does not encrypt.
- AlphaFold predicts structure from sequence; it is not anomaly or fraud detection.

Syndromic surveillance and the Google Flu Trends lesson



Syndromic surveillance streams four early signals through Kafka into Spark Streaming, which drives alerts and a Grafana dashboard.

FACTS

- Physician-reported cases reach CDC 1 to 2 weeks after symptoms (ILINet: >3,000 sentinel physicians, weekly).
- Syndromic surveillance uses pre-diagnosis signals: posts peak 7 to 10 days before ER visits; pharmacy antipyretic sales spike 3 to 5 days before confirmation.
- Other signals: ER chief complaints via NLP; search volume (1-week lead).
- BioSense: near real-time data from 3,500+ emergency departments; syndromes extracted automatically.
- Google Flu Trends: $r^2 = 0.97$ on 2003-2008 data, then over-predicted flu by 2× in 2012-2013 and was discontinued.
- Post-GFT: BioSense ED visits as the main signal, Prophet for seasonality and season-start changepoints, Twitter as a covariate, ensembles (FluSight).
- Twitter adds lead time, not accuracy: its signal arrives about 2 weeks before ILINet but with about 30% higher error (latency vs precision).

WHY · PROS · LIMITS

- Outbreaks grow exponentially, so a 1 to 2 week reporting lag is costly.
- Proxy data measures behaviour (health anxiety, media coverage), not incidence, so it drifts.
- GFT failed partly because Google's own autocomplete changes, opaque to CDC, shifted the features.
- Prophet's g + s + h parts let epidemiologists inspect each piece, unlike black-box models.

HOW IT WORKS

1. Sources: Twitter (NLP), pharmacy sales, ER chief complaints, search trends
2. All publish events to a Kafka event bus
3. Spark Streaming runs NLP and aggregates counts per region and time window
4. An alert engine fires when a count breaches its threshold
5. A Grafana dashboard shows the signals to health officers
6. Officers confirm against clinical reporting before acting

COMPARE

	Traditional (ILINet)	Syndromic
Source	Physician-reported cases	Posts, pharmacy, ER notes, searches
Speed	1 to 2 weeks behind	Days ahead
Accuracy	Ground truth	Noisy proxy
Role	Main record	Early warning, a supplement

- High in-sample r^2 does not guarantee out-of-sample accuracy when behaviour changes.

TRAPS

- GFT did not replace CDC surveillance; it failed and was discontinued.
- Syndromic signals are early warnings, not ground truth.
- Twitter enters the post-GFT model as a covariate, not as the main signal.
- $r^2 = 0.97$ was in-sample; it says nothing about future seasons.

PRACTICE

Mock final paper

One 10-mark question per chapter, in the shape of the October 2026 Data Mining final from the same department: a short scenario, then short parts. Answer either the main pair or the OR pair. Click a part for its answer.

Q1 · Week 5 10 marks

A · 4 MARKS

A bank keeps 20 TB of transaction logs with 50 columns as gzip-compressed CSV on HDFS. A fraud report reads 5 columns.
i) Why does each file get only one Mapper? ii) Which format and codec should replace it? iii) How much data would the report read after the switch? iv) What in the new format lets it skip whole blocks of rows?

i) Gzip is not splittable, so reading must start at the file's beginning. ii) Parquet + Snappy (or ORC for Hive). iii) 20 TB × 5/50 = **2 TB**, a 90% cut. iv) Min/max stats per column chunk in the footer: predicate pushdown.

B · 6 MARKS

A cluster of 1,000 nodes, each 128 GB RAM and 32 vCPU, runs YARN with queues analytics 50%, streaming 30%, adhoc 20%. Analytics runs Spark executors {8 GB, 4 vCPU}.
i) Which YARN component grants the containers? ii) Find the maximum executors in the analytics queue and the bottleneck. iii) A node running 6 executors dies: who detects it and who asks for replacements? iv) Which scheduler fits these three teams, and why?

i) The ResourceManager's Scheduler. ii) 500 nodes; RAM 64,000 GB / 8 = 8,000; vCPU 16,000 / 4 = 4,000; max **4,000**, CPU bottleneck. iii) The RM detects missed heartbeats (>10 min) and marks the 6 containers failed; the Spark ApplicationMaster requests 6 replacements. iv) Capacity: each team gets a guaranteed share and can borrow idle capacity.

OR

A · 4 MARKS

A company runs a 5,000-node Hadoop 1.x cluster and wants to add Spark jobs.
i) Name two limits of the JobTracker. ii) What did Hadoop 2.0 split it into? iii) How many ApplicationMasters run for 12 active jobs? iv) What is a container?

i) MapReduce only; about 4,000-node ceiling (5,000 is past it). ii) A global ResourceManager and a per-job ApplicationMaster. iii) 12, one per job. iv) A slice of one node: set vcores plus set RAM, in which a task runs.

B · 6 MARKS

A ride-sharing app streams trip events through Kafka; the schema gains a field each month. Each night a Spark job builds reports; 10 GB of the day's data is written per night.
i) Which format at ingest, and why? ii) What happens when a new reader opens an old file missing the new field? iii) Which format and codec for the nightly analytics copy? iv) How many Mappers can read the 10 GB night file with 128 MB row groups, and how many if it were one gzip CSV?

i) Avro: row-oriented and append-friendly, with schema evolution. ii) The missing field gets its default value from the reader schema. iii) Parquet + Snappy. iv) 10,240 / 128 = **80** Mappers; gzip CSV: **1**.

Q2 · Week 6 10 marks

A · 4 MARKS

A ride-sharing firm runs a PySpark job on its Hadoop cluster: `textFile → filter(fare > 0) → map(to (zone, fare)) → reduceByKey(sum) → saveAsTextFile`.
i) Which component builds the DAG and schedules tasks? ii) How many stages run, and where is the boundary? iii) Which call starts the computation, and why? iv) Why is reduceByKey better here than groupByKey?

i) The Driver (YARN is the Cluster Manager, Executors run the tasks). ii) Two: textFile → filter → map are narrow and pipelined in Stage 1; reduceByKey is wide, so the shuffle starts Stage 2. iii) saveAsTextFile, the only action; transformations are lazy. iv) It sums fares per zone inside each partition before the shuffle, so far fewer records cross the network.

B · 6 MARKS

The firm trains a demand model with 20 iterations over 40 GB. HDFS reads and writes at 400 MB/s; RAM cache reads at 4,000 MB/s.
i) Total HDFS I/O for MapReduce. ii) Spark's HDFS I/O and the reduction factor. iii) Wall-clock time for each. iv) The speedup, and one case where Spark would not win.

i) 20 × (40 + 40) = 1,600 GB = 1.6 TB. ii) 40 GB (read once); 1,600 / 40 = 40× less. iii) HDFS pass = 40,000 / 400 = 100 s, so T_MR = 20 × 200 = 4,000 s ≈ 66.7 min. RAM pass = 40,000 / 4,000 = 10 s, so T_Spark = 100 + 19 × 10 = 290 s ≈ 4.8 min. iv)

4,000 / 290 ≈ **13.8×**. Spark would not win on a single-pass job, or if the 40 GB did not fit in cluster RAM.

OR

A · 4 MARKS A job runs `input → filter → map` over 8 partitions and caches the result. The Executor holding partition 4 crashes.
i) What does “resilient” mean for this RDD? ii) List the recovery steps. iii) What fraction of the work is redone? iv) What changes if the next step is groupByKey?

i) Lost partitions are recomputed from lineage, not read from a replica. ii) Detect the lost partition → trace lineage to the input → re-read input partition 4 from HDFS → re-apply filter then map. iii) 1 / 8 = 12.5%. iv) groupByKey is wide: a lost output partition needs data from all 8 parent partitions, so recovery can rerun the whole earlier stage.

B · 6 MARKS A word count runs on 6 partitions. Each partition holds 2,000 `(word, 1)` pairs with 40 distinct words.
i) Records shuffled by groupByKey. ii) Maximum records shuffled by reduceByKey. iii) Percentage reduction. iv) With numPartitions = 5, which partitions do keys with hash 23 and 40 go to, and is this step narrow or wide?

i) 6 × 2,000 = 12,000. ii) 6 × 40 = 240. iii) 1 − 240 / 12,000 = 0.98 → **98%** fewer records. iv) 23 % 5 = 3 and 40 % 5 = 0, so partitions 3 and 0. Hash partitioning for a shuffle is a wide dependency.

Q3 · Week 7 10 marks

A · 4 MARKS A food-delivery company stores 4 TB of orders in one Hive table. Managers run a daily report filtered by order_date and city, and analysts join orders with a 15 M-row riders table on rider_id. Today every report scans the whole table.
i) Which feature lets the daily report read only one folder? ii) Name the columns you would use for it. iii) Why not use rider_id for that feature? iv) What should be done to rider_id instead?

i) Partitioning (partition pruning). ii) order_date and city. iii) High cardinality: millions of directories would overload the NameNode. iv) Bucket both tables on rider_id with the same number of buckets for a bucket map join.

B · 6 MARKS For the table in (a): i) The table is partitioned by 730 days and 20 cities. What fraction does a one-day, one-city report read? ii) With 4 TB and 8 buckets per partition, what is the size of each bucket file? iii) Which bucket holds rider_id 1,000,003? iv) Write the CREATE TABLE line that declares the partitions and buckets.

i) P = 730 × 20 = 14,600; 1 / 14,600 ≈ 0.0068%. ii) 4,000 GB / (14,600 × 8) = 4,000 / 116,800 ≈ 34 MB (small; fewer buckets, e.g. 2, give ≈ 137 MB). iii) 1,000,003 % 8 = 3. iv) `PARTITIONED BY (order_date STRING, city STRING) CLUSTERED BY (rider_id) INTO 8 BUCKETS STORED AS ORC`.

OR

A · 4 MARKS A telecom's analysts send HiveQL from Tableau. A query `SELECT region, COUNT(*) FROM calls GROUP BY region` takes 9 minutes.
i) Which component receives the query? ii) Where does Hive find the table's HDFS path? iii) What does the map phase emit? iv) Why is it slow on MapReduce?

i) HiveServer2. ii) The Metastore. iii) (region, 1), or partial counts per region with map-side aggregation. iv) MapReduce writes intermediate results to HDFS between stages.

B · 6 MARKS i) The calls table has 60 million rows and 300 regions; one mapper uses map-side aggregation. By what factor does the shuffle shrink? ii) The team moves the dashboard to Presto and the query drops from 9 minutes to 27 seconds. Find the speed-up. iii) Give two Presto design decisions behind it. iv) Does the team need to redefine its tables for Presto? Why?

i) 60,000,000 / 300 = 200,000×. ii) 540 s / 27 s = 20×. iii) No intermediate disk writes (in-memory pipeline); vectorized execution on 1,024-row batches (also: cost-based optimizer). iv) No: Presto reads the same Hive Metastore.

Q4 · Week 8 10 marks

A · 4 MARKS A ride-sharing firm streams driver GPS pings every 3 seconds to a Kafka topic with 10 partitions, keyed by driver_id, replication factor 3. The routing engine and the billing service both read the topic.
i) Why key by driver_id? ii) How do routing and billing each get every ping? iii) How many broker failures can the topic survive? iv) Routing scales to 14 consumers in one group. How many sit idle?

i) Same key → same partition, so each driver's pings stay in order. ii) Each is its own consumer group. iii) R − 1 = 2. iv) 14 − 10 = 4 idle.

B · 6 MARKS The same firm loads its 5 M-row trips table from MySQL into HDFS nightly and also wants a fraud check on payments within 1 second. Name the tool for each, choose Lambda or Kappa for the whole system, and give one weakness of your choice.

Trips table: Sqoop (incremental import by last-value, parallel map tasks, retry on failure). Payments: Kafka topic + Spark Streaming (milliseconds, RF for fault tolerance, replay from offset). Architecture: Lambda, because a real nightly batch path exists (batch = Sqoop + HDFS + Hive; speed = Kafka + Spark Streaming; serving merges). Weakness: two code paths for shared logic, 30–40% of effort to keep in sync; reduce it by making Kafka the single source both layers read.

OR

A · 4 MARKS A hospital stores five years of lab CSVs, scanned reports and device logs in S3 with no catalogue. Analysts find four versions of the same table, and patient IDs are open to every user.
i) Lake or warehouse? ii) What has it become? iii) Name the layer that is missing. iv) Give two fixes.

i) Data lake. ii) A data swamp. iii) Catalogue/Governance layer. iv) Catalogue with an owner per dataset; access control on PII (also quality checks, Iceberg/Delta Lake versioning).

B · 6 MARKS The hospital's team runs Lambda: nightly Spark jobs plus a Flink stream for ICU alerts. The CTO wants Kappa. State what must be true of the stream engine, one risk of migrating, and one reason Kappa may not suit them. Show how Kappa reprocesses history.

Must be true: the engine replays history about as fast as it handles live data, and full history is kept in a replayable log (Kafka long retention or Iceberg on S3). Risk: replaying TB+ of records may take days, and all batch jobs must be rewritten. Not suitable if: health regulators require a separate batch reconciliation, or batch SQL is preferred. Reprocessing: deploy new job version → start at offset 0 → catch up to live → swap in, retire the old job.

Q5 · Week 9 10 marks

A · 4 MARKS A ride-hailing company in Chattogram shows a live map of 2 million driver GPS pings. The scatter is a solid blob. A filter on the 80-billion-row trip table in Hive takes minutes.
i) Name the problem with the map. ii) Which fix shows the count of pings per area? iii) Name the problem with the filter. iv) Which fix gives sub-second filters, and its trade-off?

i) Overplotting. ii) Hexbin aggregation. iii) Latency. iv) A pre-aggregated data cube; data can be up to 24 h stale.

B · 6 MARKS The same company's report shows trips per district as 3-D bars with a gradient, a one-series legend, and a y-axis starting at 900. Trips rose from 1,000 to 1,050, but the bar grew from 100 px to 150 px.
i) Name two chartjunk elements. ii) Compute the effect in the data. iii) Compute the effect shown. iv) Compute the lie factor and say what it means.

i) 3-D bars, gradient fill (also the single-series legend). ii) (1,050 − 1,000) ÷ 1,000 = **5%**. iii) (150 − 100) ÷ 100 = **50%**. iv) 50 ÷ 5 = **10**: the chart overstates the rise ten times; start y at 0.

OR

A · 4 MARKS A news site builds a live election dashboard. It opens on a national map, has a party dropdown, and shows a constituency's full result on click.
i) Which design rule does this follow? ii) What is the dropdown? iii) What is the click? iv) Data type of constituency results on a map?

i) Shneiderman's mantra. ii) Zoom and filter (dynamic query). iii) Details on demand. iv) 2-D (map).

B · 6 MARKS

Votes arrive at 2,000 events per second through Kafka. Results must refresh in under a second on the public page, and every 5 seconds on the operations screen.
i) Events in a 1-minute tumbling window? ii) Events between two 5 s refreshes? iii) Tool for the ops screen, and why? iv) Delivery method for the public page?

i) 2,000 × 60 = **120,000**. ii) 2,000 × 5 = **10,000**. iii) Grafana: reads the time-series DB natively and auto-refreshes every 5 s. iv) WebSocket push to a D3.js chart.

Q6 · Week 10 10 marks

A · 4 MARKS

Dhaka's health office streams Bangla Facebook posts and pharmacy paracetamol sales into a cluster to catch dengue early.
i) Which tool should carry both event streams? (1)
ii) A dengue phrase rises from 20 to 6,000 posts an hour. What is the spike ratio? (1)
iii) A post scores a VADER compound of 0.04. Which class? (1)
iv) Why must this system not replace DGHS reports? (1)

i) Kafka. ii) 6,000 / 20 = **300×**. iii) Neutral (between −0.05 and 0.05). iv) Proxy signals reflect behaviour, not cases; Google Flu Trends over-predicted by 2×. They supplement clinical reporting.

B · 6 MARKS

A hospital group forecasts weekly admissions for 400 wards with Prophet.
i) Write the Prophet model and name each term. (2)
ii) In an Eid week, g = 80, s = +6, h = +12, ε = 0. Forecast? (1)
iii) Each ward's model takes 2 s; Spark fits 20 at a time. Sequential time, parallel time? (1)
iv) The model missed a sudden COVID wave. Which term should catch it, and which hyperparameter controls that? (2)

i) $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$: trend, seasonality, holidays, noise. ii) 80 + 6 + 12 + 0 = **98**. iii) 400 × 2 = **800 s**; 800 / 20 = **40 s** (20×). iv) Trend g(t) via changepoints; `changepoint_prior_scale` (raise from 0.05 to bend faster).

OR

A · 4 MARKS

DGHS wants one sepsis model from 30 hospitals and a public dashboard of cases by upazila.
i) How can it train without pooling records? (1)
ii) How can it stop small counts revealing a patient? (1)
iii) Which US law requires removing 18 identifiers? (1)
iv) A linked genomics study has 5,000 patients at 200 GB each. Storage in PB? (1)

i) Federated learning. ii) Differential privacy. iii) HIPAA. iv) 5,000 × 200 = 1,000,000 GB = **1 PB**.

B · 6 MARKS

A researcher studies a misinformation cascade on X.
i) Give the steps of Louvain for finding the groups that spread it. (2)
ii) Average out-degree is 40. Upper bound on users reached in 2 hops? (1)
iii) Why is that an overestimate? (1)
iv) Kwak et al. found 22.1% reciprocity. What does that say about Twitter compared with Facebook? (2)

i) Each node alone; move nodes to the neighbour community with the biggest modularity gain; collapse to super-nodes; repeat until no gain. ii) 40 + 1,600 = **1,640**. iii) Followers overlap (clustering), so people are counted twice, and not everyone reshares. iv) Most follows are one-way, so Twitter is a directed news medium; Facebook friendship is mutual (≈100%), a social network.

LAST TEN MINUTES

Formula and fact sheet

Formulas

What	Formula	Note
W5 · Columnar data read	`Read = D × k / N`	D table size, k columns selected, N total columns
W5 · Columnar I/O reduction	`Reduction = 1 - k / N; speedup ≈ N / k`	10 TB, 3 of 100: 97%, 33×
W5 · Pushdown saving	`Speedup = D / R`	R = bytes actually read; 10 TB / 50 GB = 200×
W5 · Mappers per file	`Splits = ⌈F / B⌉, or 1 if gzip`	B = block or row group, about 128 MB
W5 · Containers per queue	`min(nodes·f·RAM / c_ram, nodes·f·CPU / c_cpu)`	smaller term is the bottleneck
W5 · Nodes for a job	`⌈T / min(RAM / c_ram, CPU / c_cpu)⌉`	T containers needed at once
W5 · Extra work from utilisation	`U_new / U_old - 1`	78/45 − 1 = 73%
W6 · MapReduce iterative disk I/O	`MR_IO = k × (D_read + D_write)`	k iterations; slide: 100 × (50 + 50) GB = 10 TB
W6 · Spark HDFS I/O	`Spark_IO = D (read once)`	later iterations read RAM, not HDFS
W6 · Transfer time	`t = size / speed`	50,000 MB / 500 MB/s = 100 s
W6 · Iterative wall-clock time	`T_MR = k × (D/R_hdfs + D/W_hdfs); T_Spark = D/R_hdfs + (k - 1) × D/R_ram`	k − 1 RAM passes, not k
W6 · Speedup	`Speedup = T_old / T_new`	A1: 20,000 / 1,090 ≈ 18.3×; RDD paper 127 / 0.96 ≈ 132×
W6 · Hash partitioner	`partition = key.hashCode() % numPartitions`	17 % 4 = 1
W6 · Shuffle records	`groupByKey = all pairs; reduceByKey ≤ partitions × distinct keys per partition`	4,000 vs 200 records
W6 · Efficiency per node	`(T_old × N_old) / (T_new × N_new)`	GraySort (72 × 2,100) / (23 × 206) ≈ 32×
W7 · Map-side aggregation	`reduction = N_rows / G (with M mappers: N_rows / (M × G))`	50M rows, 200 countries: 250,000×
W7 · Partition pruning	`fraction read = P_read / P_total`	P_total = product of values per partition column
W7 · Bucket number	`bucket = hash(column) % N`	integer key: hash = value; 98 % 4 = 2
W7 · Bucket file size	`size = S / (P × N); files = P × N`	aim near the 128 MB HDFS block
W7 · ALS model size	`(users + items) × k × bytes`	(75M + 30M) × 50 × 4 B = 21 GB
W7 · Speed-up	`T_old / T_new; saved = (T_old - T_new) / T_old`	8 h to 2 h: 4×, 75% saved
W7 · Pairs in a playlist	`L(L - 1) / 2`	20 songs: 190 pairs
W8 · Point-to-point ETL jobs	`N × (N - 1); bus = N`	50 systems: 2,450 jobs vs 50 connections
W8 · Partition copies	`P × R`	bkash-txns: 3 × 2 = 6
W8 · Broker failures survived	`R - 1`	default R = 3 survives 2

W8 · Active consumers in a group	`min(P, C); idle = C - P (only if C > P)`	one consumer per partition per group
W8 · Messages per second	`MB/s × 1,000,000 ÷ message bytes`	50.5 MB/s, 100 B = 505,000 msgs/s
W8 · Speedup	`Kafka ÷ other`	50.5 / 4.3 ≈ 11.7×; 50.5 / 14.7 ≈ 3.4×
W8 · Sequential vs random disk	`seq ÷ random`	600 / 100 = 6× (paper figures)
W8 · Zero-copy	`4 copies → 2 copies`	about 2× throughput, CPU near zero
W8 · Events per second	`events per day ÷ 86,400`	Netflix 100 B/day ≈ 1.16 M/s
W9 · Data-ink ratio	`ink encoding data ÷ total ink`	Maximise it; 1 is the ceiling
W9 · Lie factor	`effect shown in graphic ÷ effect in data`	1 truthful; above 1 overstates; below 1 understates
W9 · Hexbin reduction	`points ÷ hex cells`	10 M ÷ 10,000 = 1,000×
W9 · Alpha stacking	`opacity = 1 - (1 - α)^n`	α = 0.02: n = 100 → 0.867, saturates
W9 · Cube rows	`≤ d₁ × d₂ × … × dₖ; reduction = fact rows ÷ cube rows`	100 B ÷ 5 M = 20,000×
W9 · Group-by combinations	`2^k for k dimensions`	k = 4 → 16
W9 · Events per window	`rate × window seconds`	10,000/s × 60 = 600,000
W9 · Updates per day	`86,400 ÷ refresh seconds`	Every 5 min → 288
W10 · Prophet model	`y(t) = g(t) + s(t) + h(t) + ε(t)`	trend + seasonality + holidays + noise
W10 · Prophet trend	`g(t) = (k + a(t)^Tδ) t + (m + a(t)^Tγ)`	δ slope changes at changepoints, sparse prior
W10 · Fourier seasonality	`s(t) = Σ [aₙ cos(2πnt/P) + bₙ sin(2πnt/P)], n = 1..N`	P = 365.25 yearly, 7 weekly
W10 · ARIMA(p, d, q)	`AR past p values; I difference d times; MA past q errors`	needs stationary input
W10 · Speedup	`T_sequential / T_parallel`	480 min / 12 min = 40× (5,000 SKUs)
W10 · Reach in h hops	`d + d² + … + d^h`	upper bound; d = 45, h = 3 gives 93,195
W10 · Greedy influence bound	`spread ≥ (1 - 1/e) × optimal ≈ 0.632 × optimal`	Kempe et al. 2003
W10 · Modularity (words)	`fraction of edges inside communities - fraction expected by chance`	Louvain maximises it
W10 · Power law	`P(k) ∝ k^(-α), α ≈ 2.276`	Twitter follower counts (Kwak 2010)
W10 · Spike ratio	`current rate / baseline rate`	10 to 10,000 per hour = 1,000×
W10 · VADER classes	`> 0.05 positive; < -0.05 negative; else neutral`	compound in [−1, +1]
W10 · STL anomaly rule	`|residual| > k × IQR, IQR = Q3 - Q1`	IQR of residuals, not raw data
W10 · Genomics storage	`200 GB × patients (× replication)`	10,000 patients = 2 PB

Every topic in one line

Wk	Topic	One-line answer
W5	**YARN and its three components**	YARN is Hadoop 2's resource manager: one ResourceManager gives out containers, a NodeManager runs them on each node, and one ApplicationMaster manages each job.
W5	**YARN fault tolerance**	When a NodeManager stops heartbeating, the RM marks its containers failed and the AM requests replacements on other nodes.
W5	**Schedulers and capacity planning**	The Capacity Scheduler splits the cluster into queues with guaranteed shares and lets queues borrow idle capacity.
W5	**Row vs columnar storage and Parquet**	Columnar storage keeps each column's values together on disk, so a query reads only the columns it needs.
W5	**File formats: Avro, Parquet, ORC**	Avro is a row format built for schema evolution; Parquet and ORC are columnar formats built for analytics.
W5	**Compression codecs and splittability**	A splittable file can be read from any point, so multiple Mappers process different parts at the same time.
W6	**Why Spark: the MapReduce iteration bottleneck**	MapReduce writes every intermediate result to HDFS, so iterative algorithms pay disk I/O on every pass; Spark caches the data in RAM once.
W6	**Spark architecture**	Spark has three tiers: a Driver that builds the DAG and schedules tasks, a Cluster Manager that allocates resources, and Executors that run tasks and cache data.
W6	**RDDs and lineage-based fault tolerance**	An RDD is a read-only, partitioned collection of records that Spark rebuilds from its lineage if a partition is lost.
W6	**Transformations, actions and lazy evaluation**	Lazy evaluation means Spark records transformations in a DAG and runs nothing until an action is called.
W6	**Narrow vs wide dependencies, stages and shuffle**	A wide dependency is one where an output partition needs data from many input partitions, which forces a shuffle and a new stage.
W6	**DataFrames, Spark SQL and the ecosystem**	A DataFrame is a distributed table with named columns and a schema, optimised by Catalyst.
W7	**Hive architecture and query compilation**	Hive is a data warehouse layer on Hadoop that compiles SQL-like HiveQL queries into MapReduce, Tez or Spark jobs over files on HDFS.
W7	**Schema-on-read and HiveQL vs SQL**	Schema-on-read means Hive applies the table's column types when data is queried, not when it is loaded.
W7	**Partitioning and partition pruning**	Partitioning stores a Hive table in separate HDFS sub-directories by the value of a column, so queries filtering on it read only matching directories.
W7	**Bucketing and bucket map join**	Bucketing splits each Hive partition into a fixed number of files by hash(column) % N.
W7	**Presto and the SQL-on-Hadoop engines**	Presto is Meta's in-memory SQL engine that answers interactive queries over HDFS in seconds by never writing intermediate results to disk.
W7	**Case study: Spotify Discover Weekly**	Discover Weekly is Spotify's weekly recommendation playlist, built from playlist co-occurrence in HiveQL and ALS collaborative filtering in Spark, scheduled by Luigi.
W8	**From point-to-point ETL to event pipelines**	Point-to-point ETL connects every pair of systems with its own job, giving $O(N^2)$ jobs.
W8	**Batch ingestion: Sqoop and Flume**	Sqoop is a batch tool that copies tables in bulk between an RDBMS and HDFS.
W8	**Kafka: the append-only log**	A Kafka offset is the integer position of a message inside a partition.
W8	**Why Kafka is fast**	Zero-copy uses sendfile() to send data from disk to the network without copying it through user space.
W8	**Lambda and Kappa architectures**	Kappa architecture drops Lambda's batch layer and reprocesses history by replaying the Kafka log.
W8	**Data lakes and the data swamp**	A data swamp is a data lake with no governance: files without metadata, owners or quality checks.
W9	**Tufte: data-ink, chartjunk, lie factor**	The data-ink ratio is the share of a chart's ink that encodes data, and Tufte says to maximise it.
W9	**Shneiderman's mantra and choosing the chart**	Shneiderman's mantra is "overview first, zoom and filter, then details on demand".

W9	**Overplotting: alpha, hexbin, KDE**	Overplotting is when millions of scatter points overlap into a solid blob that shows nothing.
W9	**Latency: data cubes and AQP**	A data cube is a set of pre-computed aggregate tables over combinations of dimensions, read instead of the raw fact table.
W9	**Streaming dashboards and tools**	Grafana is the tool for real-time ops dashboards that poll a time-series database every few seconds.
W9	**Case study: NYT COVID-19 tracker**	The NYT COVID tracker showed that cleaning data from 50 sources was harder than drawing the charts.
W10	**Social media as a graph: communities, influence, Twitter's shape**	Community detection finds groups of nodes more densely linked to each other than to the rest of the graph, usually with the Louvain algorithm.
W10	**Sentiment (VADER), trend detection and ethics**	VADER is a lexicon-and-rule-based sentiment analyser for social media that outputs a compound score in [−1, +1].
W10	**Time series, ARIMA and Prophet at scale**	Prophet is a time series forecasting model that splits a series into trend, seasonality, holidays and noise: $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$.
W10	**Anomaly detection: STL and Isolation Forest**	An anomaly is a data point that does not conform to the expected pattern of the series.
W10	**Healthcare big data, privacy and AlphaFold**	Federated learning trains a model at each hospital and shares only model updates, so raw patient records never leave the site.
W10	**Syndromic surveillance and the Google Flu Trends lesson**	Syndromic surveillance uses non-traditional signals, such as posts, pharmacy sales and ER notes, that show illness before a diagnosis is reported.